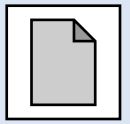
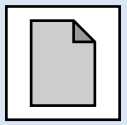
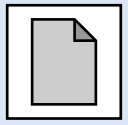
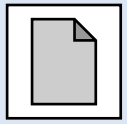
铁头山羊
TIETOUSHANYANG

第六章 I2C总线

第一节、I2C通信协议 

第二节、I2C模块的使用方法 

第三节、OLED显示器 

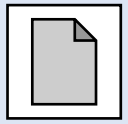
第四节、MPU6050 

第一节、I2C通信协议

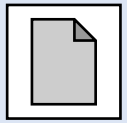
1.1. I2C简介

铁头山羊 TIETOUSHANYANG

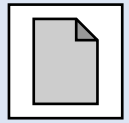
1.1.1. 串口通信的缺点



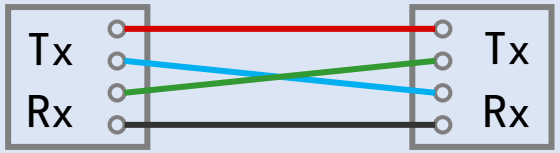
1.1.2. 总线的概念



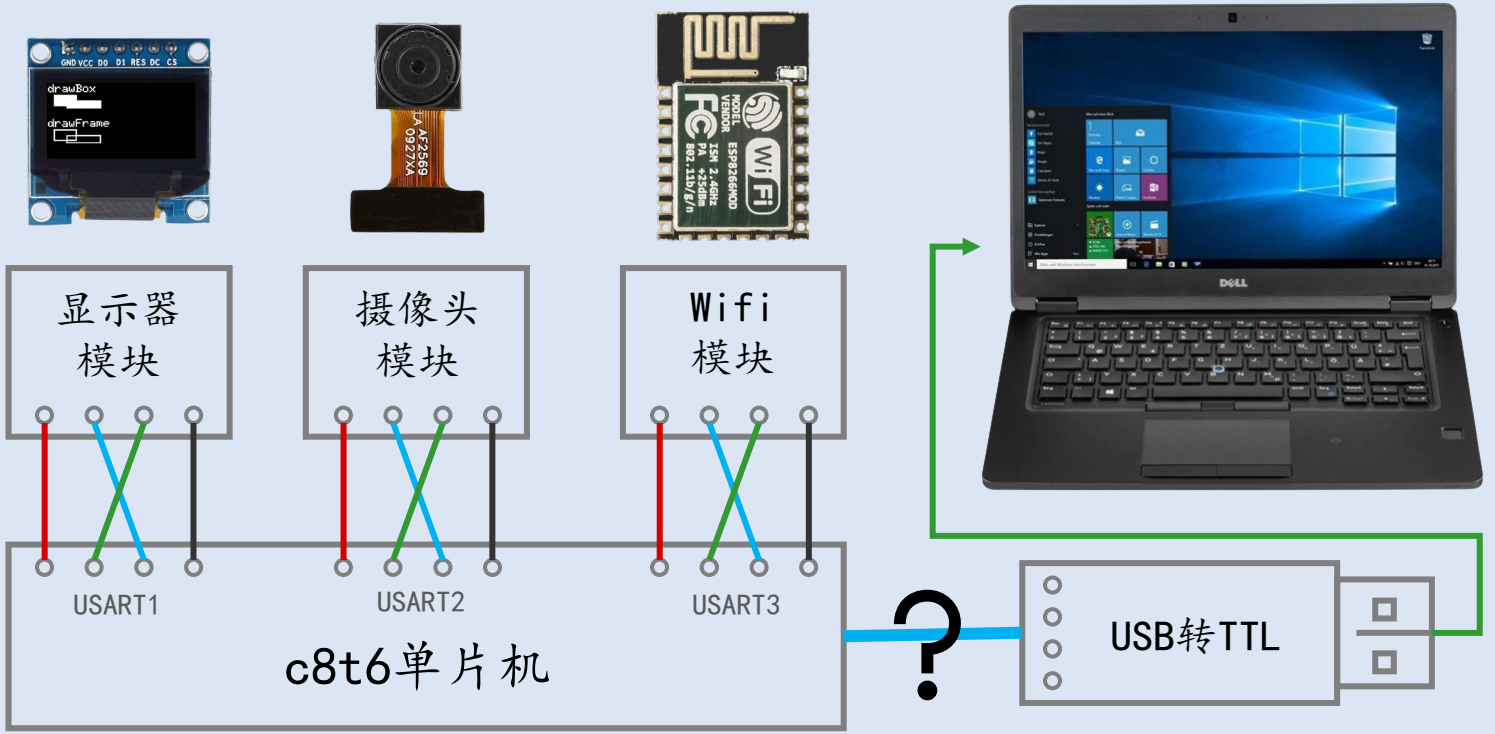
1.1.3. I2C总线



1.1.1. 串口通信的缺点



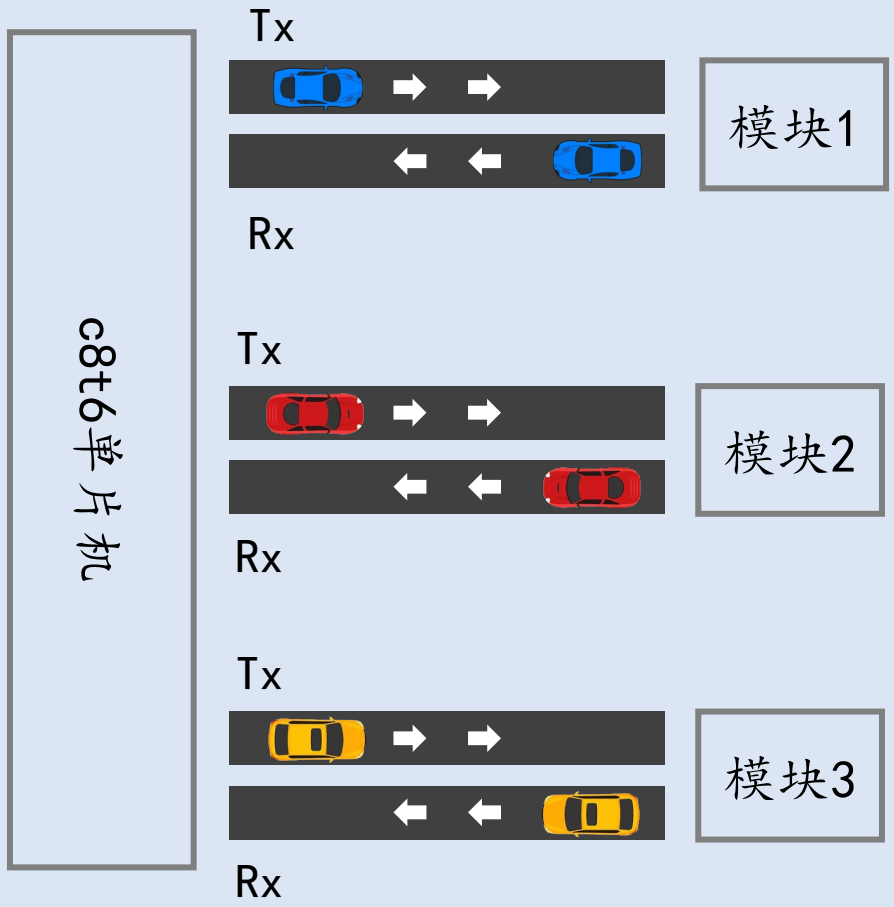
USART 全双工 波特率可调



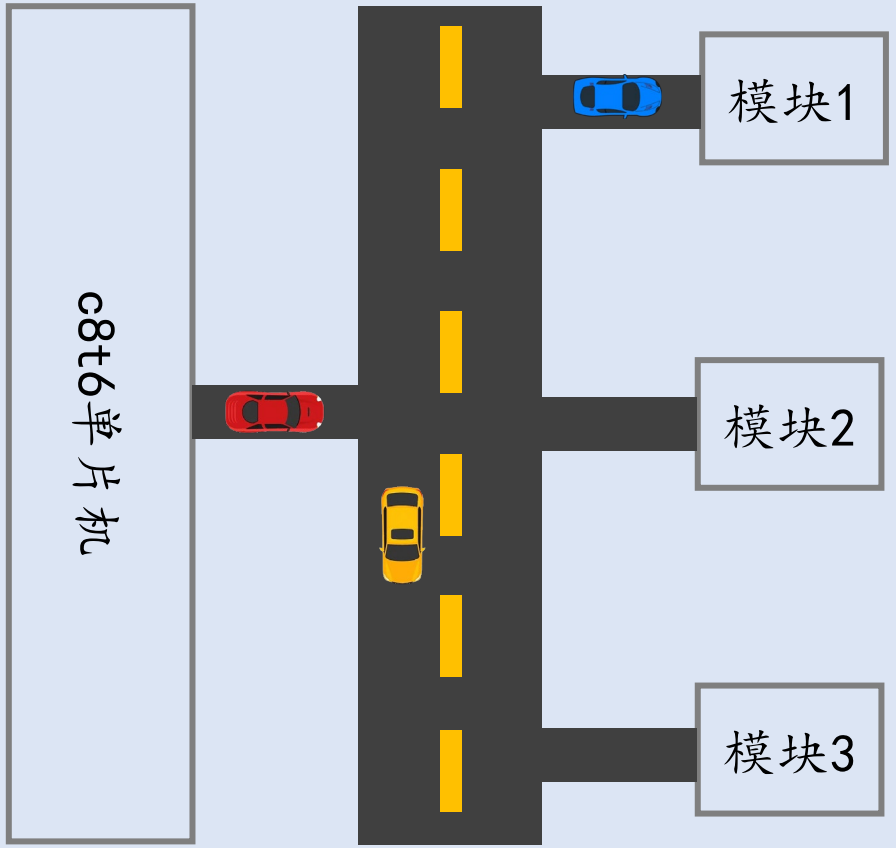
1.1.2. 总线的概念

铁头山羊 TIETOUSHANYANG

非总线（独占通信链路）

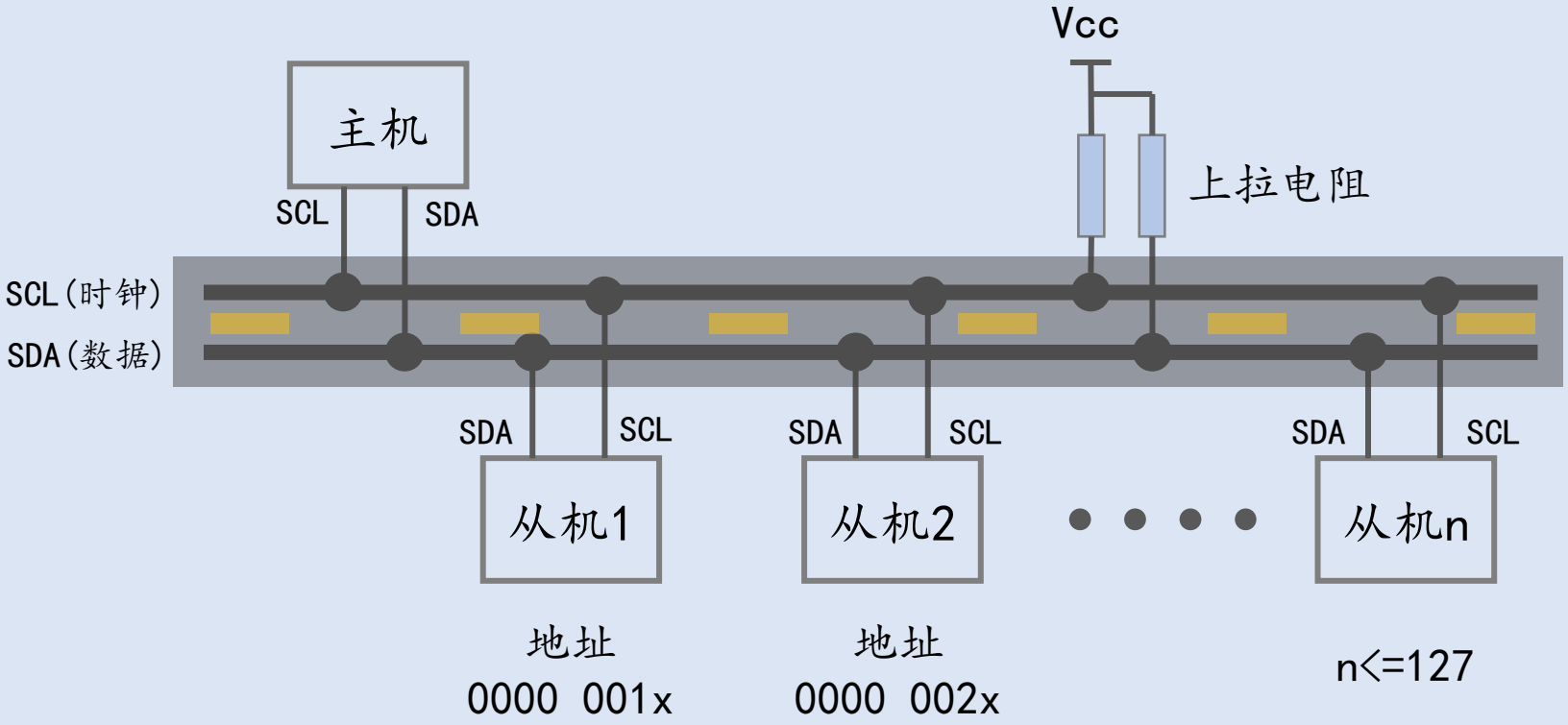


总线（共享通信链路）



1.1.3. I2C总线

I2C(IIC - Inter-Integrated Circuit 内部集成电路)



1.2. 主机与从机

铁头山羊 TIETOUSHANYANG

1.2.1. 主机的职责

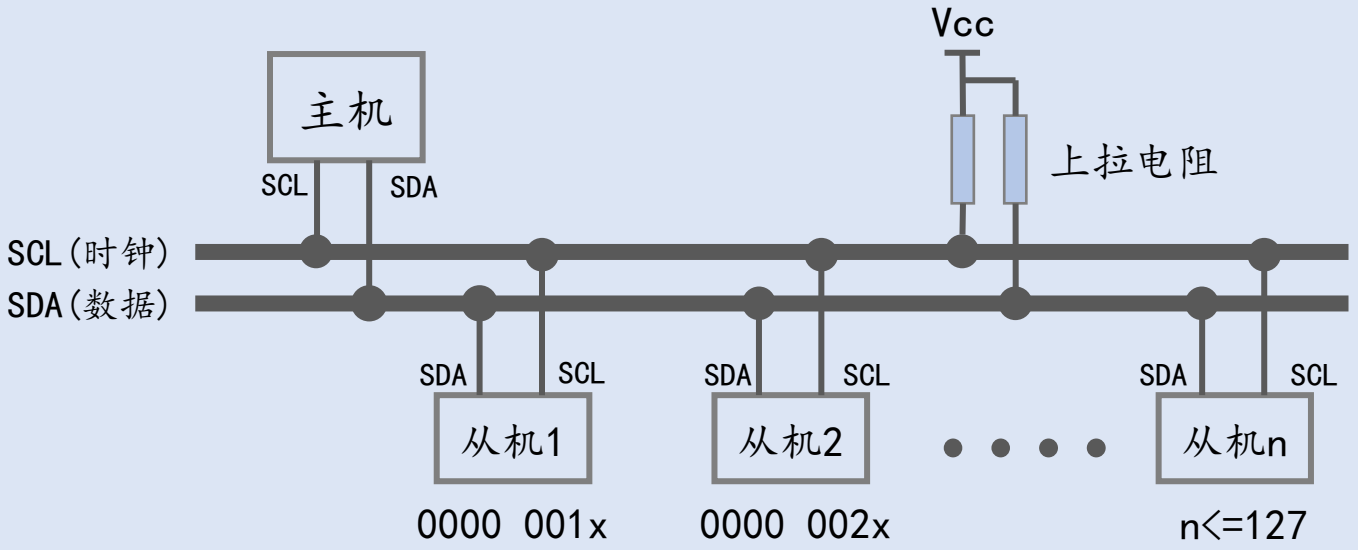
1.2.2. 波特率的选择

1.2.3. 数据通信的方向

1.2.4. 数据的发送和接收

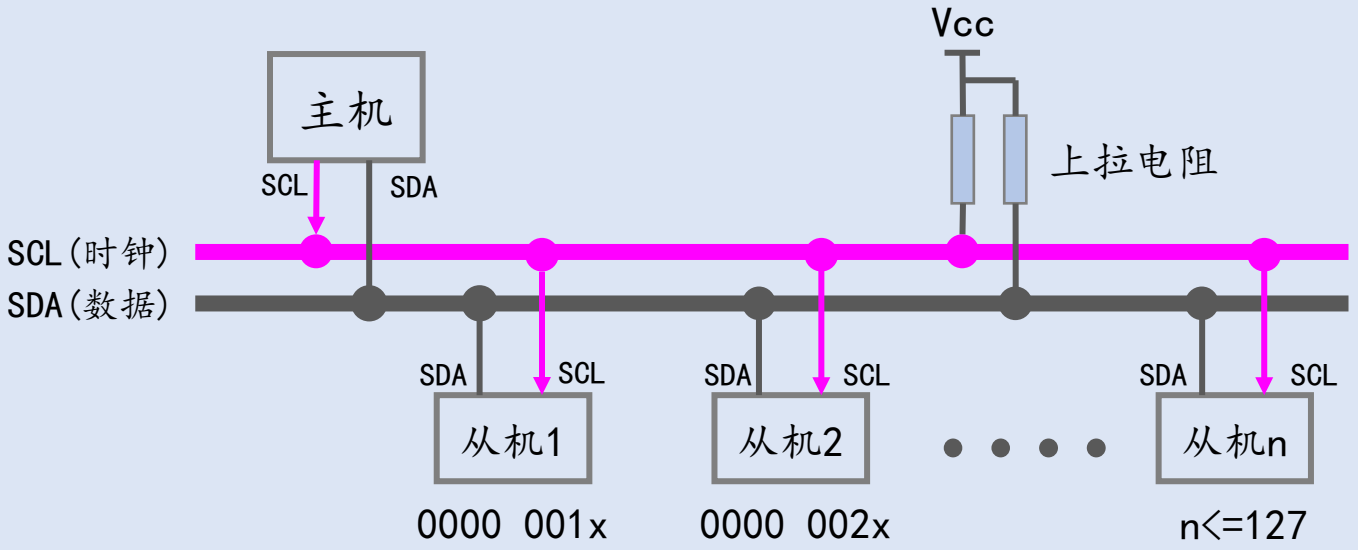
1.2.1. 主机的职责

- 主机是总线的管理员



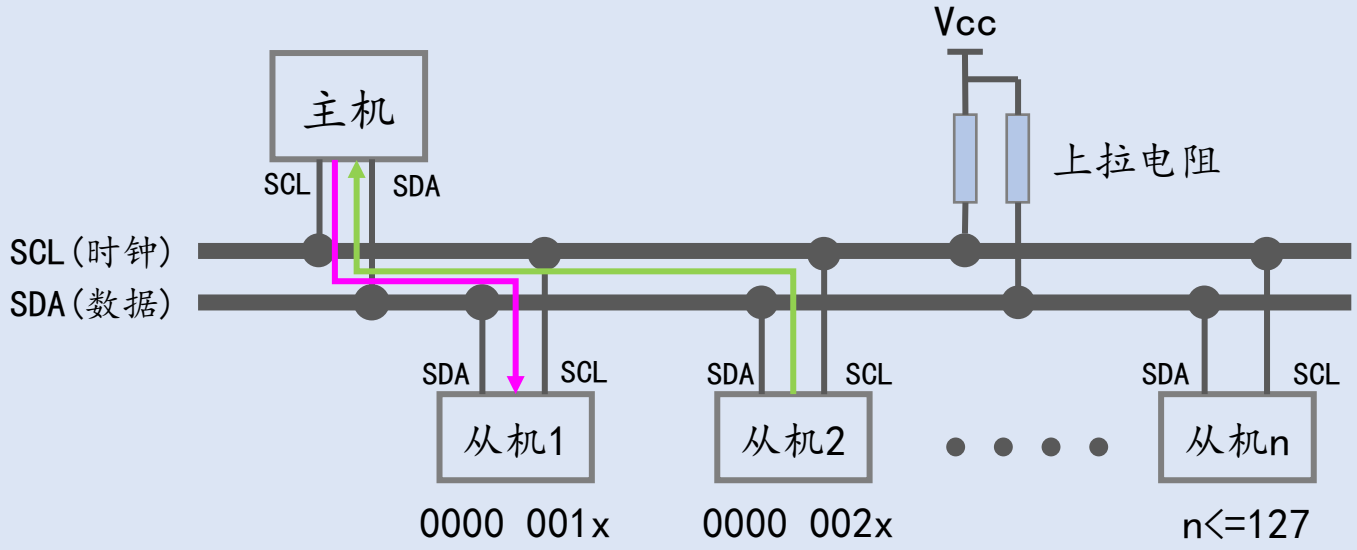
1.2.2. 波特率的选择

■ 主机产生SCL信号，从而控制通信的速度

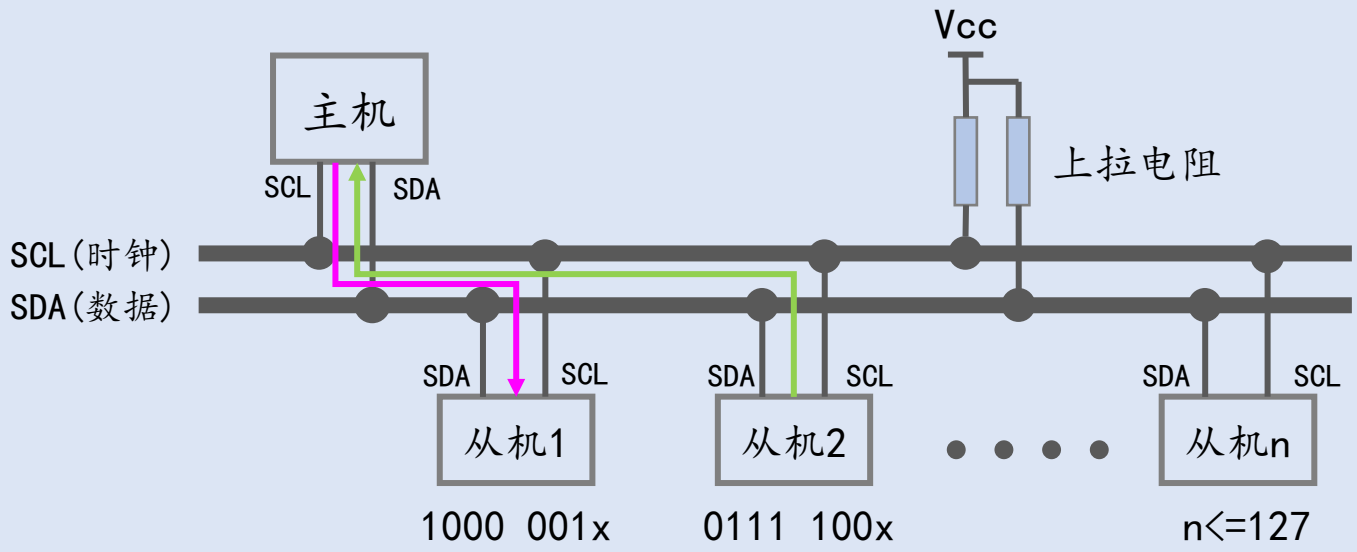


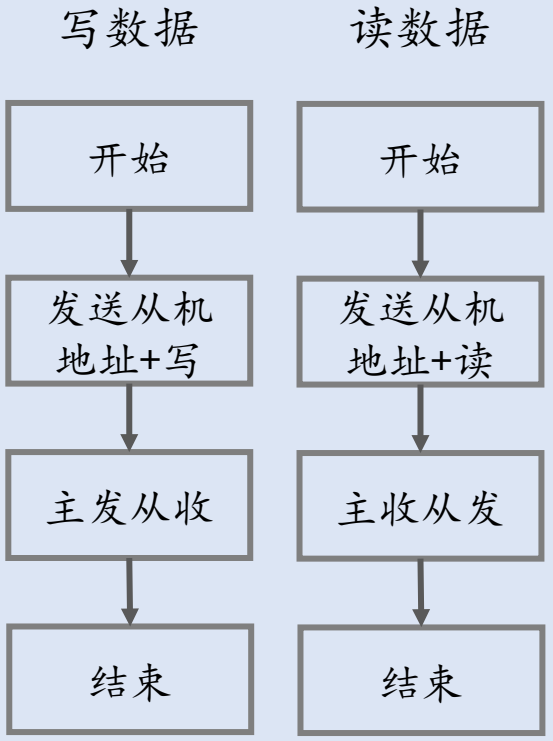
1.2.3. 数据通信的方向

- 仅有主机可以主动发起数据传输
- 传输方向：主机->从机，从机->主机，不存在从机->从机



1.2.4. 数据的发送和接收



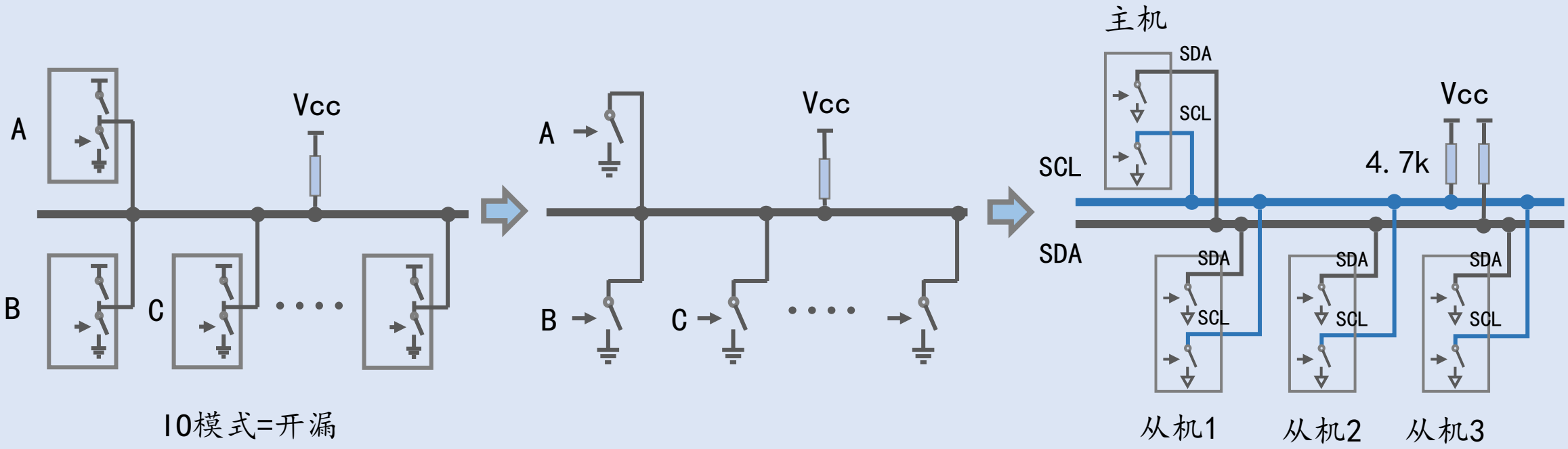


1.3. 开漏和“线与”

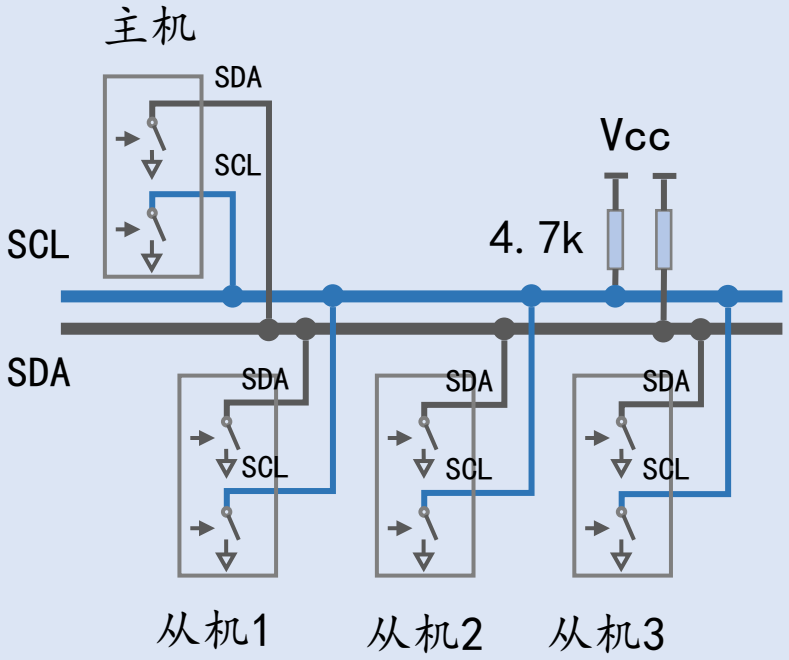
1.3.1. 线与

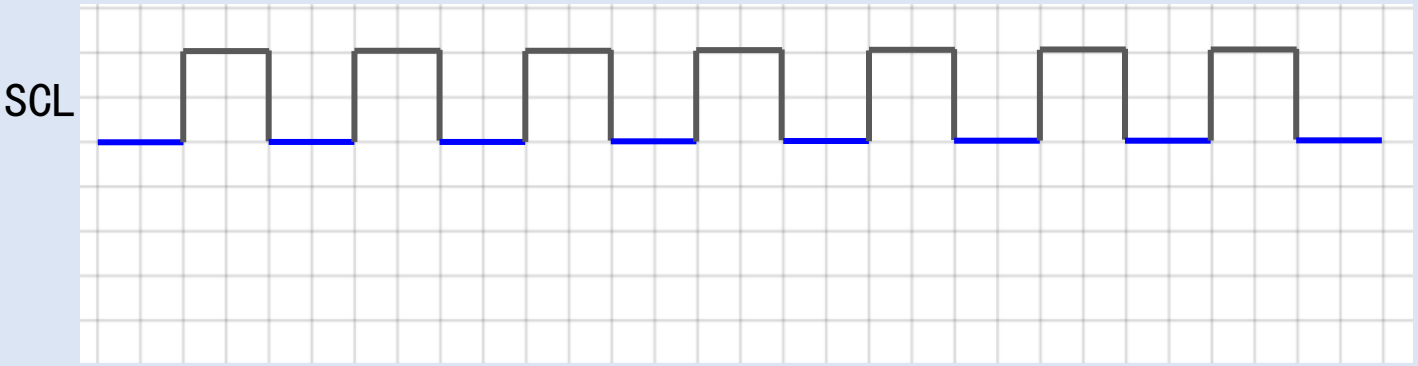
铁头山羊 TIETOUSHANYANG

使用硬件电路实现逻辑“与” 输出 = A&B&C&…

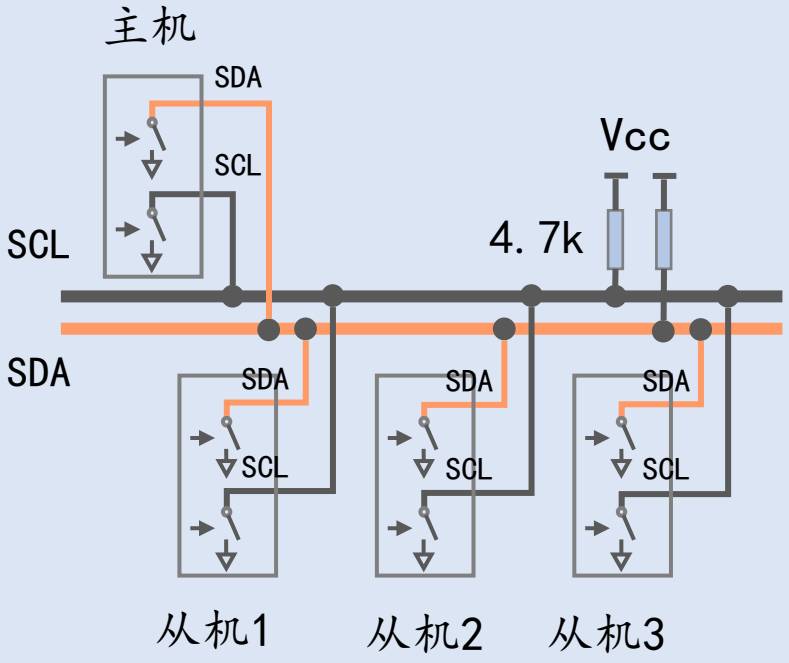


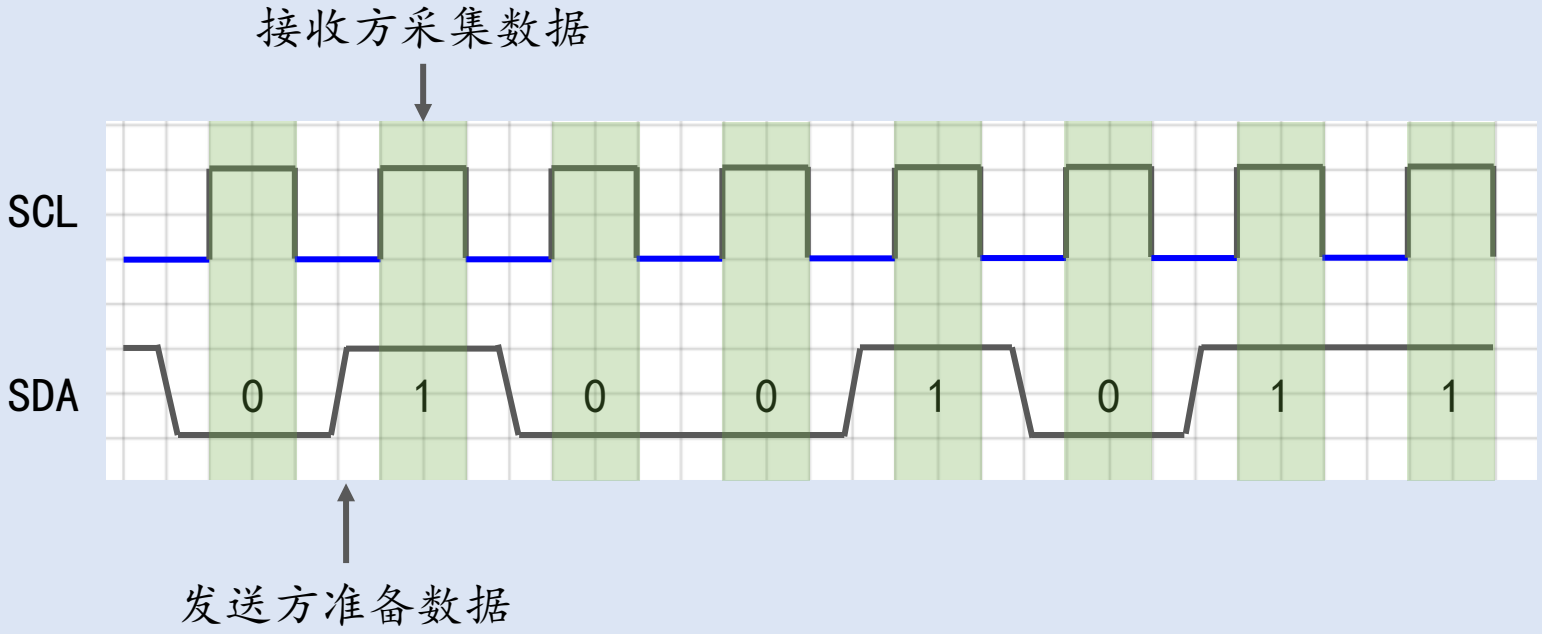
1.3.2. SCL信号的产生



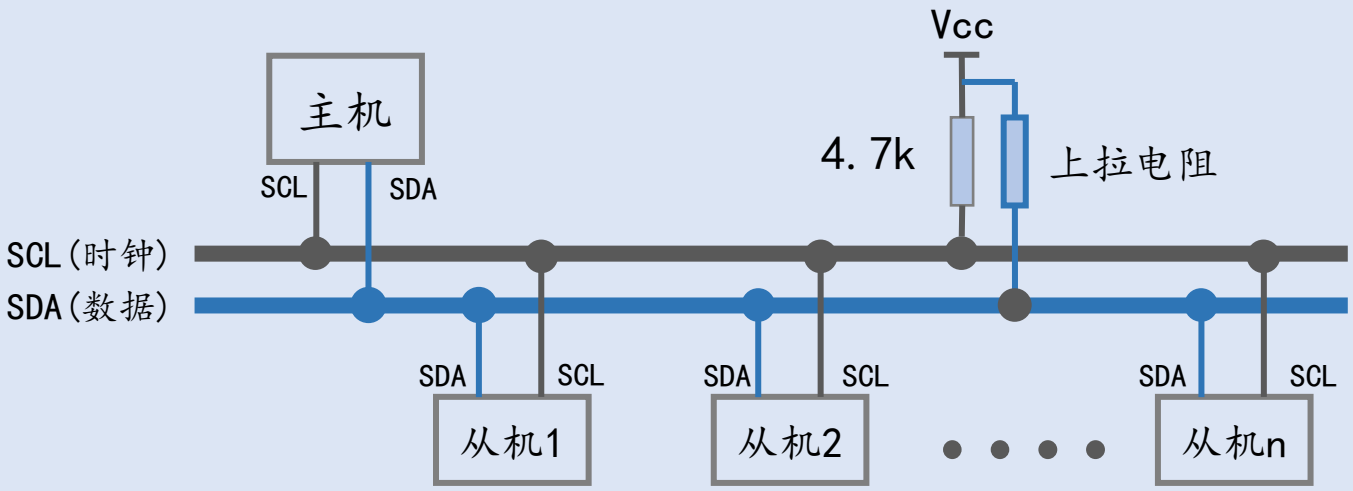


1.3.3. SDA信号的产生

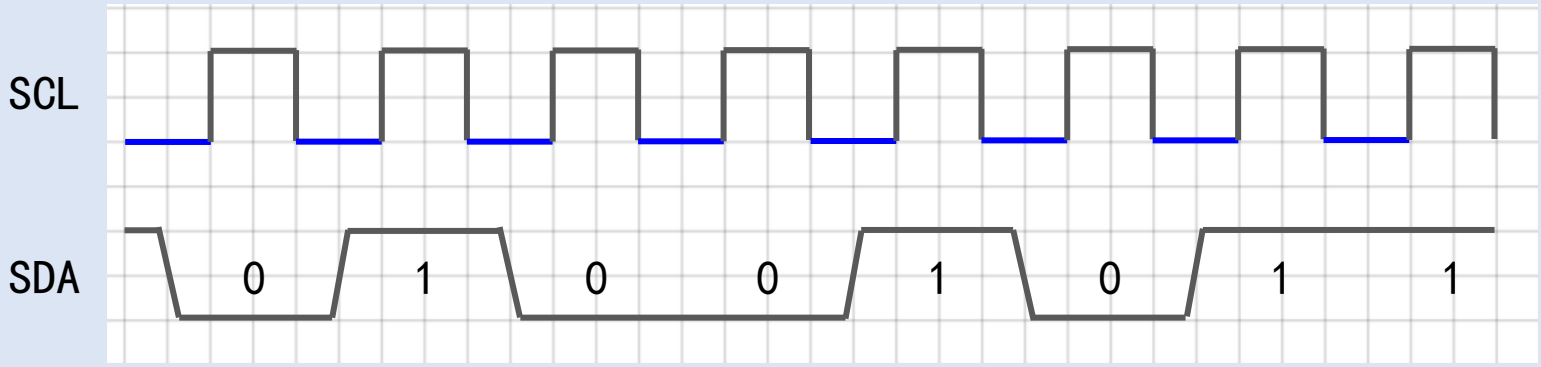


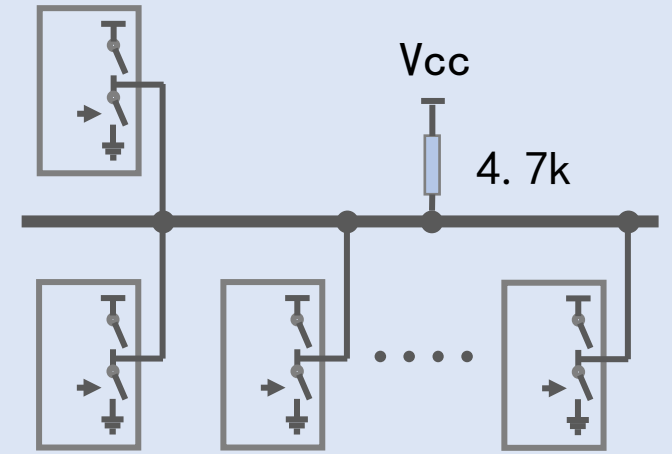


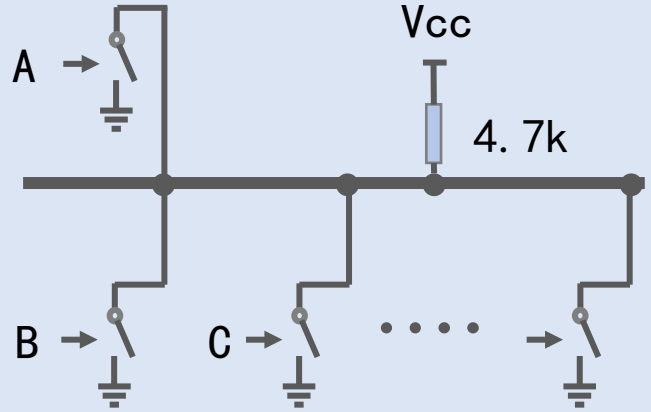
1.3. 开漏和“线与”



SCL=高，SDA保持（接收）；SCL=低，SDA切换（发送）



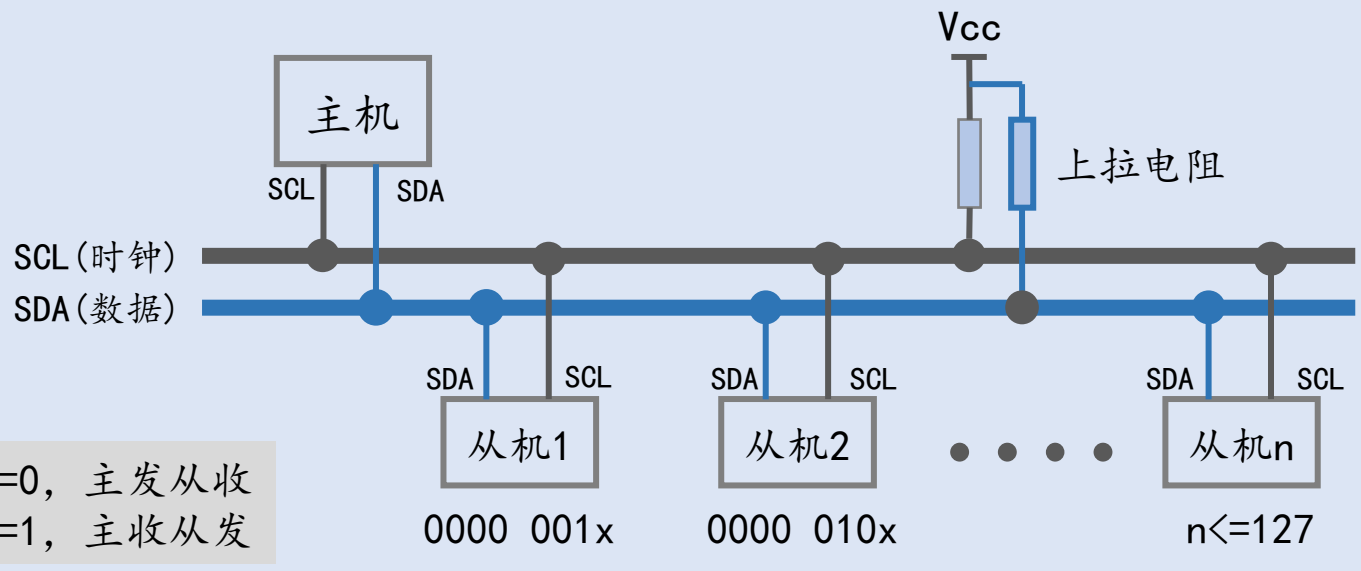




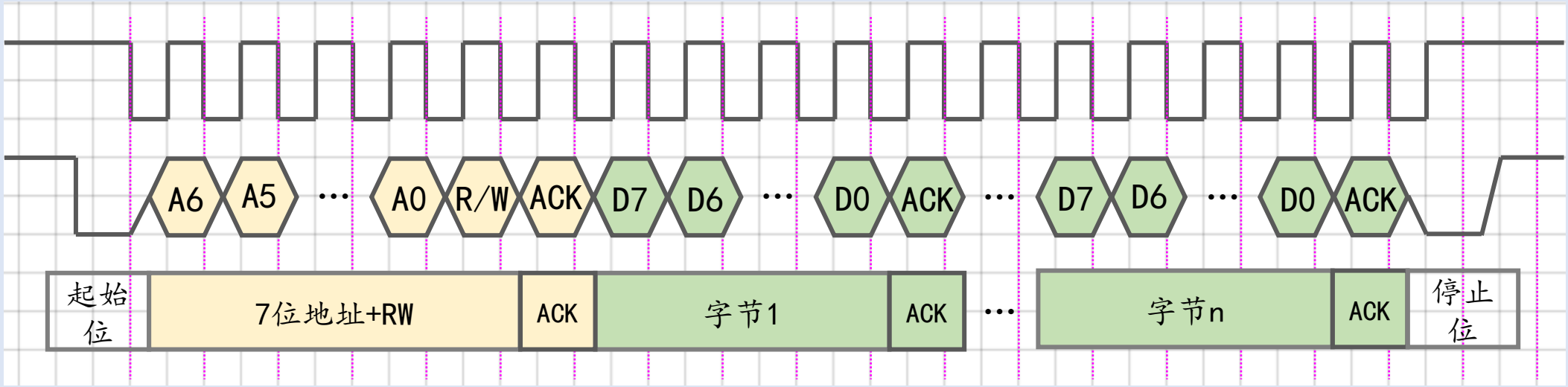
output = A & B & C & …

1.4. I2C的数据帧格式

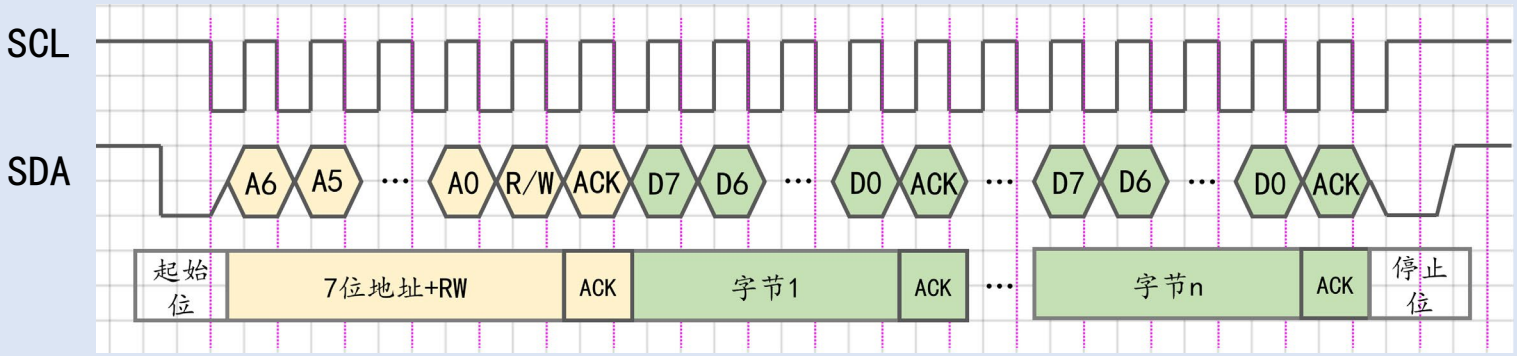
1.4.1. 数据帧格式概述

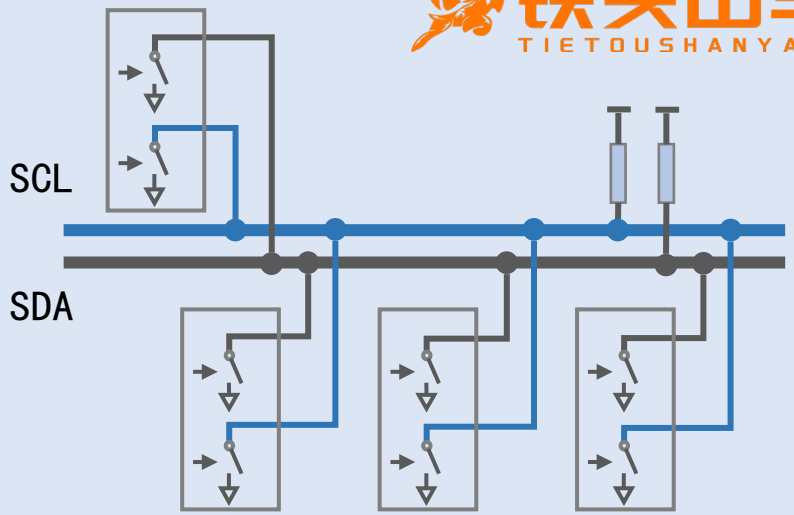


RW=0，主发从收
RW=1，主收从发



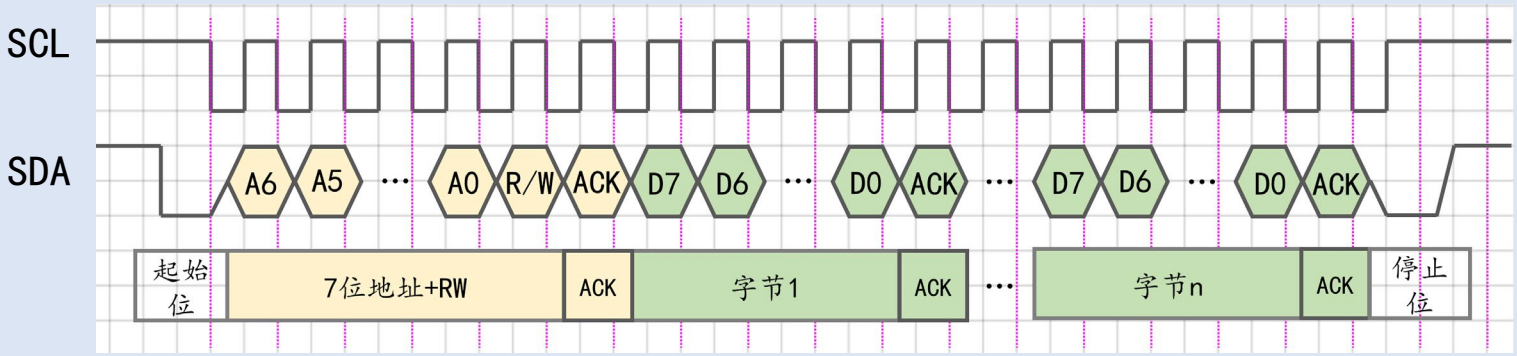
1.4.2. 空闲

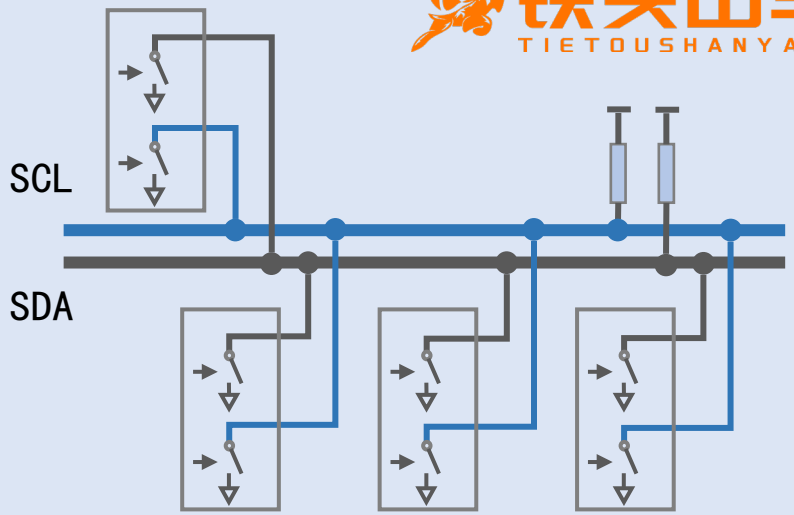


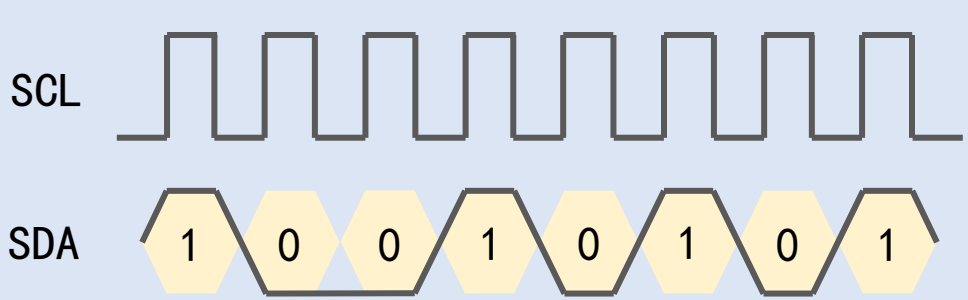


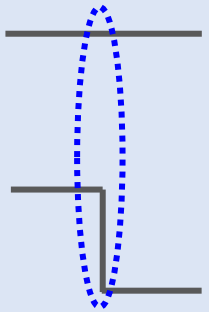
空闲：SCL=H，SDA=H

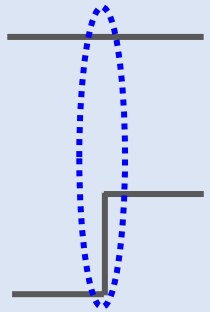
1.4.3. 起始位和停止位











正常情况下：

SCL=L，SDA可以变化；SCL=H，SDA保持不变

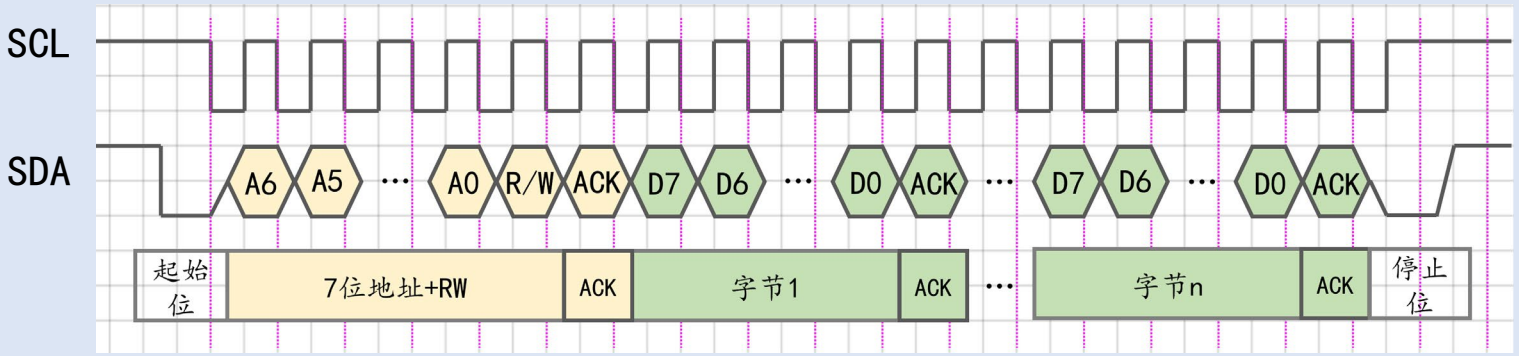
起始位**Start**

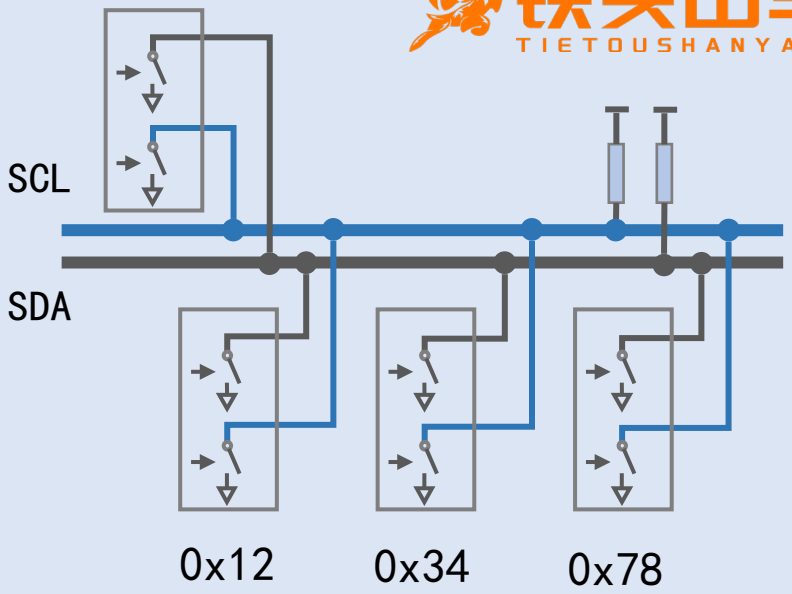
SCL=1，SDA↓

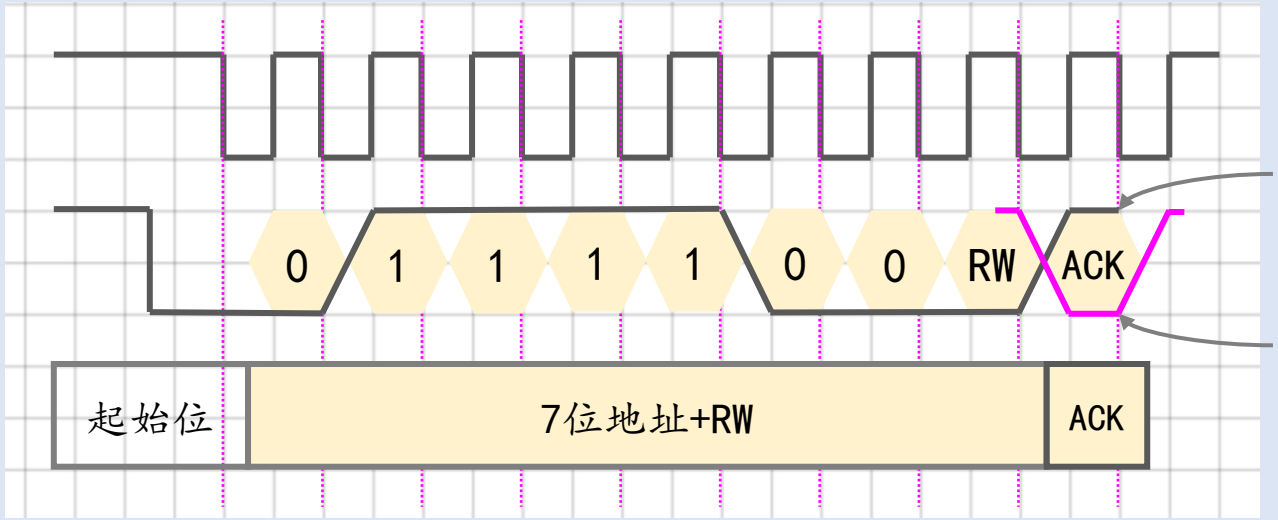
停止位**Stop**

SCL=1，SDA↑

1.4.4. 寻址与应答







Ack - Acknowledge - 告知收到

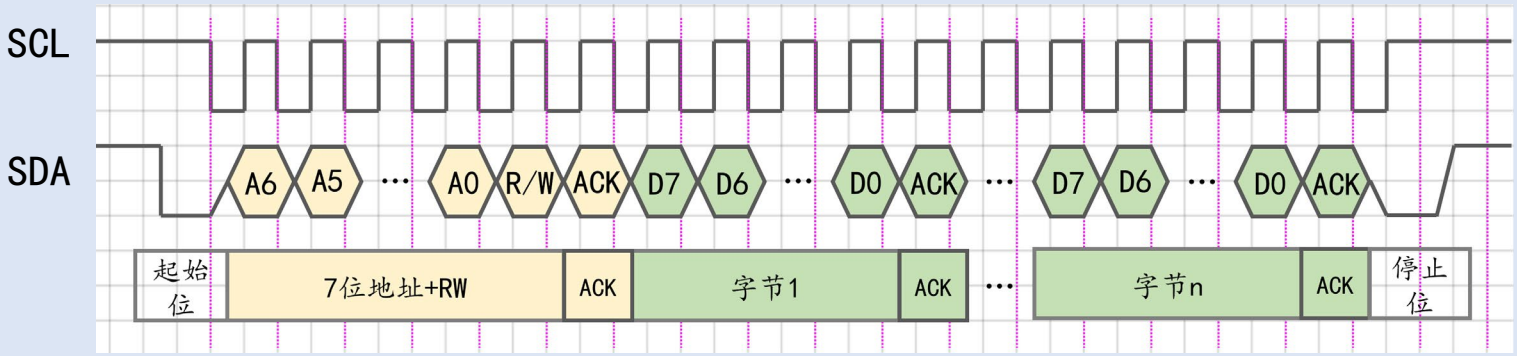
NAK - Negative Acknowledgment

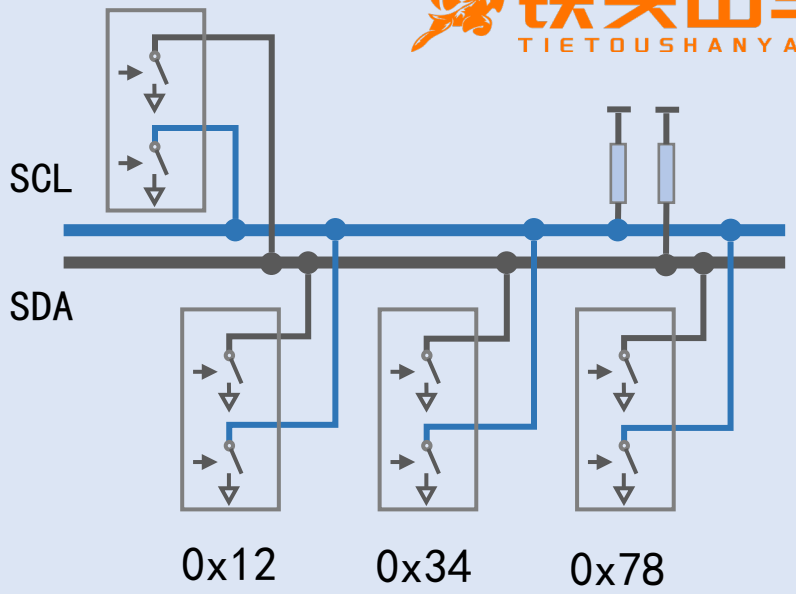
R/W - 读/写:

0 - 主发从收

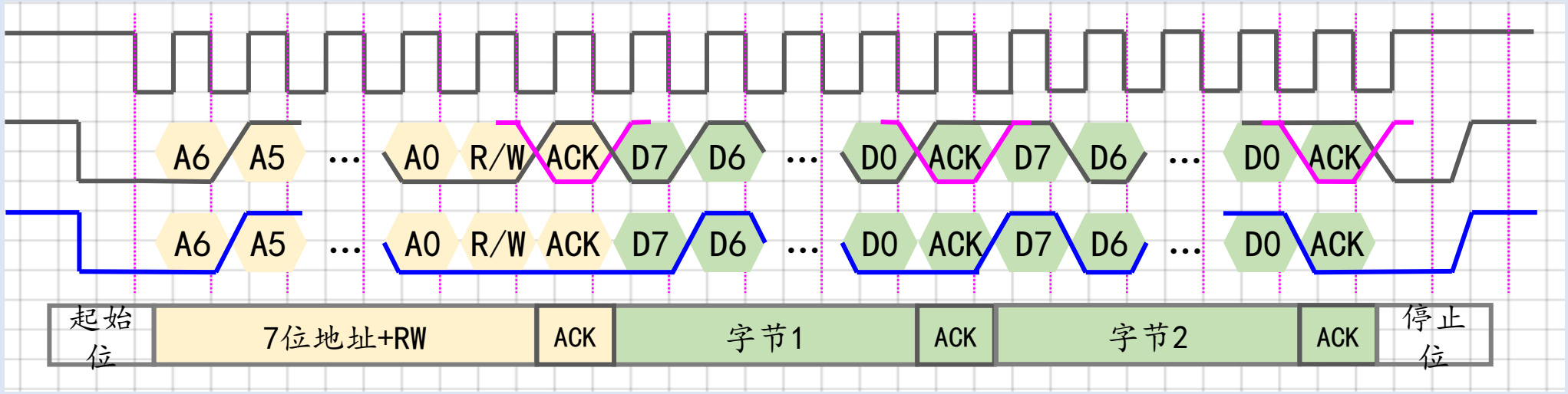
1 - 主收从发

1.4.5. 数据发送与应答

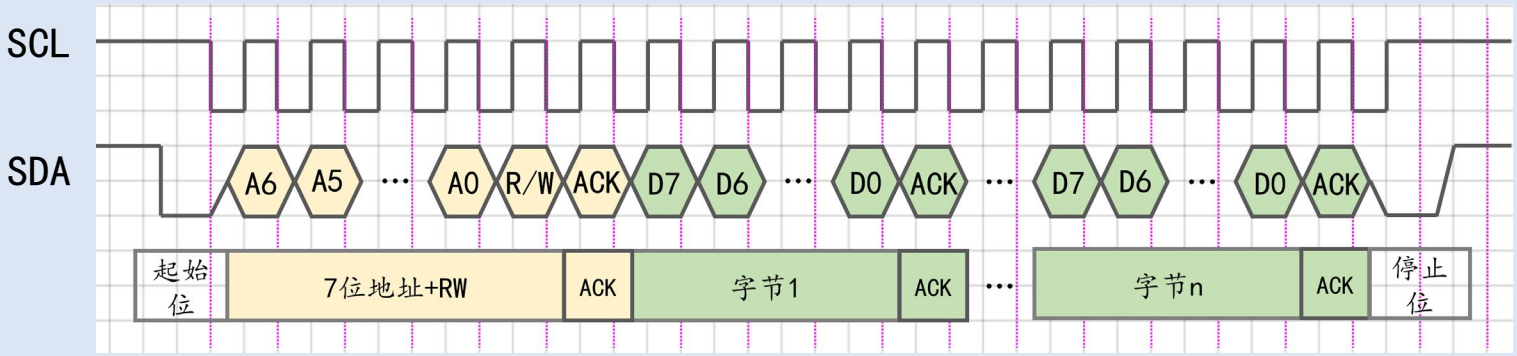


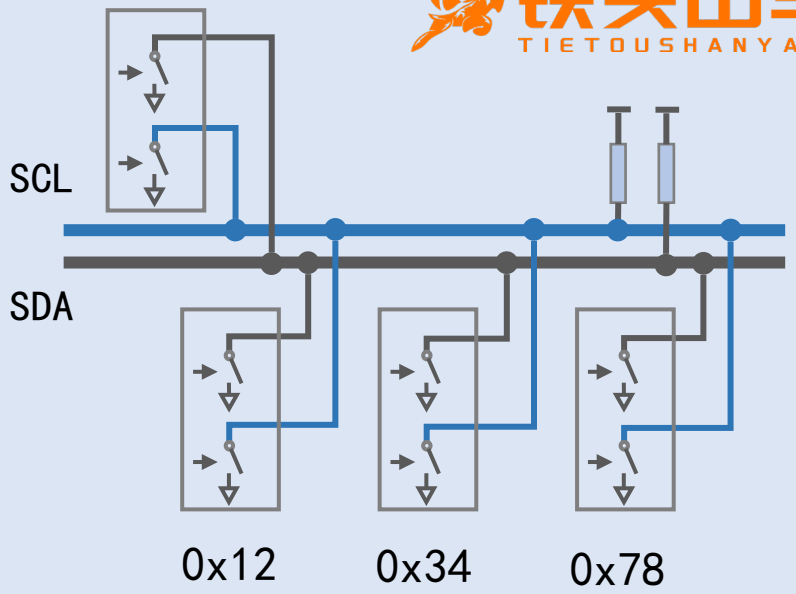


往0x78发两个字节：0x5a，0xa3

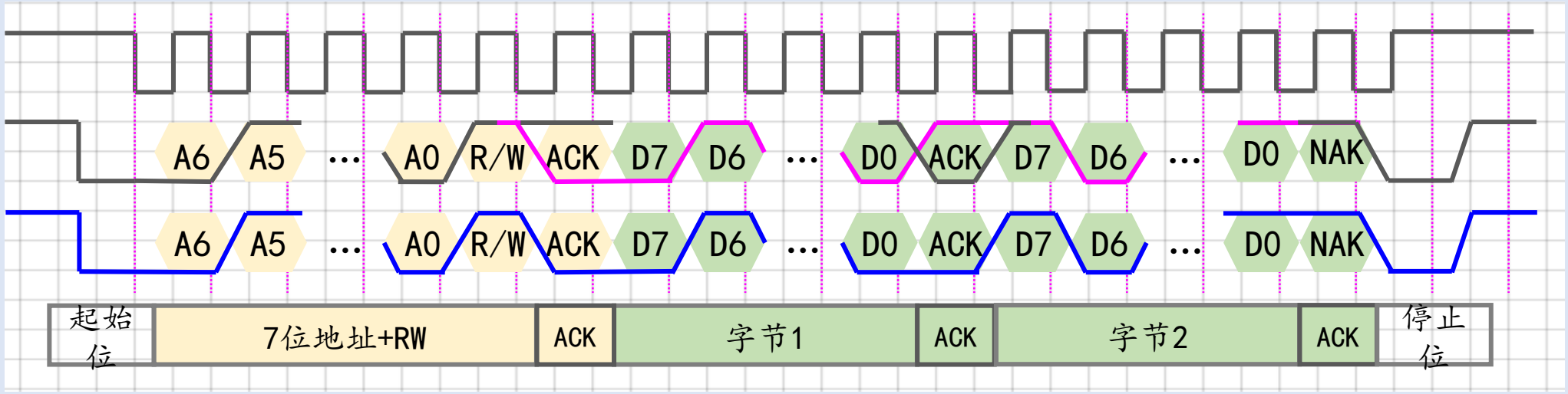


1.4.6. 数据接收与应答





从0x78接收两个字节：0x5a(0101 1010)，0xa3(1010 0011)



1.5. 随堂测验

铁头山羊 TIETOUSHANYANG

题目1：I2C是哪几个英文单词的缩写？

题目2：I2C总线由几根线组成？

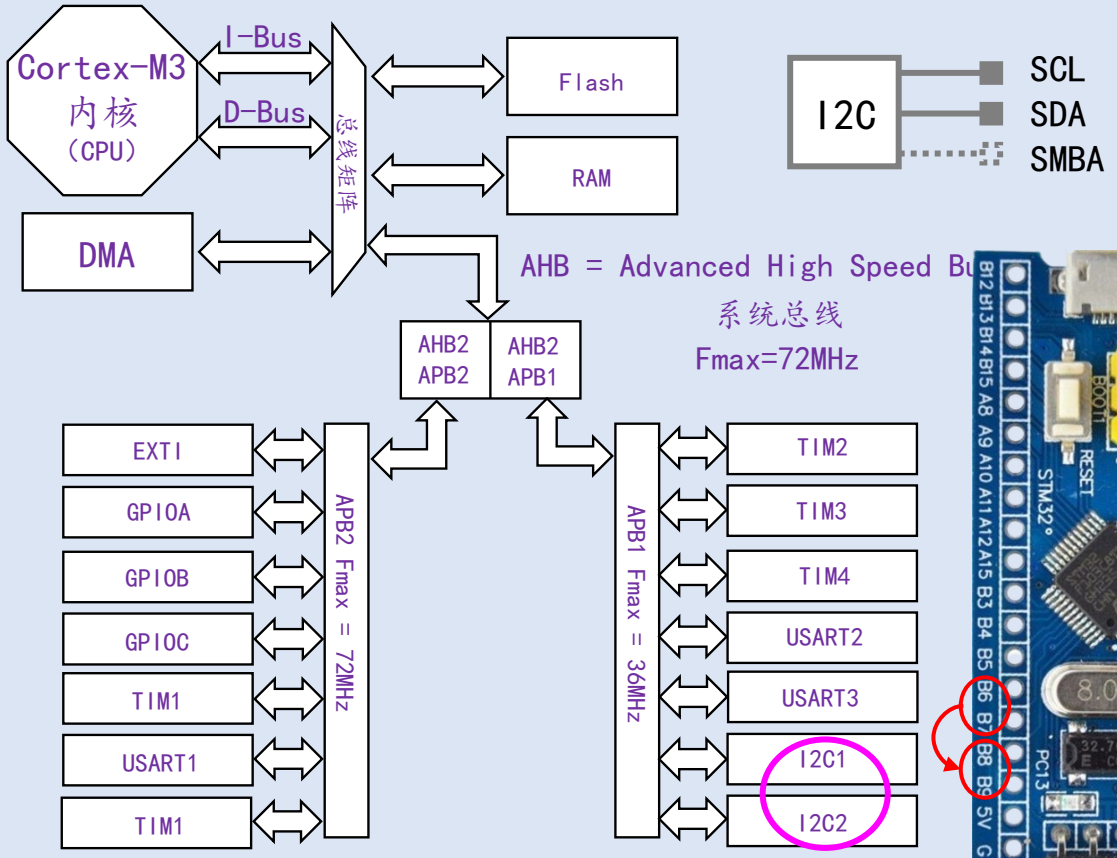
题目3：“线与”是如何实现的？

题目4：从机的地址是如何构成的？

题目5：I2C每个数据帧由多少位组成？

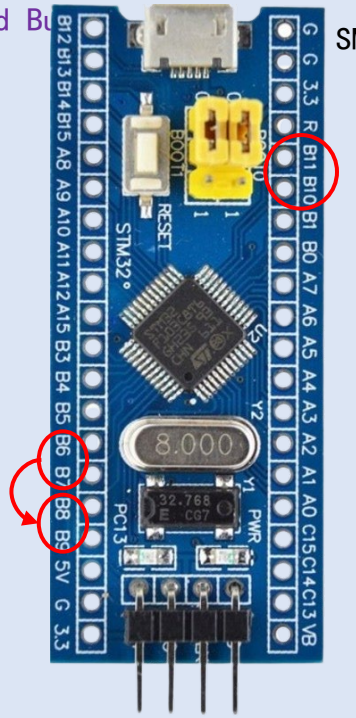
第二节、I2C模块的使用方法

2.1. I2C模块简介

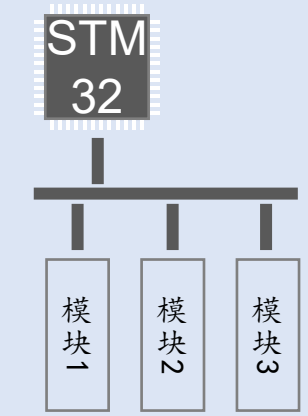


复用功能	Remap=0	Remap=1
I2C1_SCL	PB6	PB8
I2C1_SDA	PB7	PB9
I2C1_SMBA	PB5	

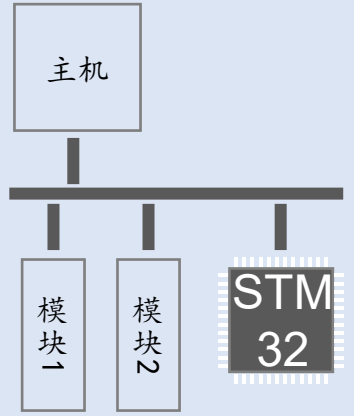
复用功能	Remap=0
I2C2_SCL	PB10
I2C2_SDA	PB11
I2C2_SMBA	PB12



SMBA=System Management Bus Alert(系统管理总线报警引脚)

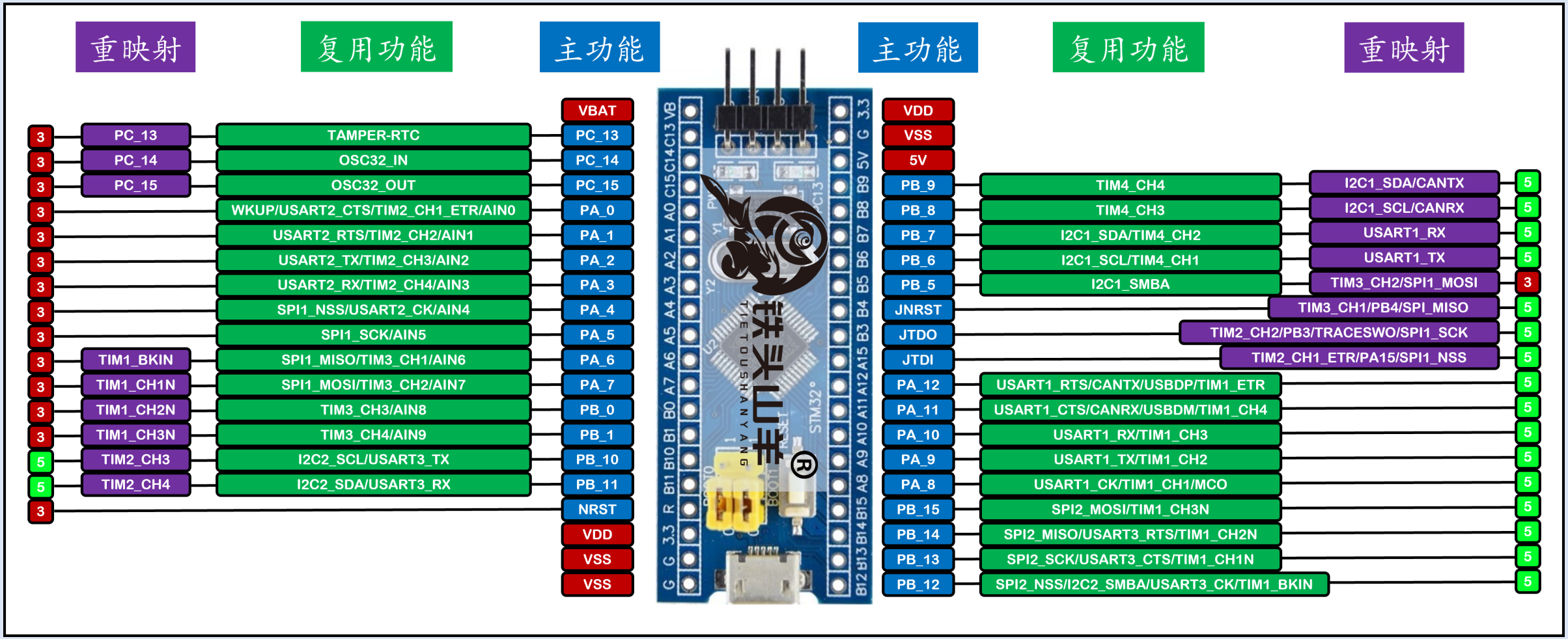


32单片机做主机

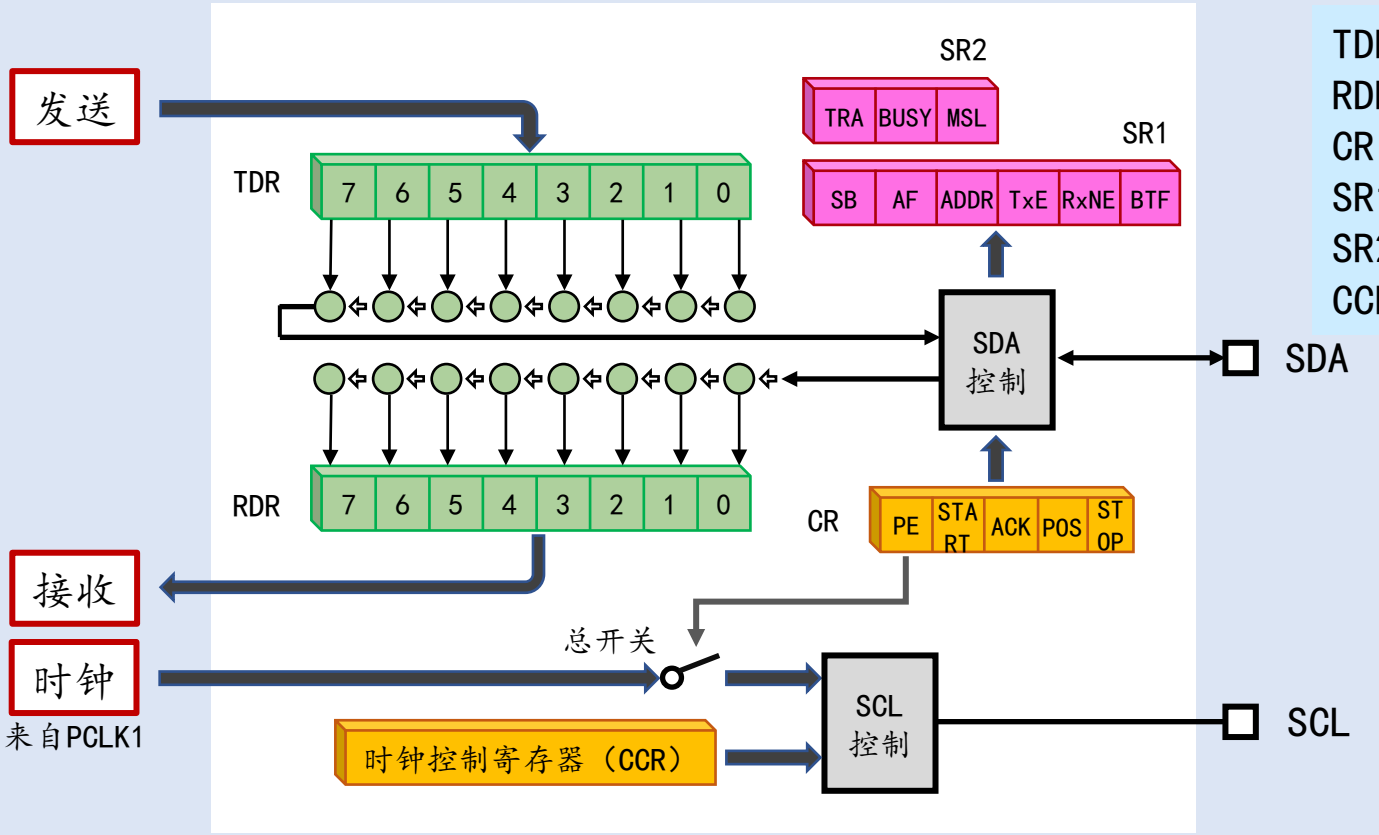


32单片机做从机

铁头山羊 TIETOUSHANYANG

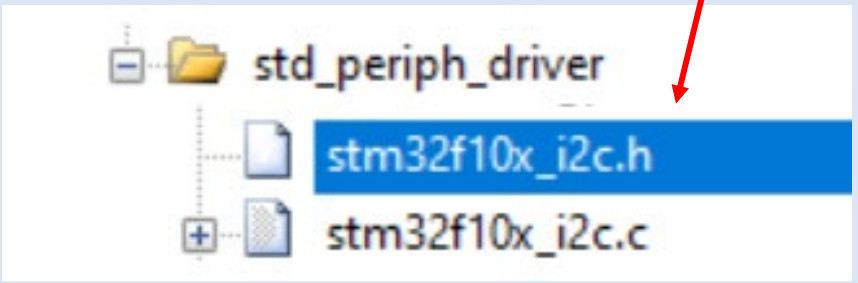


2.2. 寄存器组与内部结构框图



TDR - Transmit Data Register - 发送数据寄存器
RDR - Receive Data Register - 接收数据寄存器
CR - Control Register - 控制寄存器
SR1 - Status Register 1 - 状态寄存器1
SR2 - Status Register 2 - 状态寄存器2
CCR - Clock Control Register - 时钟控制寄存器

标准库的编程接口

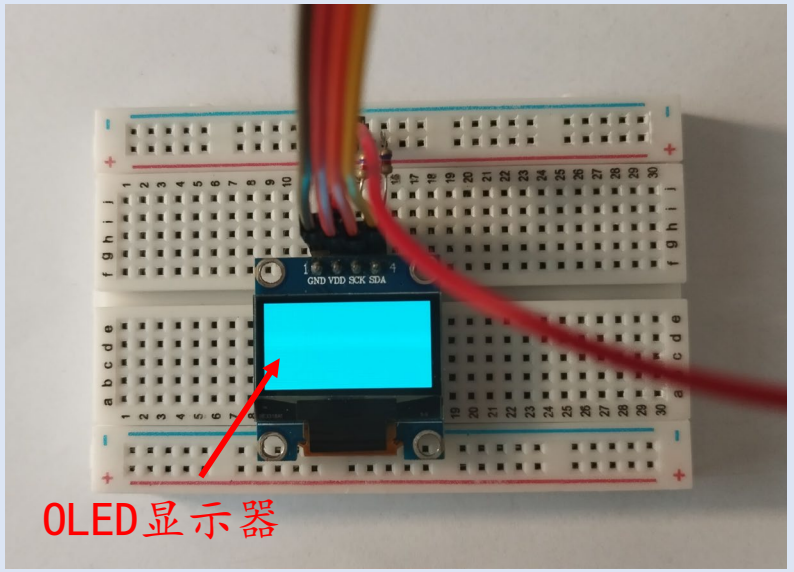


2.3. I2C的初始化

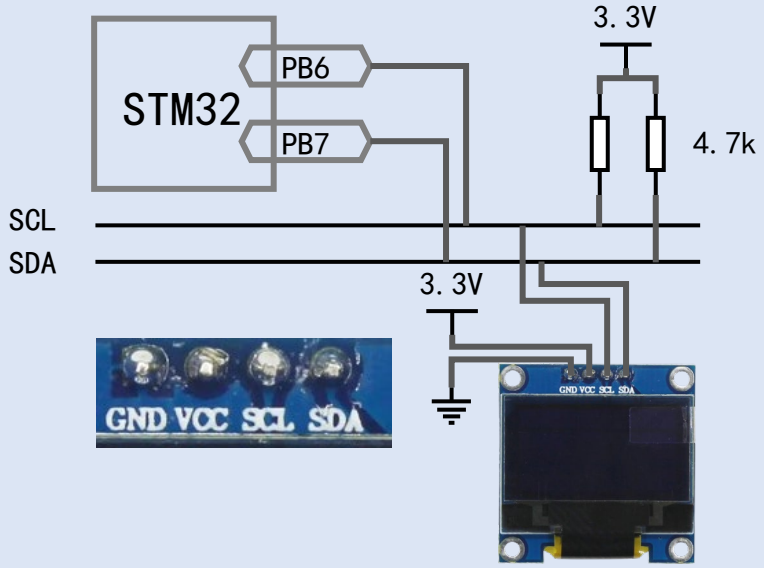
铁头山羊 TIETOUSHANYANG

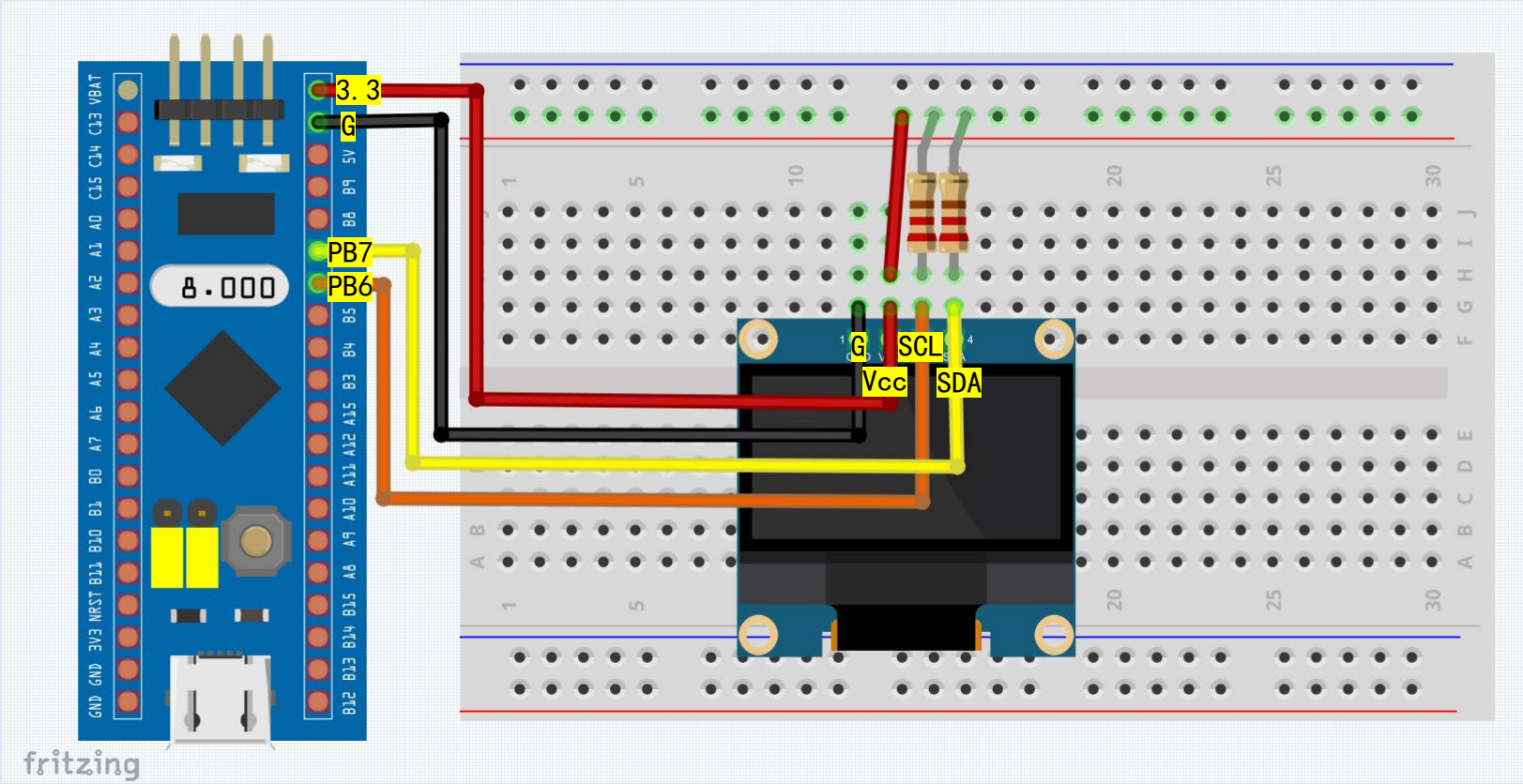
2.3.1. 实验简介


```
// I2C初始化
void App_I2C_Init(…);
// I2C主机发送数据
ErrorStatus App_I2C_MasterTransmit(…);
// I2C主机接收数据
ErrorStatus App_I2C_MasterReceive(…);
```



2.3.2. 电路连接





STM32最小系统板　杜邦线（公对公）*1

ST-Link调试器　杜邦线（公对母）*4

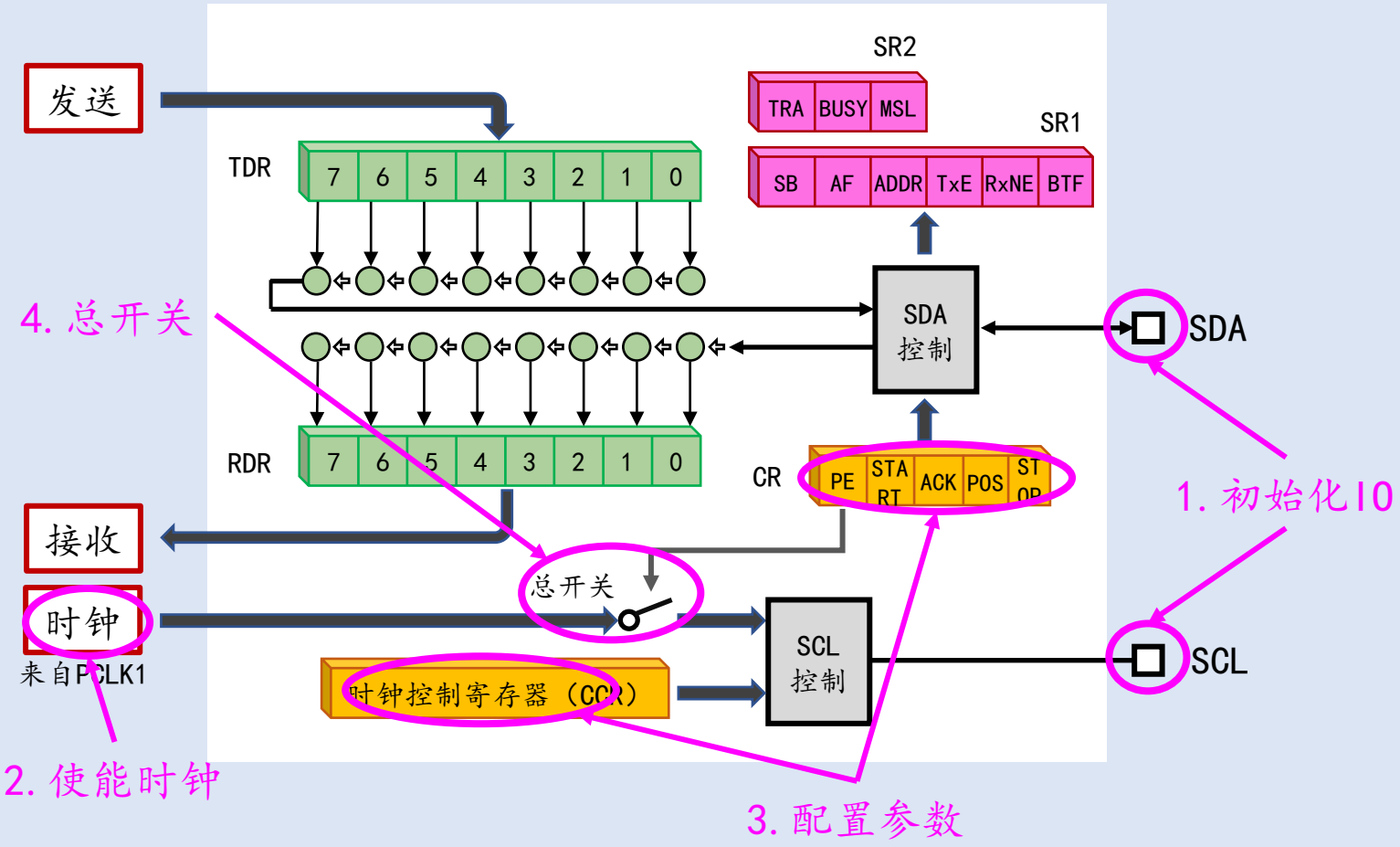
OLED显示器　4.7k电阻*2

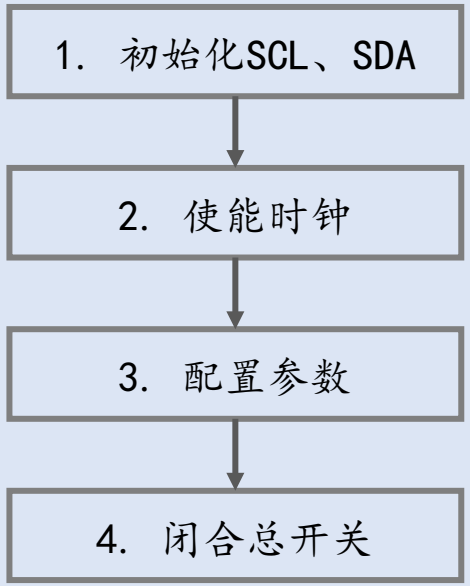
面包板

32_PB6<->OLED_SCL　32_PB7<->OLED_SDA　32_3V3<->OLED_VCC

32_G <->OLED_GND　OLED_SCL<-4.7k->OLED_VCC　OLED_SDA<-4.7k->OLED_VCC

2.3.3. I2C初始化流程梳理





2.3.4. 配置IO引脚

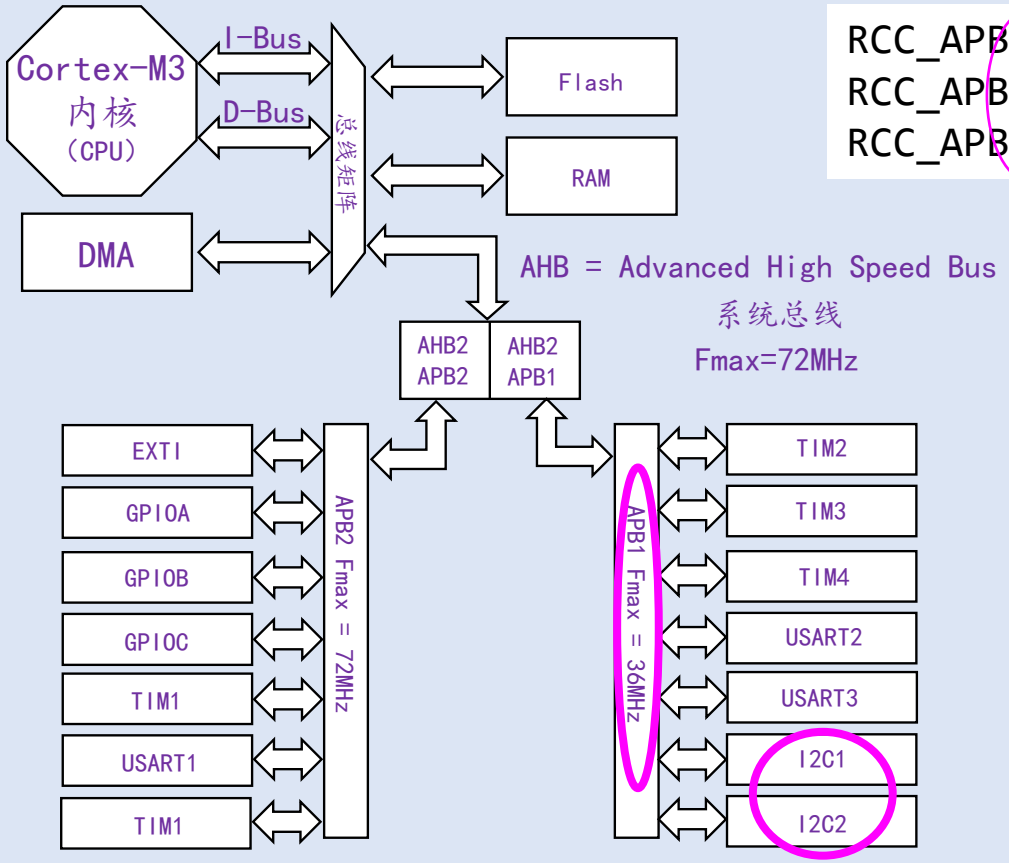
参考手册 8.1.11

表24　I2C接口

I2C引脚	配置	GPIO配置
I2Cx_SCL	I2C时钟	开漏复用输出
I2Cx_SDA	I2C数据	开漏复用输出
I2Cx_SMBA	SMBus报警	I2C模式下不使用

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);

GPIO_InitTypeDef GPIO_InitStruct;
GPIO_InitStruct.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AF_OD;
GPIO_InitStruct.GPIO_Speed = GPIO_Speed_10MHz;
GPIO_Init(GPIOB, &GPIO_InitStruct);
```

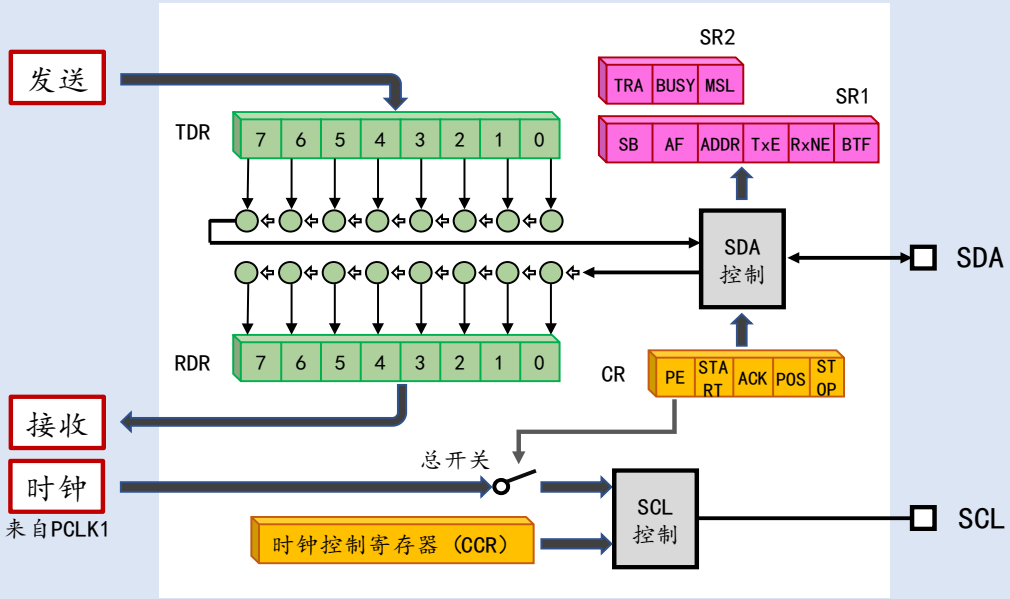
2.3.5. 开启时钟




```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_I2C1, ENABLE); //使能
RCC_APB1PeriphResetCmd(RCC_APB1Periph_I2C1, ENABLE); //复位
RCC_APB1PeriphResetCmd(RCC_APB1Periph_I2C1, DISABLE);
```

首次使用时进行复位是个好习惯

2.3.6. 配置I2C参数



速度模式	波特率	速度模式	波特率
标准模式(Sm)	100k	高速模式(HSm)	3.4M
快速模式(Fm)	400k	超快速模式(UFm)	5M
快速增强模式(Fm+)	1M		

填写参数

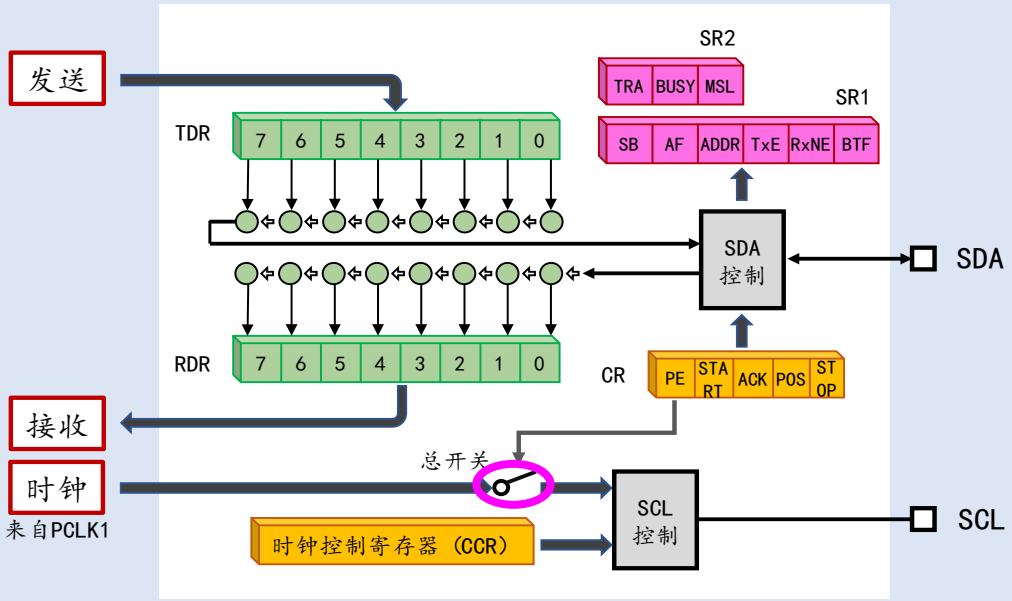
标准库编程接口：

```
// @简介：初始化I2C总线
void I2C_Init(I2C_TypeDef* I2Cx, I2C_InitTypeDef* I2C_InitStruct)
```

示例代码：

```
I2C_InitTypeDef I2C_InitStruct;  ← 声明一个初始化变量
I2C_InitStruct.I2C_ClockSpeed = 400000; // 波特率，最高400k
I2C_InitStruct.I2C_Mode = I2C_Mode_I2C; // @I2C_Mode_SMBusDevice
                                        // @I2C_Mode_SMBusHost
I2C_InitStruct.I2C_DutyCycle = I2C_DutyCycle_2; // I2C_DutyCycle_16_9
I2C_InitStruct.I2C_OwnAddress1 = 0x12; // 从机地址1（从机模式有效）
I2C_InitStruct.I2C_Ack = I2C_Ack_Disable; // ACK初始值
I2C_InitStruct.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
                                                              _10bit
I2C_Init(I2C1, &I2C_InitStruct);  ← 调用I2C_Init
```

2.3.7. 使能I2C



标准库编程接口：

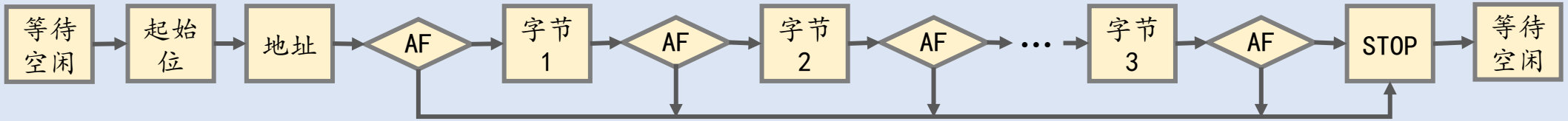
```
// @简介: 控制I2C的使能和禁止
// @参数: NewState ENABLE - 使能, DISABLE - 禁止
void I2C_Cmd(I2C_TypeDef *I2Cx, FunctionalState NewState);
```

示例代码：

```
I2C_Cmd(I2C1, ENABLE);
```

2.4. 主机数据发送

2.4.1. 数据发送流程概述



2.4.2. 等待空闲

2.4.3. 发送起始位

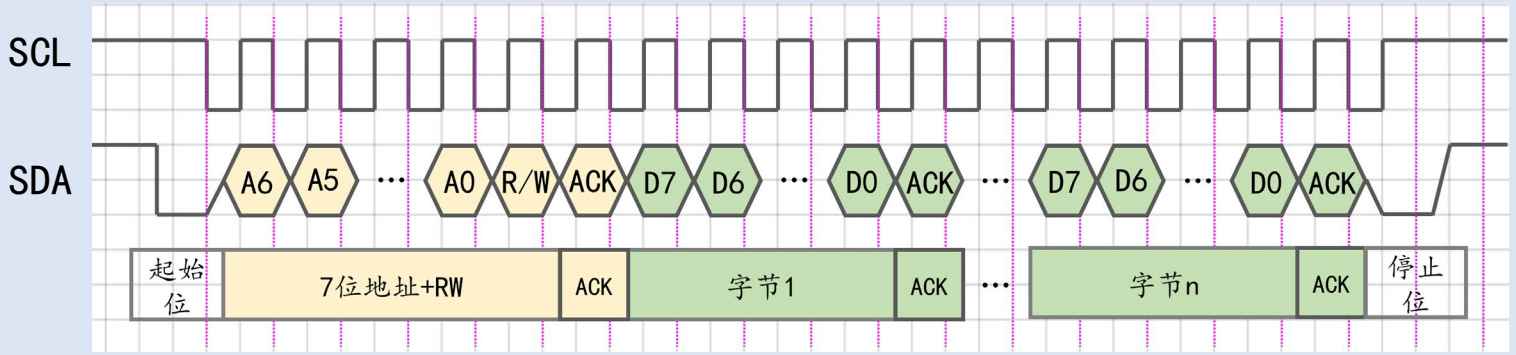
2.4.4. 发送地址

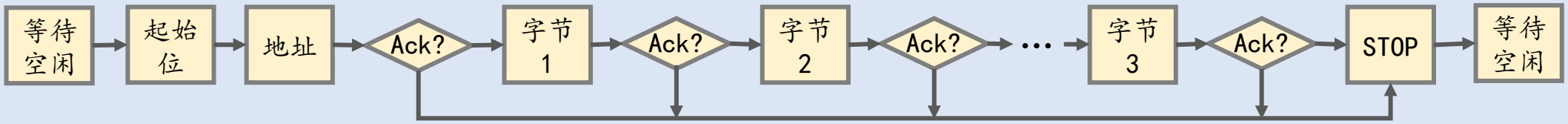
2.4.5. 发送数据

2.4.6. 发送停止位

2.4.7. 测试

2.4.1. 数据发送流程概述

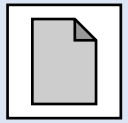




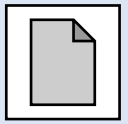

```
ErrorStatus App_I2C_MasterTransmit(uint8_t SlaveAddr, const uint8_t *pData, uint16_t Size);
```

2.4.2. 等待空闲

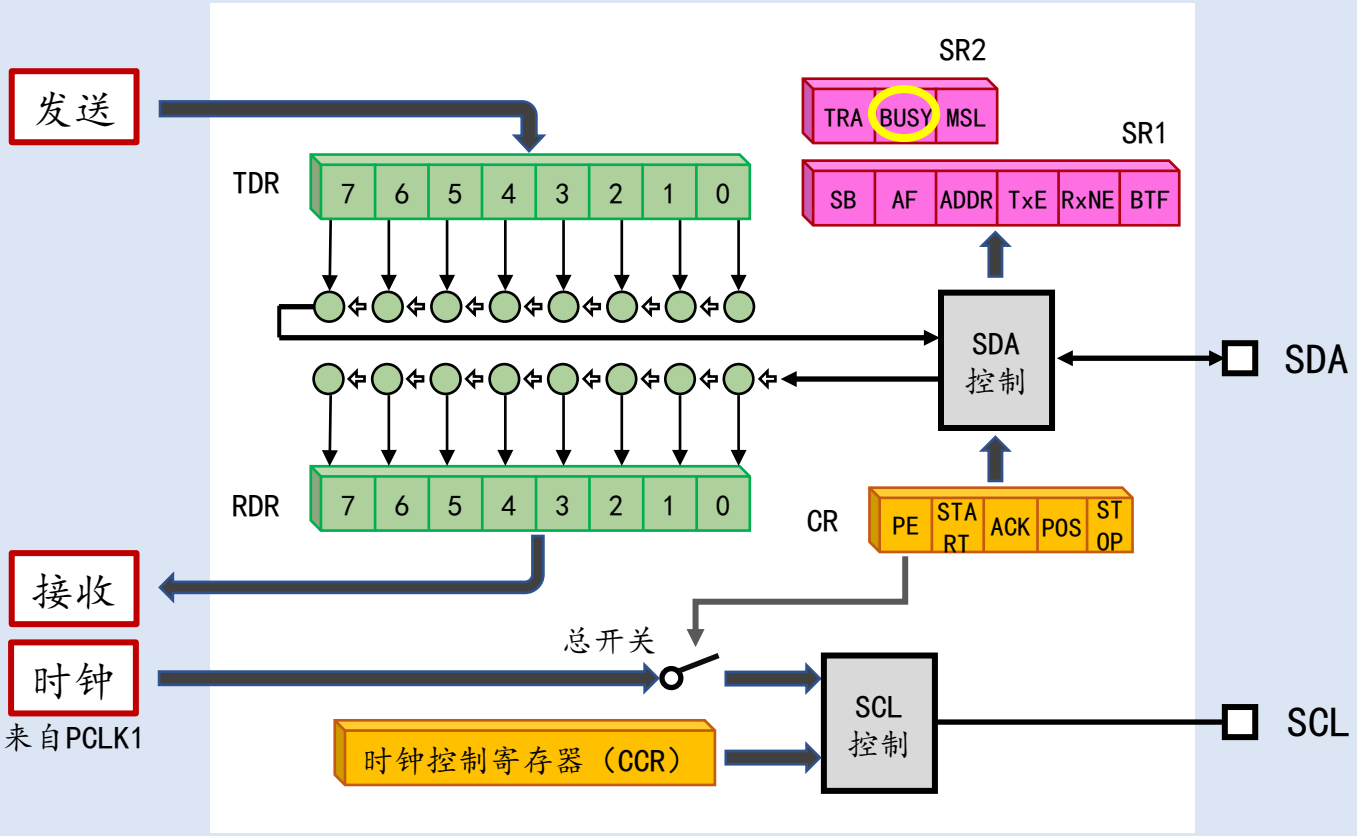
2.4.2.1. BUSY标志位



2.4.2.2. 编程接口和示例代码

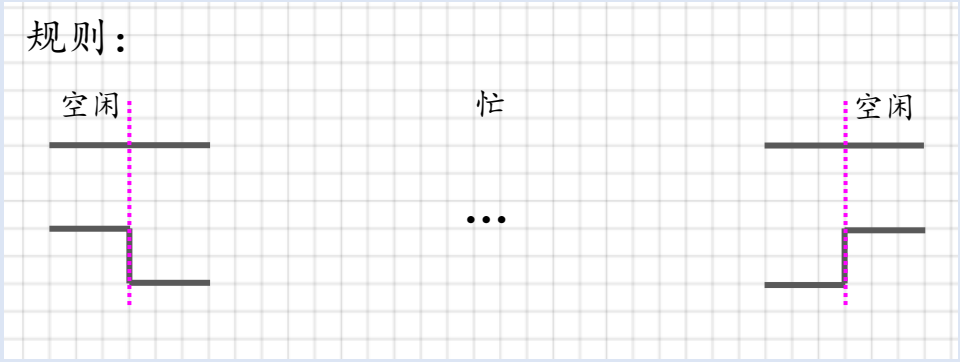


2.4.2.1. BUSY标志位

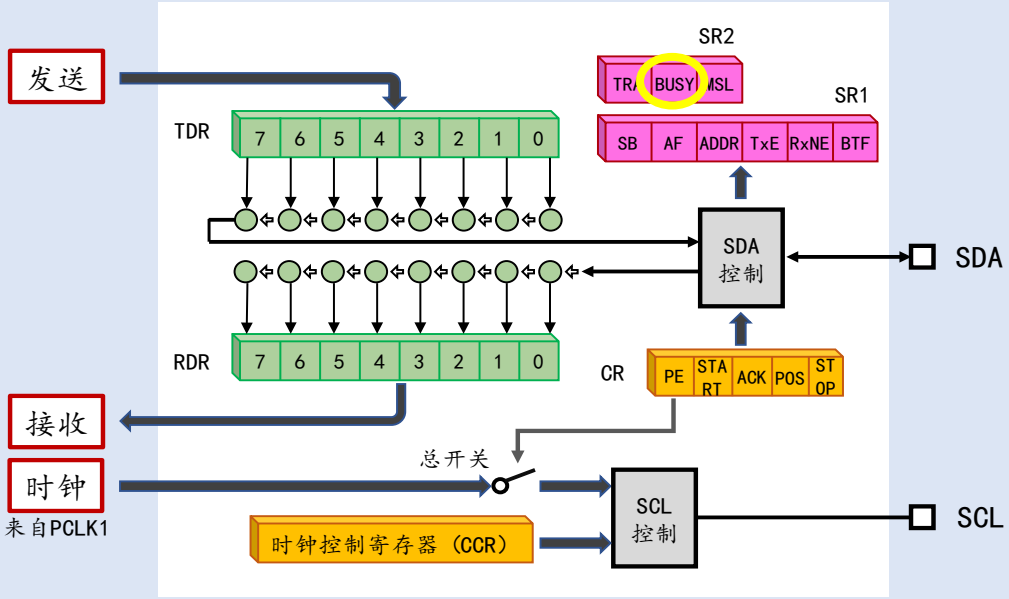


BUSY 用于查询总线的空闲状态

0 - 总线空闲 1 - 总线忙



2.4.2.2. 编程接口和示例代码



标准库编程接口：

```
// @简介: 查询标志位
// @参数: I2C_FLAG 标志位的名称
//         I2C_FLAG_TRA I2C_FLAG_BUSY I2C_FLAG_MSL
//         I2C_FLAG_SB …
// @返回: RESET - 标志位等于0, SET - 标志位等于1
FlagStatus I2C_GetFlagStatus(I2C_TypeDef *I2Cx, uint32_t I2C_FLAG)
```

示例代码：

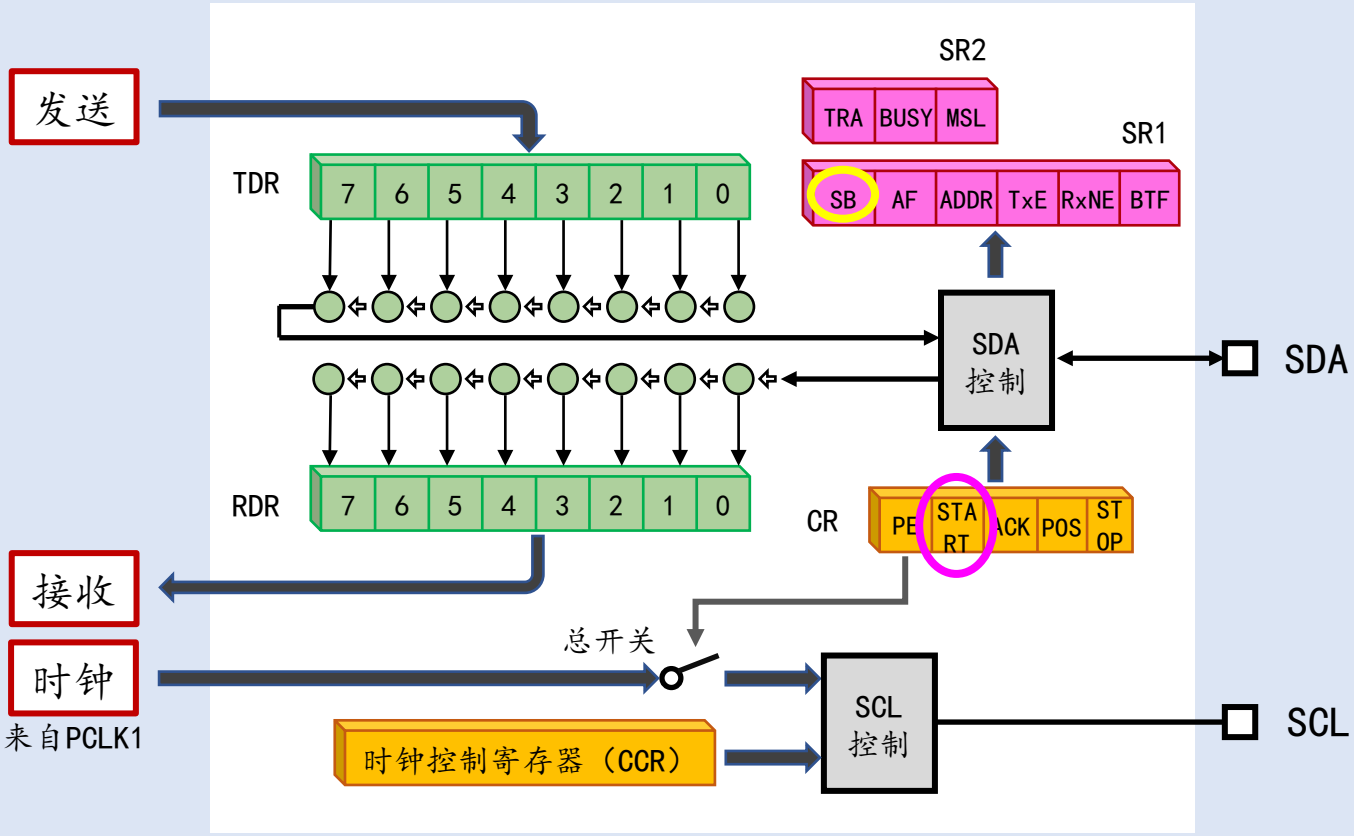
```
while(I2C_GetFlagStatus(I2C1,I2C_FLAG_BUSY)==SET); // 等待空闲
```

2.4.3. 发送起始位

2.4.3.1. START和SB

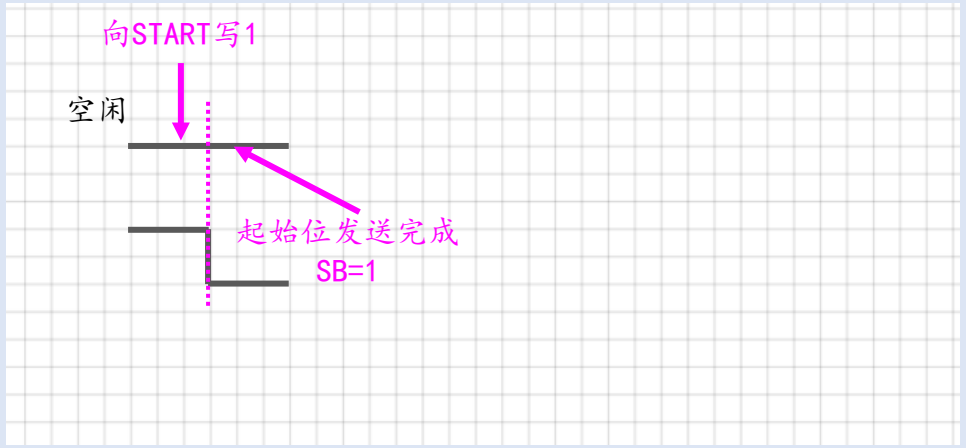
2.4.3.2. 编程接口和示例代码

2.4.3.1. START和SB

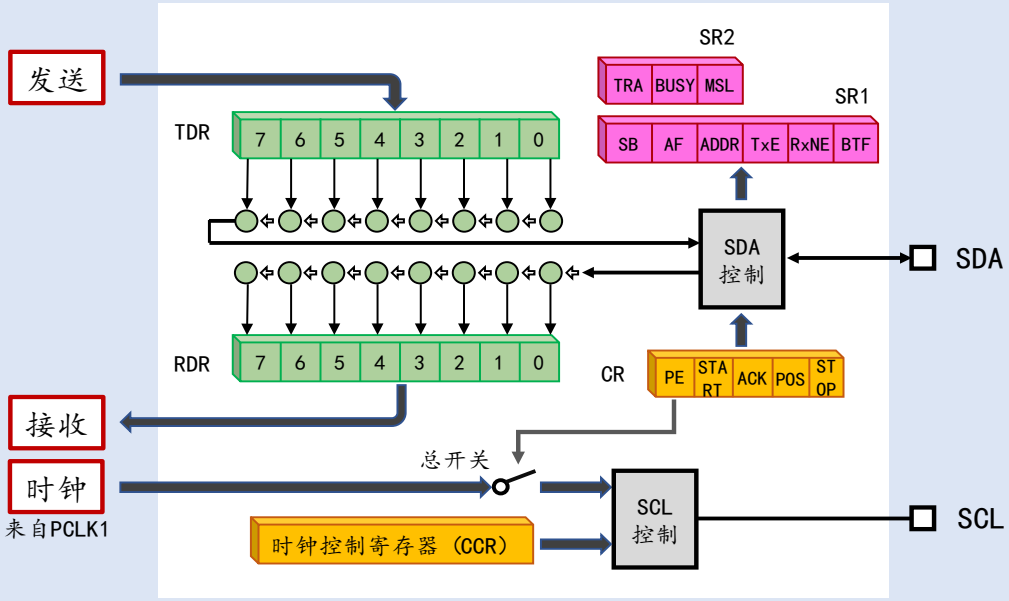


START – 用于产生一个起始位
写1产生起始位，产生后START归零

SB – 起始位发送完成
0 – 没有起始位产生
1 – 检测到了起始位



2.4.3.2. 编程接口和示例代码



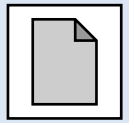
标准库编程接口：

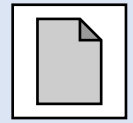
```
// @简介: 产生起始位
// @参数: NewState: ENABLE - 向START写1, DISABLE - 向START写0
void I2C_GenerateSTART(I2C_TypeDef *I2Cx, FunctionalState NewState)
```

示例代码：

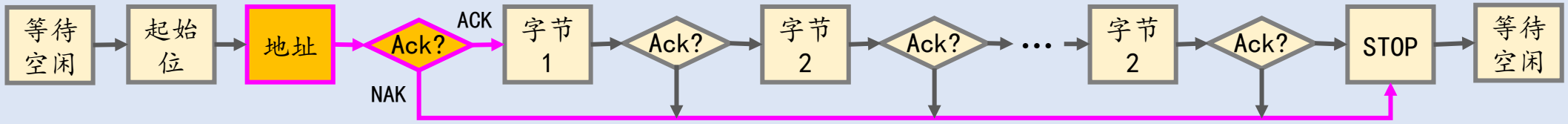
```
I2C_GenerateSTART(I2C1,ENABLE); // 发送起始位
while(I2C_GetFlagStatus(I2C1,I2C_FLAG_SB)==RESET); // 起始位已发送
```

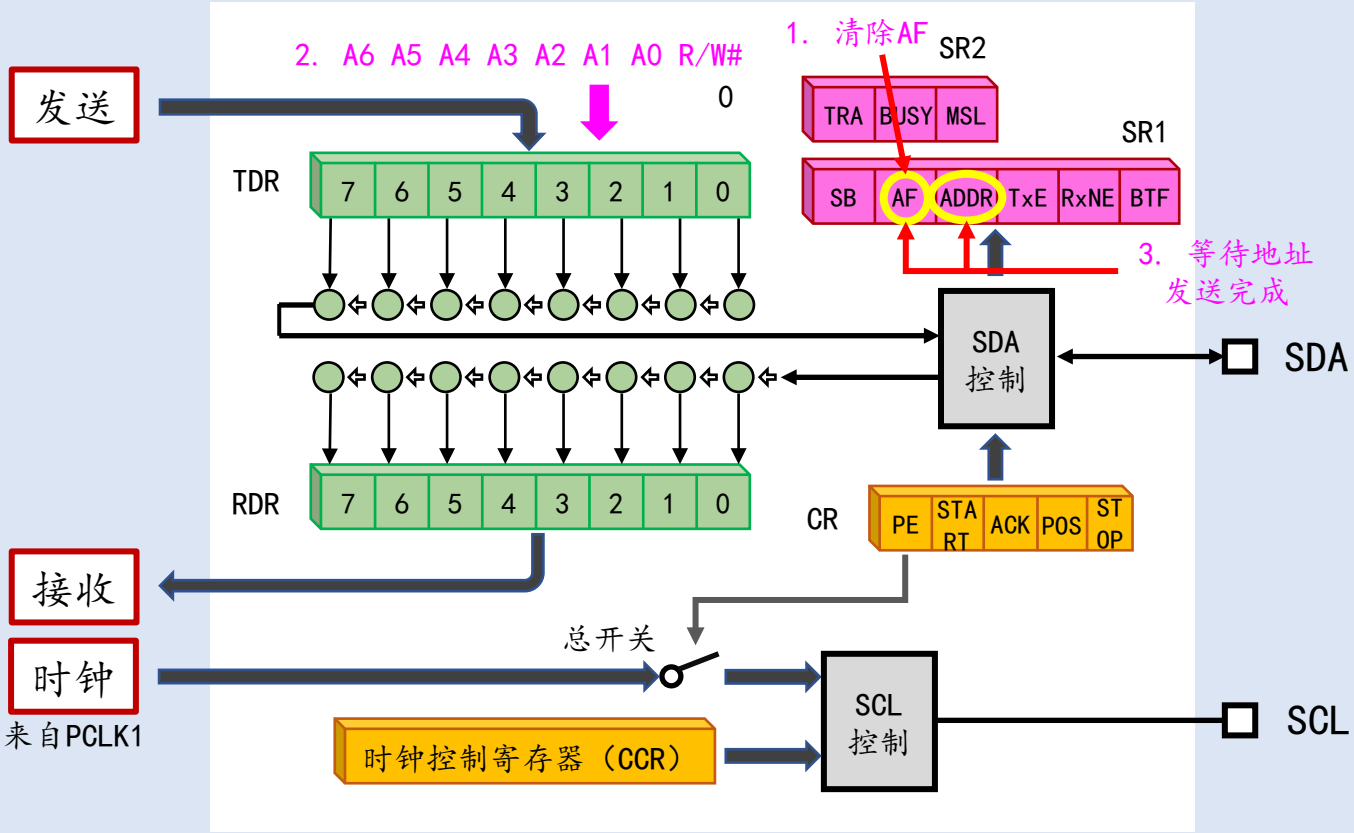
2.4.4. 发送地址 

2.4.4.1. 地址发送流程 

2.4.4.2. 编程接口和示例代码 

2.4.4.1. 地址发送流程

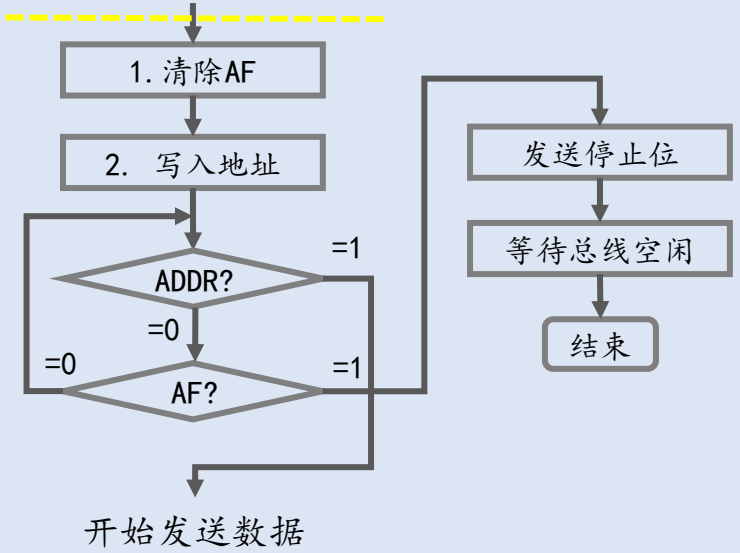




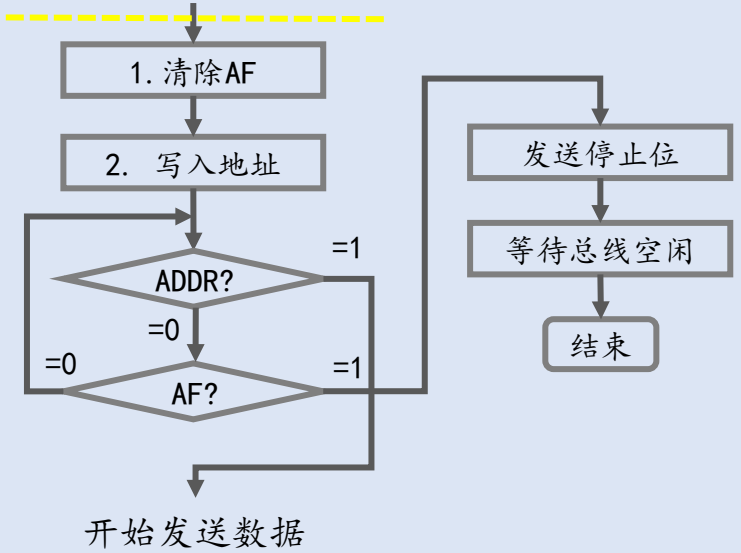
AF – (Acknowledge failure) 接收到了NAK

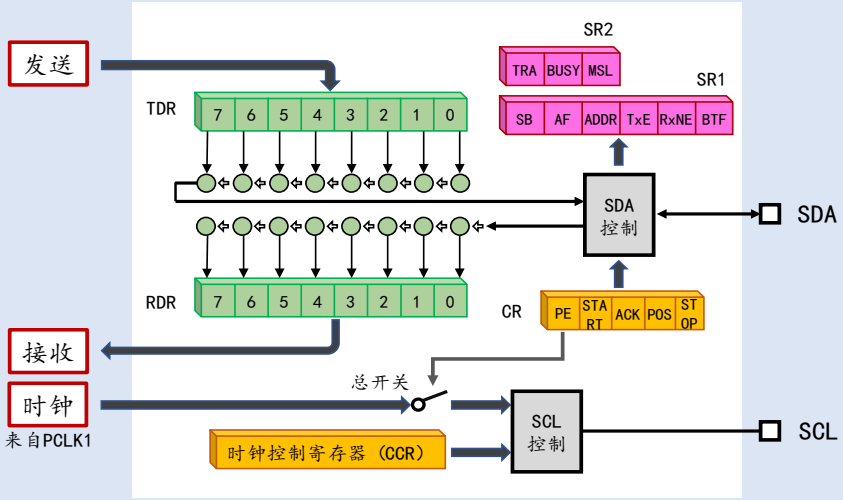
ADDR – (Address) 寻址成功

寻址成功 – ADDR=1 寻址失败 – AF=1



2.4.4.2. 编程接口和示例代码





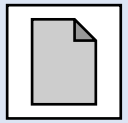

```
// @简介: 清除标志位
// @参数: I2C_FLAG 标志位的名称
//        I2C_FLAG_AF I2C_FLAG_BUSY I2C_FLAG_MSL I2C_FLAG_SB …
void I2C_ClearFlag(I2C_TypeDef *I2Cx, uint32_t I2C_FLAG)
// @简介: 发送数据（向TDR写值）
// @参数: Data: 要发送的数据
void I2C_SendData(I2C_TypeDef *I2Cx, uint8_t Data);
// @简介: 发送停止位
void I2C_GenerateStop(I2C_TypeDef *I2Cx);
```

```
  ErrorStatus ret = SUCCESS;

  I2C_ClearFlag(I2C1, I2C_FLAG_AF); // 清除AF
  I2C_SendData(I2C1, SlaveAddr & 0xfe); // 发送地址
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_ADDR)==RESET){ // 等待数据发送完成
    if(I2C_GetFlagStatus(I2C1, I2C_FLAG_AF)==SET){
      ret = ERROR;
      goto STOP;
  }}
  // 发送数据
  …
STOP:
  I2C_GenerateSTOP(I2C1); // 发送停止位
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BUSY)==SET); // 等待总线空闲
  return ret;
```

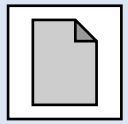
2.4.5. 发送数据

铁头山羊 TIETOUSHANYANG

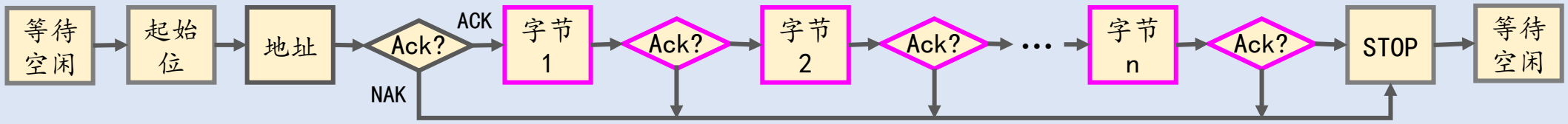
2.4.5.1. 数据发送流程

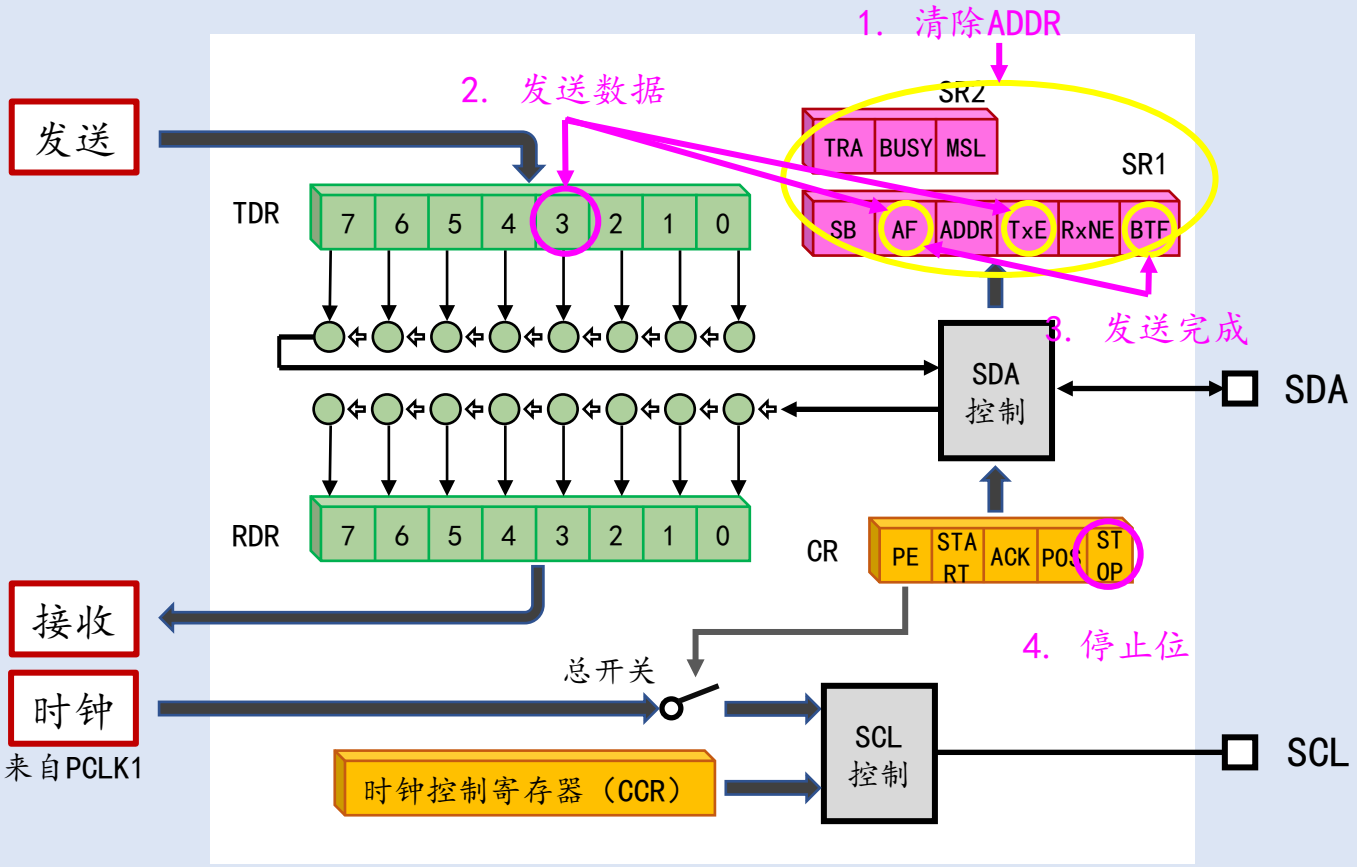


2.4.5.2. 编程接口和示例代码



2.4.5.1. 数据发送流程

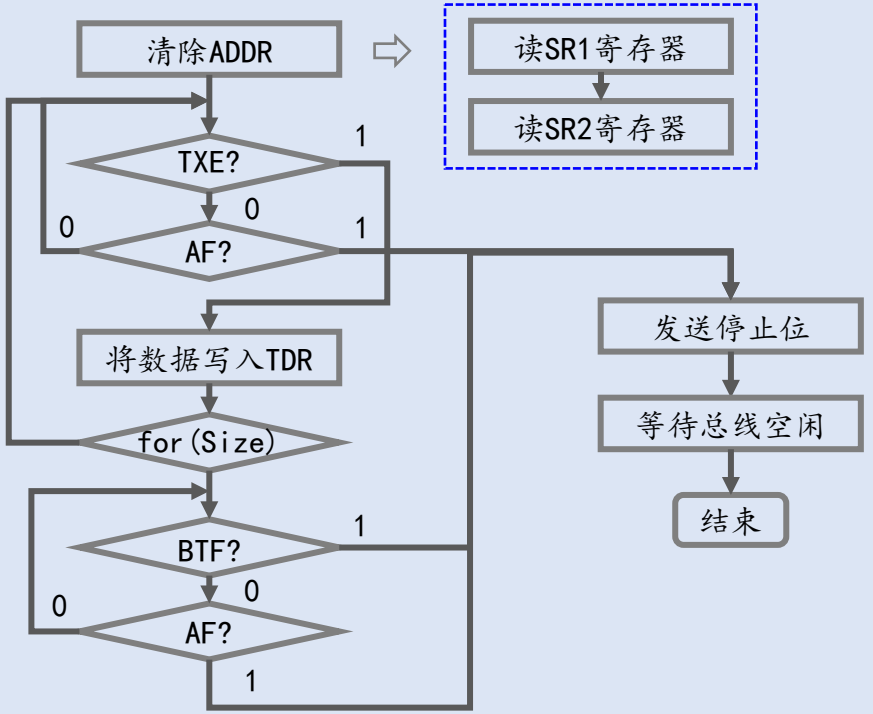




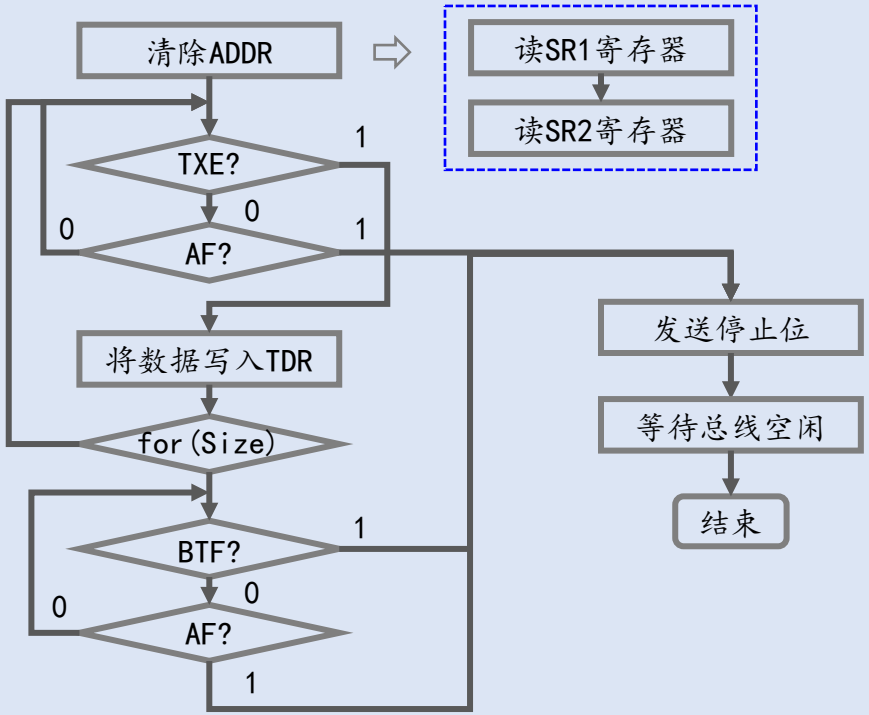
AF – (Ack Failure) 收到了NAK

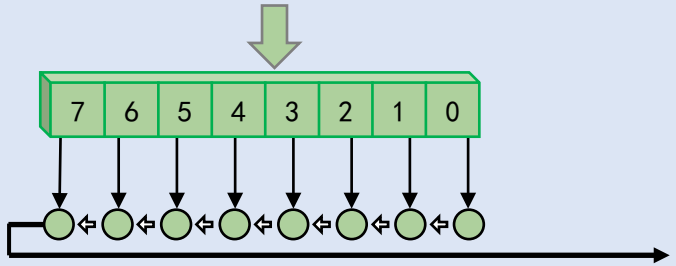
TXE – (TDR Empty) 发送数据寄存器空

BTF – (Byte Transmit Complete) 发送完成



2.4.4.2. 编程接口和示例代码






```
// @简介: 读取寄存器
// @参数: I2C_Register - 寄存器名称
//        I2C_CR1 I2C_CR2 … I2C_SR1 I2C_SR2 …
uint16_t I2C_ReadRegister(I2C_TypeDef *I2Cx, uint32_t I2C_Register)
```

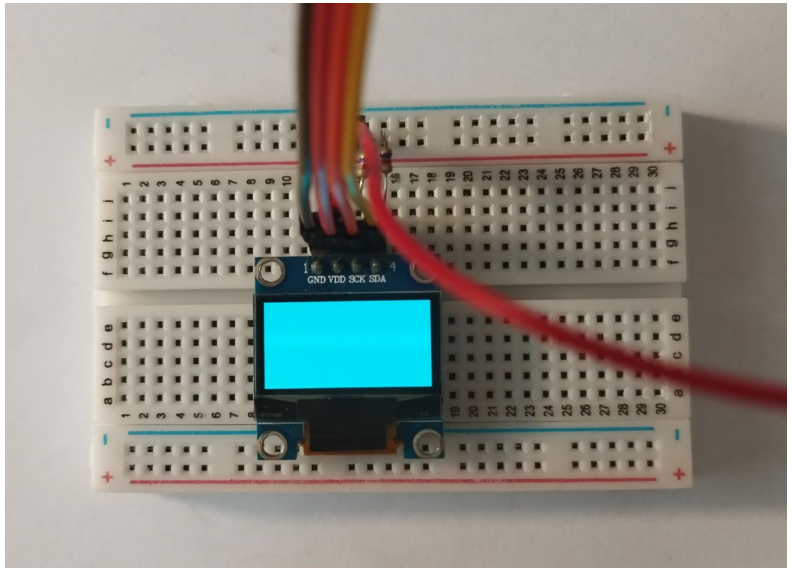
```
  I2C_ReadRegister(I2C1,I2C_Register_SR1); // 清除ADDR
  I2C_ReadRegister(I2C1,I2C_Register_SR2);
  for(i=0;i<Size;i++){
    // 等待TXE
    while(I2C_GetFlagStatus(I2C1,I2C_FLAG_TXE)==RESET){
      if(I2C_GetFlagStatus(I2C1,I2C_FLAG_AF)==SET){
        ret = ERROR;
        goto STOP;
    }}
    I2C_SendData(I2C1, pData[i]); // 发送数据
  }
  // 等待BTF
  while(I2C_GetFlagStatus(I2C1,I2C_FLAG_BTF)==RESET){
    if(I2C_GetFlagStatus(I2C1,I2C_FLAG_AF)==SET){
      ret = ERROR;
      goto STOP;
  }}
STOP:
  I2C_GenerateSTOP(I2C1);
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BUSY)==SET);
  return ret;
```

2.4.7. 测试

```
const uint8_t oled_init_command[] = {
  0x00, // Command Stream
  0xa8, 0x3f, // Set MUX Ratio
  0xd3, 0x00, // Set Display Offset
  0x40, // Set Display Start Line
  0xa0, // Set Segment re-map
  0xc0, // Set COM Output Scan Direction
  0xda, 0x02, // Set COM Pins hardware configuration
  0x81, 0x7f, // Set Contrast Control
  0xa5, // Enable Entire Display On
  0xa6, // Set Normal Display
  0xd5, 0x80, // Set OSC Frequency
  0x8d, 0x14, // Enable charge pump regulator
  0xaf, // Display on
};

App_I2C_Init();
App_I2C_MasterTransmit(0x78, oled_init_command, sizeof(oled_init_command)/sizeof(uint8_t));
```

全屏点亮

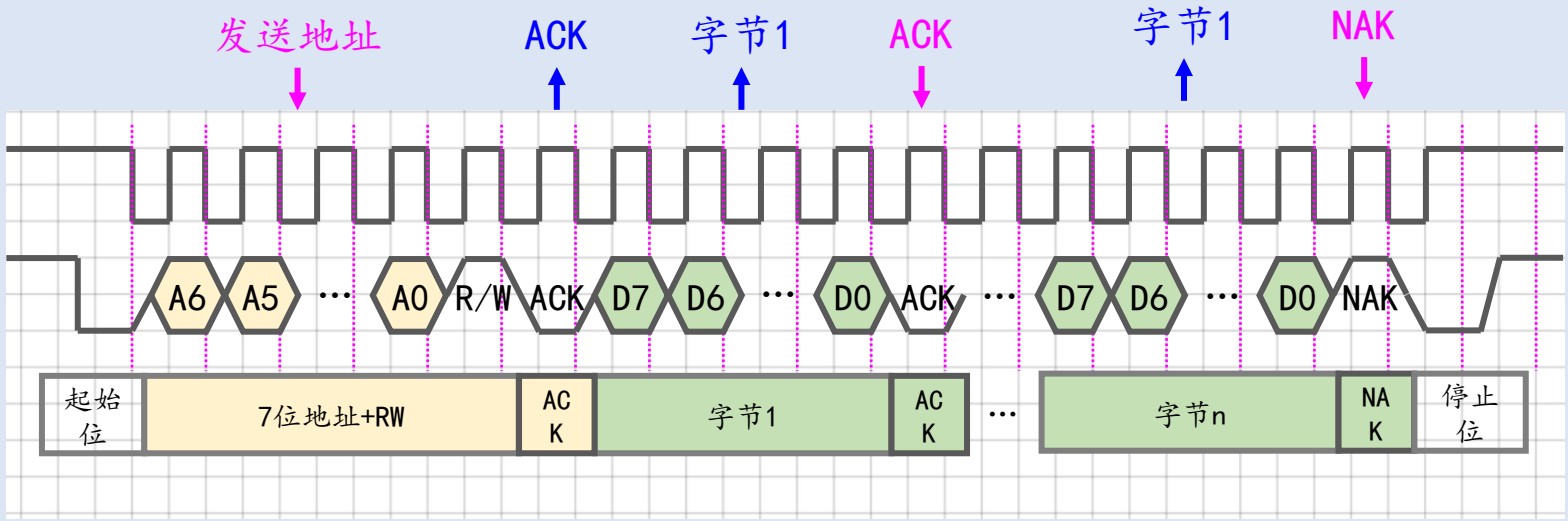
打开屏幕

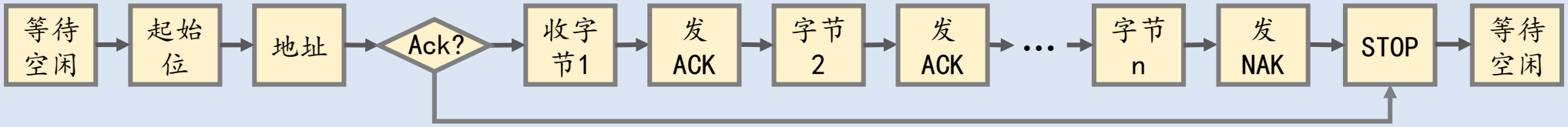


从机地址　　命令　　命令长度

2.5. 主机数据接收

2.5.1. 数据接收的流程



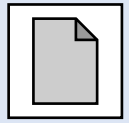



```
ErrorStatus App_I2C_MasterReceive(uint8_t SlaveAddr, uint8_t *pDataOut, uint16_t Size);
```

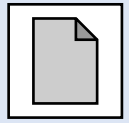
2.5.2. 双缓冲与数据接收

铁头山羊 TIETOUSHANYANG

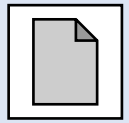
2.5.2.1. RXNE与BTF标志位



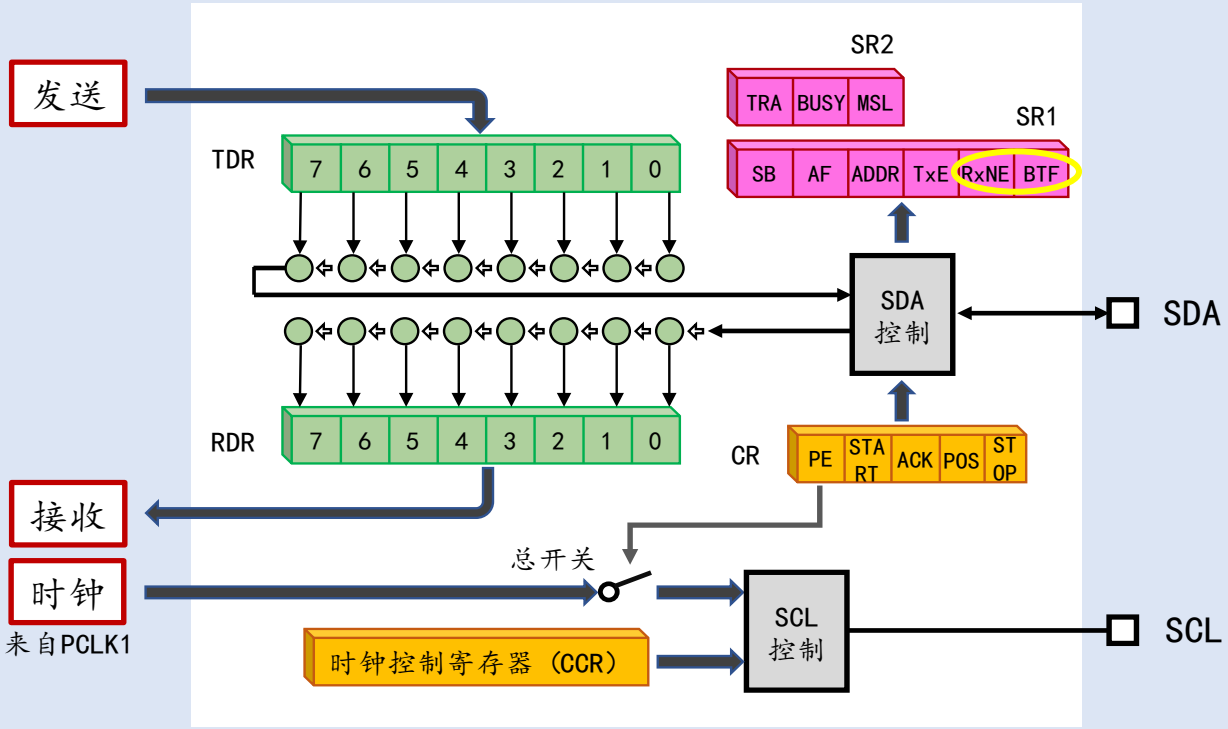
2.5.2.2. ACK和NAK的发送

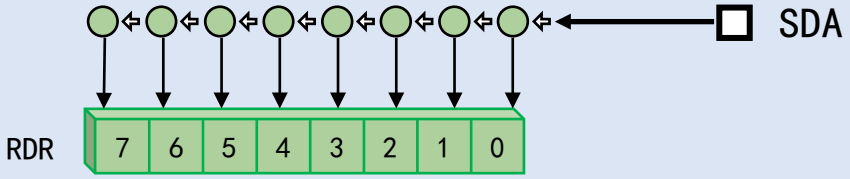


2.5.2.3. 停止位的发送



2.5.2.1. RXNE和BTF标志位

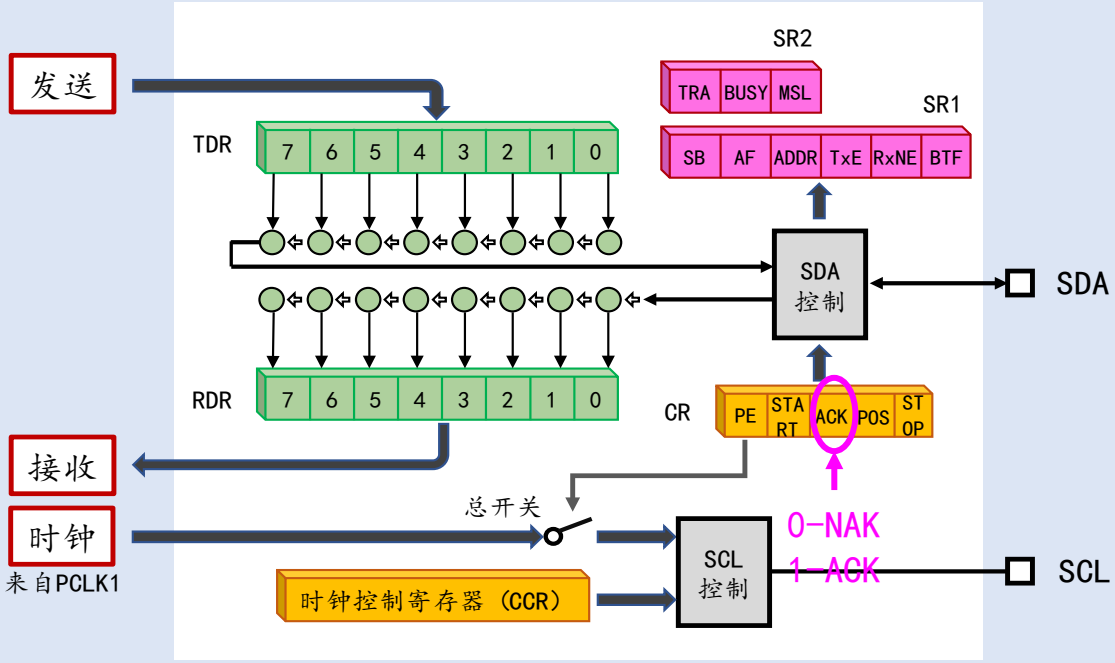


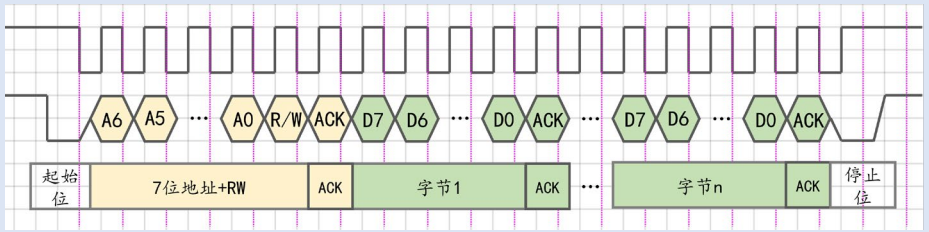


RXNE – 接收数据寄存器（RDR）非空

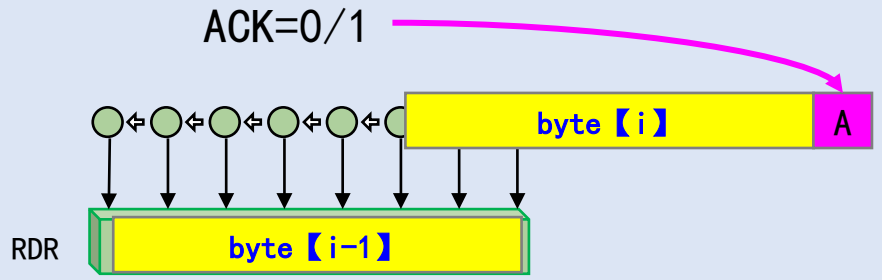
BTF – 传输完成 – 两级缓冲满

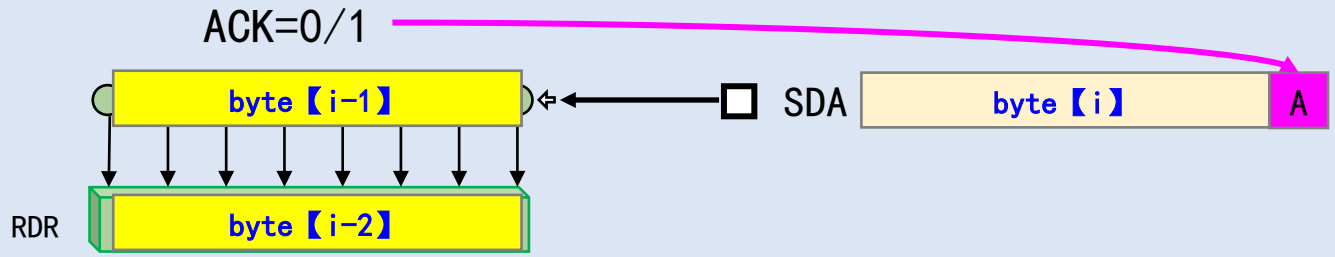
2.5.2.2. ACK和NAK的发送



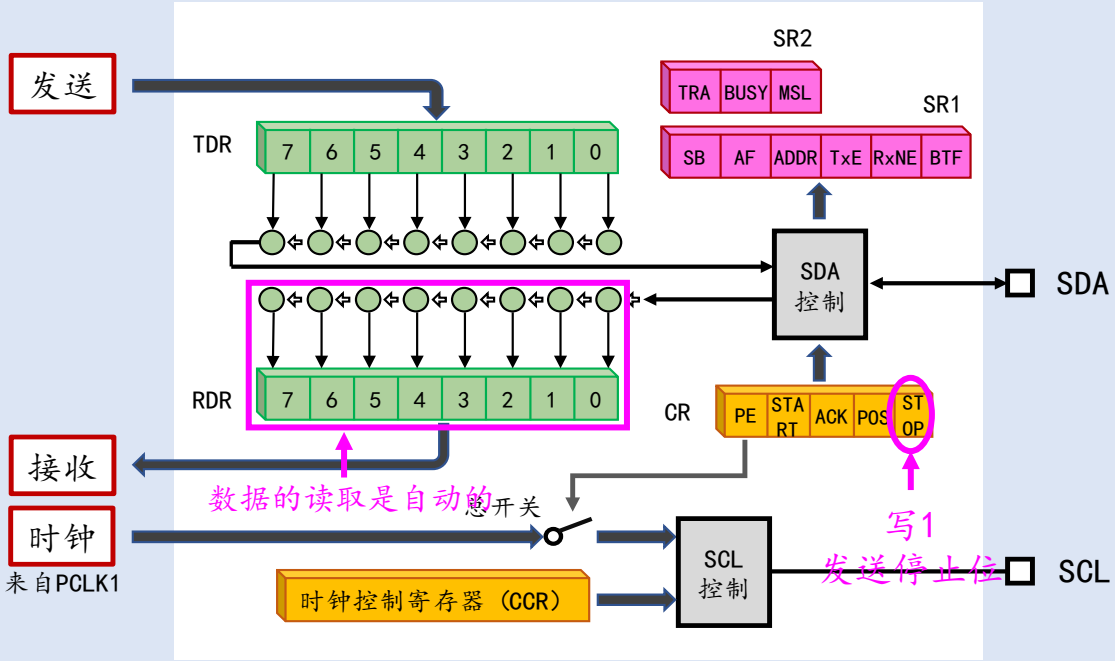


ACK影响正在接收或即将被接收的字节

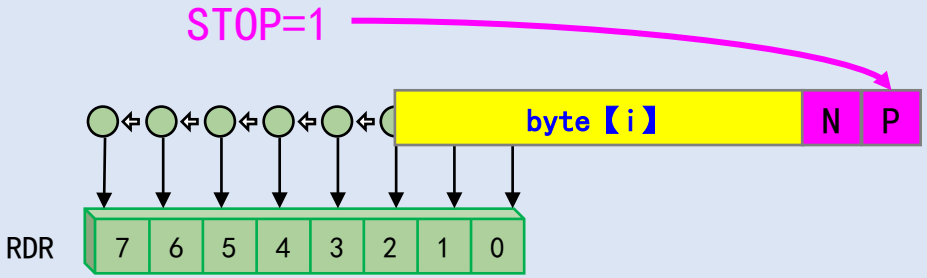


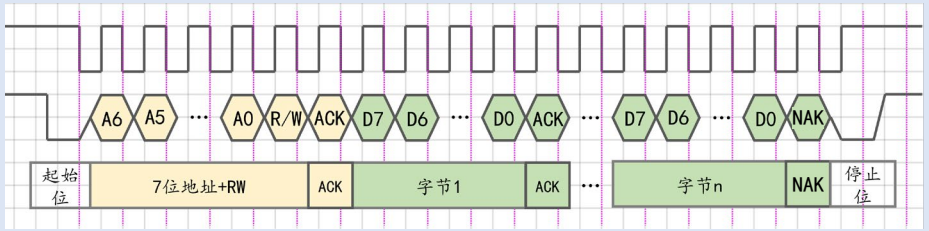


2.5.2.3. 停止位的发送

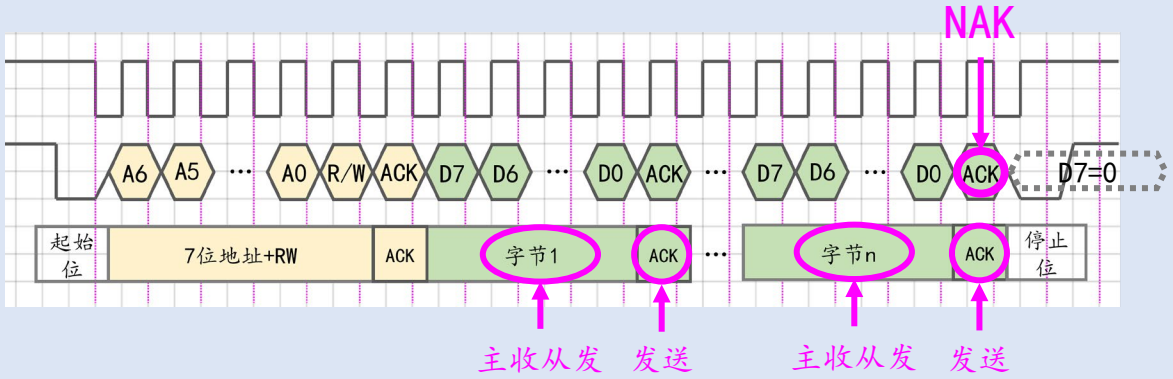


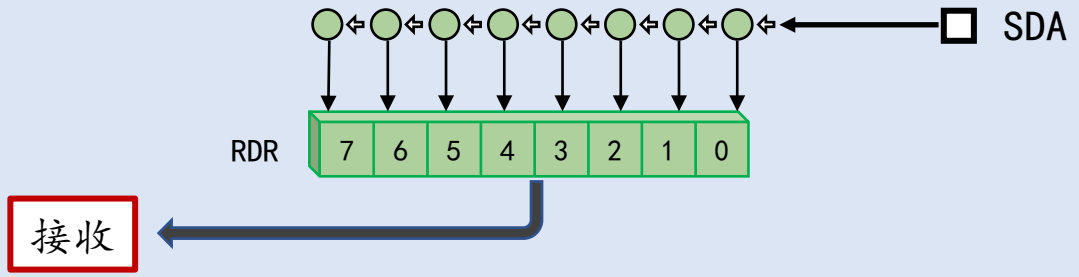
向STOP写1之后，会等到当前字节传输完成后发送停止位





2.5.1. I2C数据接收的复杂性





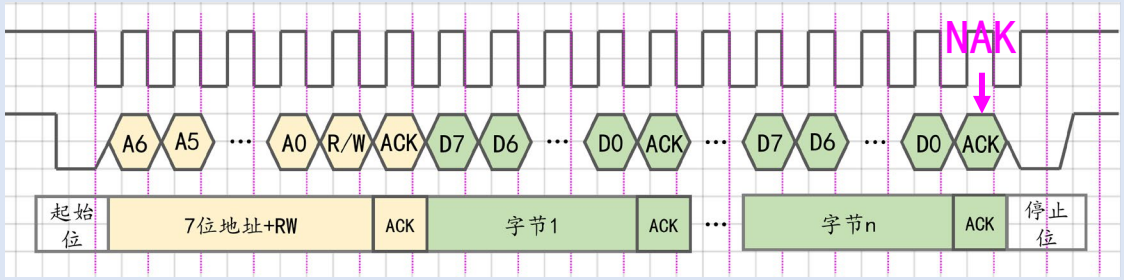
要在准确的位置发送NAK和STOP

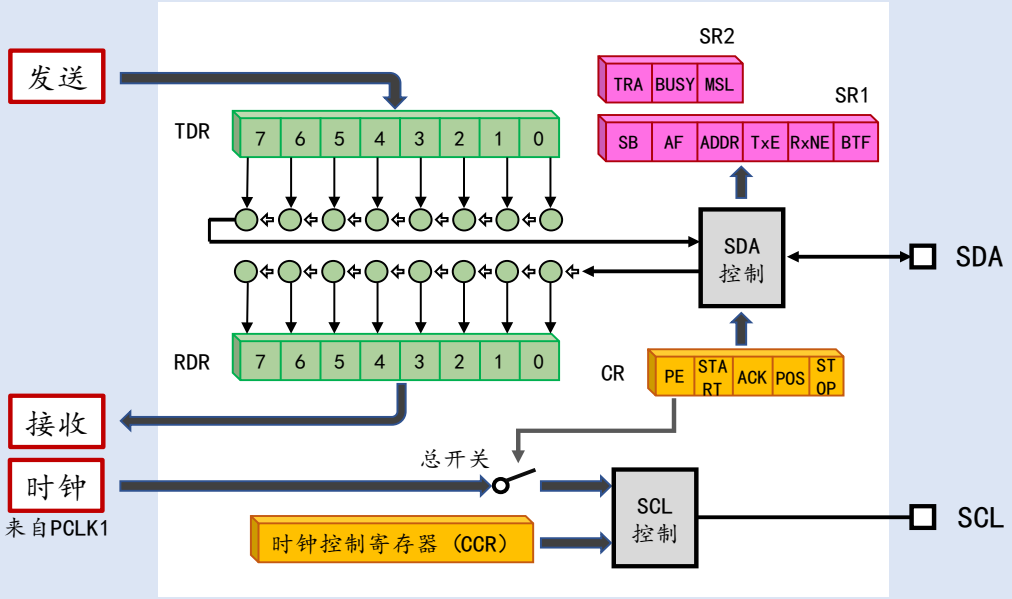
接收过程不可控
发送STOP之前
只要任意一个寄存器为空就会从I2C读取数据

复杂性

N=1　　N=2　　N>2

2.5.3. 发送起始位和地址



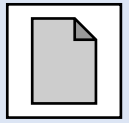



```
while(I2C_GetFlagStatus(I2C1,I2C_FLAG_BUSY)==SET);
// 发送起始位
I2C_GenerateSTART(I2C1,ENABLE);
while(I2C_GetFlagStatus(I2C1,I2C_FLAG_SB)==SET);
// 发送从机地址
I2C_ClearFlag(I2C1, I2C_FLAG_AF);        R/W#=1
I2C_SendData(I2C1, SlaveAddr | 0x01);
while(I2C_GetFlagStatus(I2C1, I2C_FLAG_ADDR)==RESET){
  if(I2C_GetFlagStatus(I2C1, I2C_FLAG_AF)==SET){
    I2C_GenerateSTOP(I2C1);
    while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BUSY)==SET);
    return Error;
}}
// 接收数据
...
```

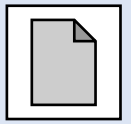
2.5.4. N=1

铁头山羊 TIETOUSHANYANG

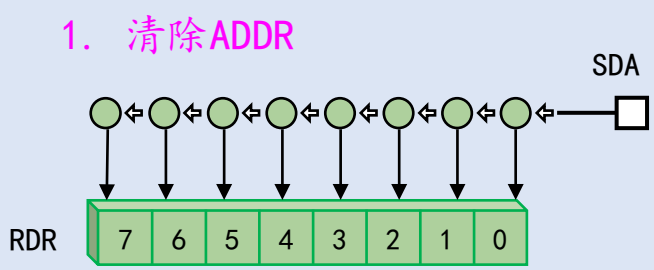
2.5.4.1. N=1的接收过程

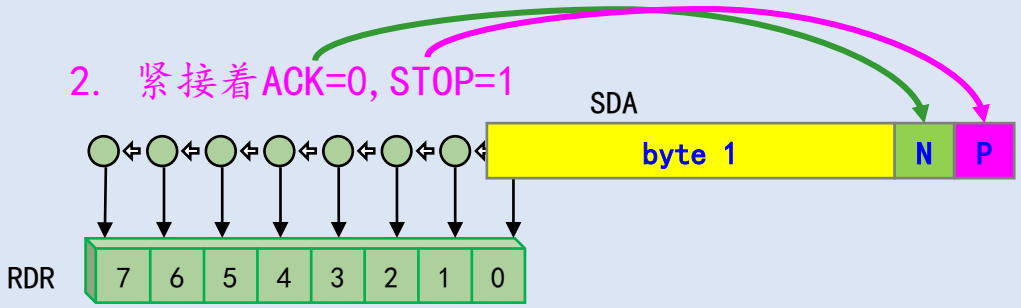


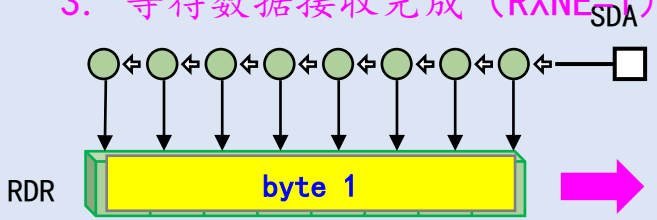
2.5.4.2. 编程接口和示例代码

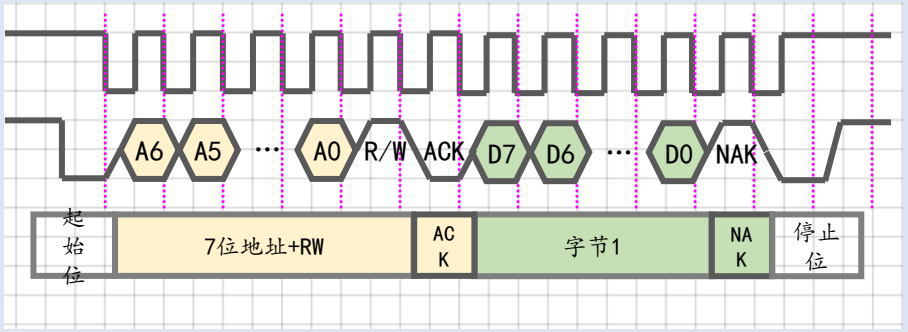


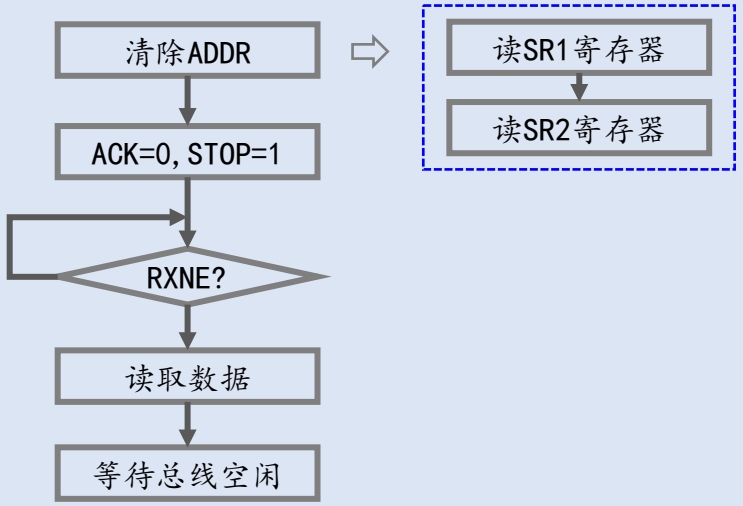
2.5.4.1. N=1的接收过程



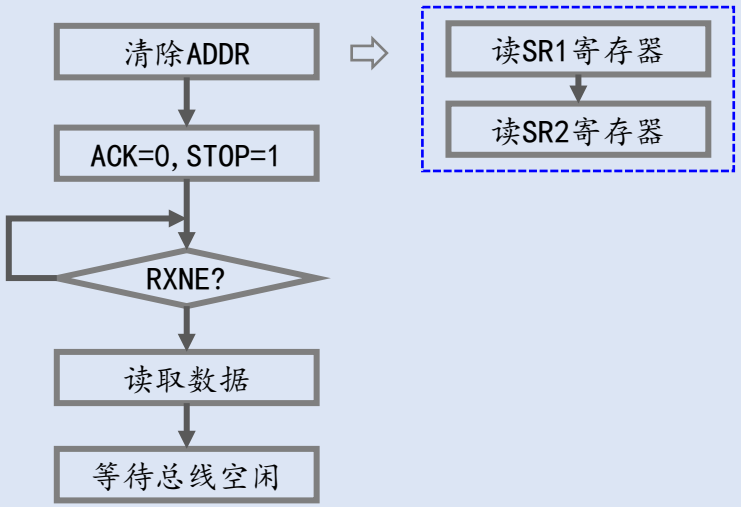








2.5.4.2. 编程接口和示例代码



编程接口：

```
// @简介：设置ACK标志位
// @参数：NewState：ENABLE - ACK=1，DISABLE - ACK=0
void I2C_AcknowledgeConfig(I2C_TypeDef *I2Cx, FunctionalState NewState)
// @简介：从RDR读取数据
// @返回：读取到的一个字节
uint8_t I2C_ReceiveData(I2C_TypeDef* I2Cx)
```

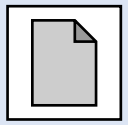
示例代码：

```
if(Size==1){
  // #1. 清除ADDR
  I2C_ReadRegister(I2C1,I2C_Reigster_SR1);
  I2C_ReadRegister(I2C1,I2C_Reigster_SR2);
  // #2. ACK=0, STOP=1
  I2C_AcknowledgeConfig(I2C1, DISABLE);
  I2C_GenerateStop(I2C1, ENABLE);
  // #3. 等待RXNE
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_RXNE)==RESET);
  // #4. 读取数据
  pDataOut[0] = I2C_ReceiveData(I2C1);
  // #5. 等待总线空闲
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BUSY)==RESET);
  return SUCCESS;
}
```

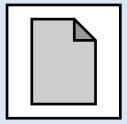
2.5.5. N=2

铁头山羊 TIETOUSHANYANG

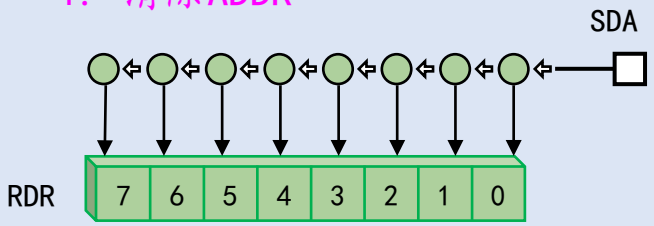
2.5.5.1. N=2的接收过程

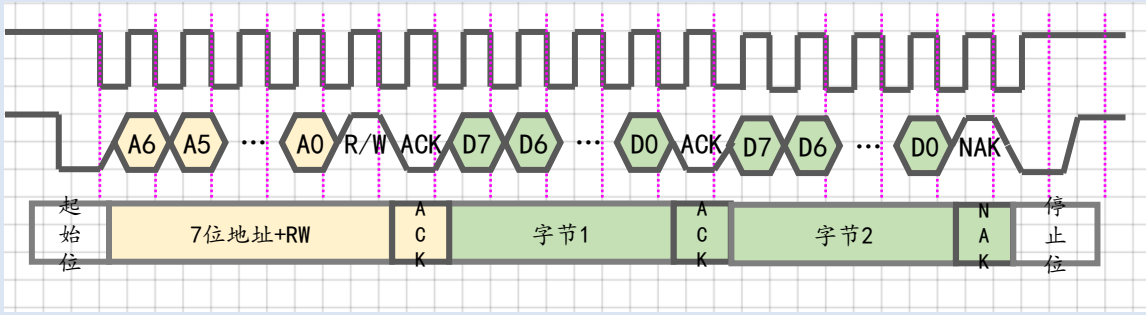


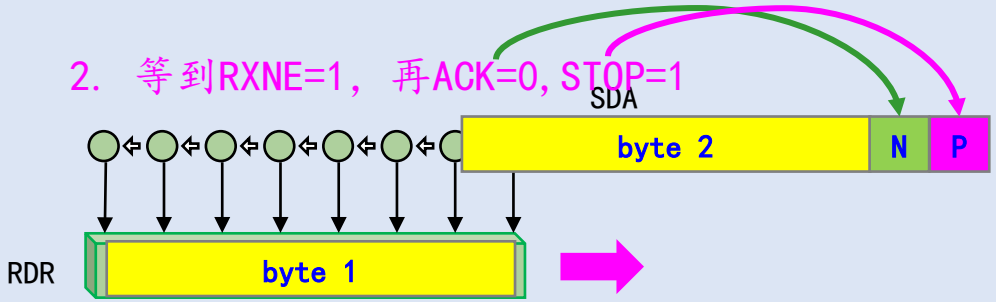
2.5.5.2. 示例代码

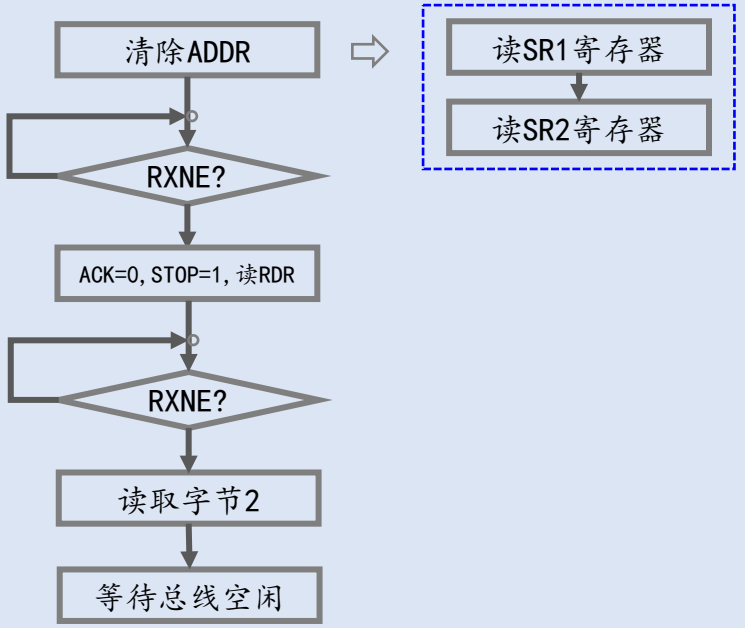


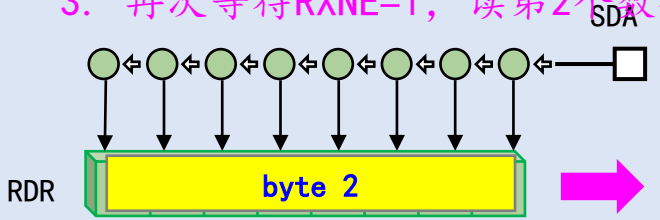
2.5.5.1. N=2的接收过程



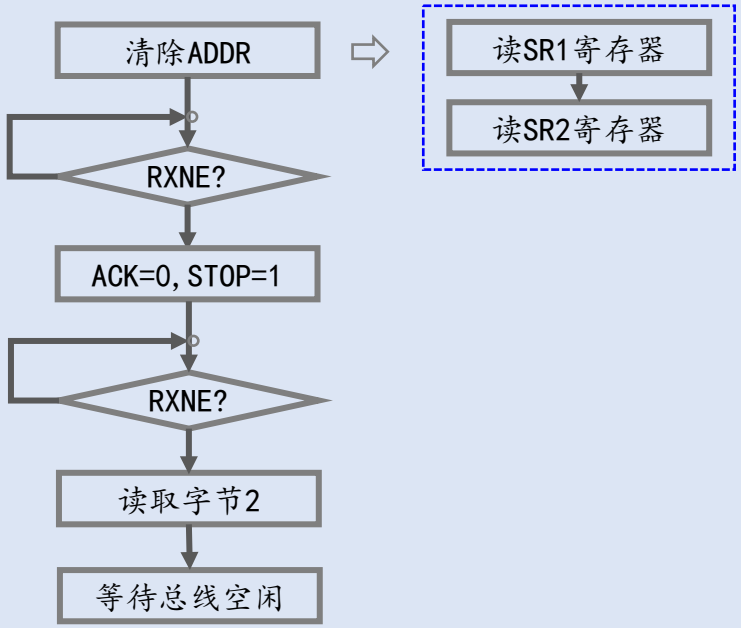








2.5.5.2. 示例代码



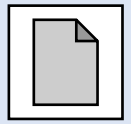
示例代码：

```
if(Size==2){
  // #1. 清除ADDR
  I2C_ReadRegister(I2C1,I2C_Reigster_SR1);
  I2C_ReadRegister(I2C1,I2C_Reigster_SR2);
  // #2. 等待RXNE=1，读取第1个数据，ACK=0，STOP=1
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_RXNE)==RESET);
  pDataOut[0] = I2C_ReceiveData(I2C1);
  I2C_AcknowledgeConfig(I2C1, DISABLE);
  I2C_GenerateStop(I2C1, ENABLE);
  // #3. 等待RXNE，读取第2个数据
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_RXNE)==RESET);
  pDataOut[1] = I2C_ReceiveData(I2C1);
  // #4. 等待总线空闲
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BUSY)==RESET);
  return SUCCESS;
}
```

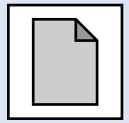
2.5.6. N>2

铁头山羊 TIETOUSHANYANG

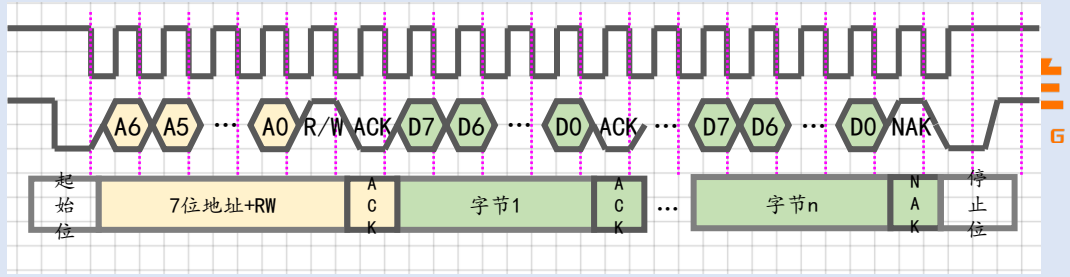
2.5.6.1. N>2的接收过程



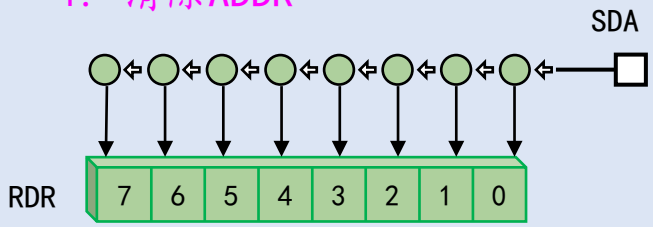
2.5.6.2. 示例代码



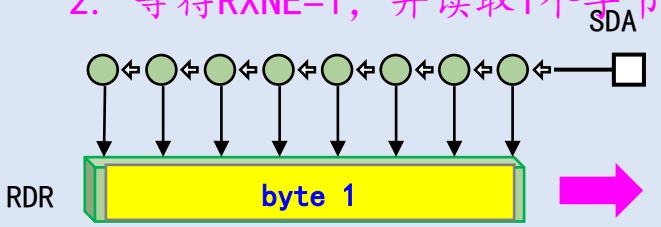
2.5.6.1. N>2的接收过程



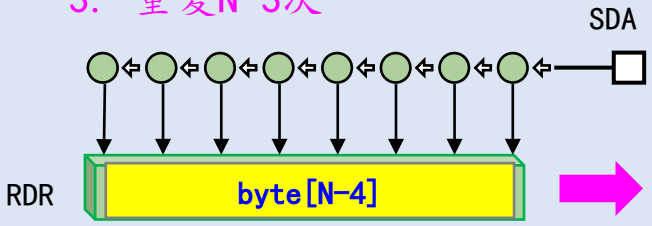
1. 清除ADDR



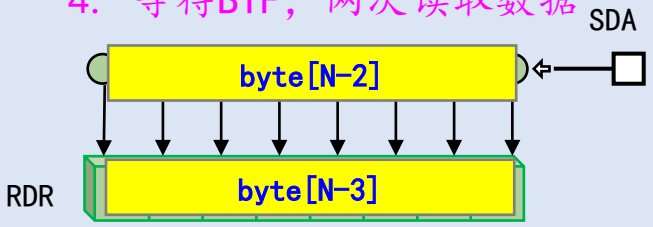
2. 等待RXNE=1，并读取1个字节



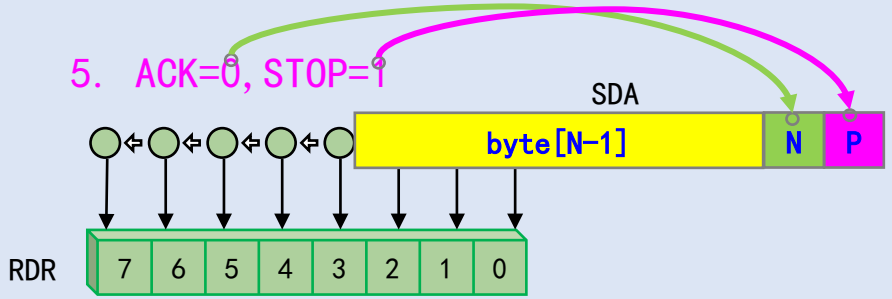
3. 重复N-3次



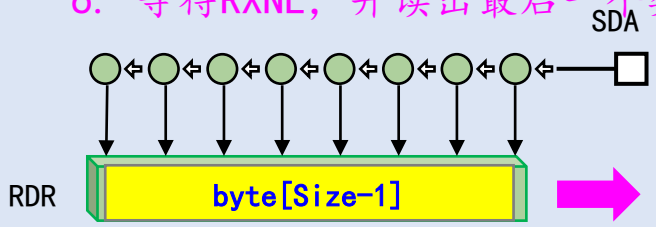
4. 等待BTF，两次读取数据

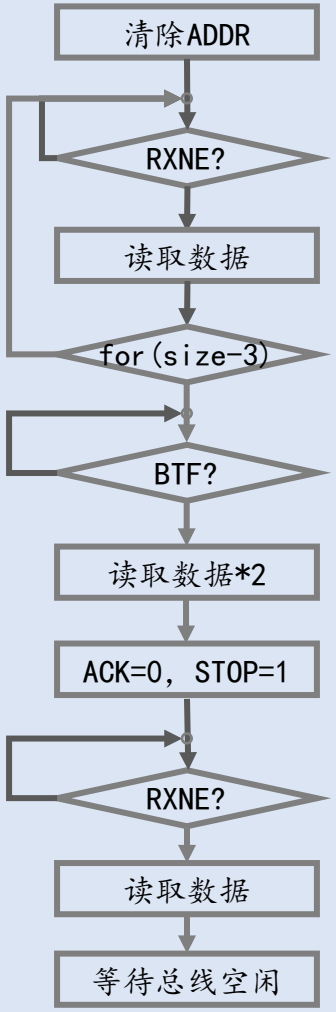


5. ACK=0,STOP=1

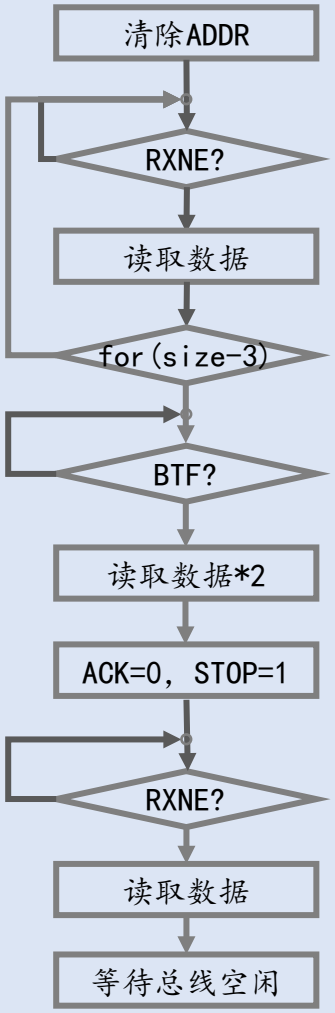


6. 等待RXNE，并读出最后一个数据





2.5.6.2. 示例代码



示例代码：

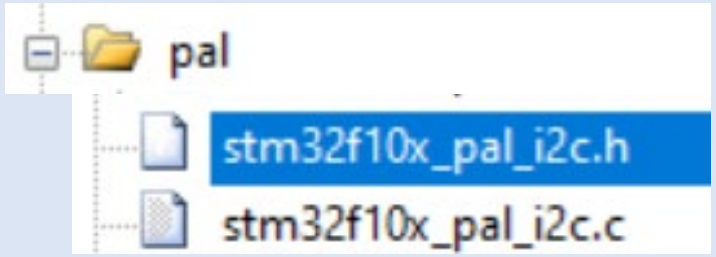
```
if(Size>2){
  // #1. 清除ADDR
  I2C_ReadRegister(I2C1,I2C_Reigster_SR1);
  I2C_ReadRegister(I2C1,I2C_Reigster_SR2);
  // #2. 读取前Size-3个数据
  for(i=0;i<Size-3;i++){
    while(I2C_GetFlagStatus(I2C1, I2C_FLAG_RXNE)==RESET);
    pDataOut[i]=I2C_ReceiveData(I2C1);
  }
  // #3. 等待BTF，连续读取两次数据
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BTF)==RESET);
  pDataOut[Size-3]=I2C_ReceiveData(I2C1);
  pDataOut[Size-2]=I2C_ReceiveData(I2C1);
  // #4. 等待RXNE，读取最后一个数据
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_RXNE)==RESET);
  pDataOut[Size-1]=I2C_ReceiveData(I2C1);

  // #5. 等待总线空闲
  while(I2C_GetFlagStatus(I2C1, I2C_FLAG_BUSY)==RESET);
  return SUCCESS;
}
```

2.5.7. 测试

D7 D6 D5 D4 D3 D2 D1 D0　D6=0 - 显示器开，D6=1 - 显示器关

```
const uint8_t oled_display_off_command[] = {0x00, 0xae}; // 用于熄灭屏幕
const uint8_t oled_display_on_command[] = {0x00, 0xaf};  // 用于点亮屏幕

uint8_t buffer[8];
App_I2C_MasterTransmit(0x78, oled_display_off_command,…); // 熄灭屏幕
App_I2C_MasterReceive(0x78, buffer, 1);  // 读取1个字节
App_I2C_MasterReceive(0x78, buffer, 2);  // 读取2个字节        D6=1
App_I2C_MasterReceive(0x78, buffer, 8);  // 读取8个字节

App_I2C_MasterTransmit(0x78, oled_display_on_command,…); // 点亮屏幕
App_I2C_MasterReceive(0x78, buffer, 1);  // 读取1个字节
App_I2C_MasterReceive(0x78, buffer, 2);  // 读取2个字节        D6=0
App_I2C_MasterReceive(0x78, buffer, 8);  // 读取8个字节
```

2.6. PAL库编程接口




```
ErrorStatus PAL_I2C_Init(PalI2C_HandleTypeDef *Handle);
ErrorStatus PAL_I2C_MasterTransmit(PalI2C_HandleTypeDef *Handle, uint16_t SlaveAddr, const uint8_t *pData, uint16_t Size);
ErrorStatus PAL_I2C_MasterReceive(PalI2C_HandleTypeDef *Handle, uint16_t SlaveAddr, uint8_t *pBufferOut, uint16_t Size);
```

示例代码

```
static PalI2C_HandleTypeDef hi2c1;

hi2c1.Init.I2Cx = I2C1;
hi2c1.Init.I2C_ClockSpeed = 400000; // 波特率
hi2c1.Init.I2C_DutyCycle = I2C_DutyCycle_2; // Fm占空比
// hi2c1.Init.Advanced.Remap = 1; // 重映射

PAL_I2C_Init(&hi2c1);
```

2.7. 随堂测验

题目1：STM32里有几个I2C接口，对应的IO引脚在什么位置？

题目2：I2C有哪几种速度模式？

题目3：Fm下2:1和16:9的占空比代表什么含义？

题目4：如何清除ADDR标志位？

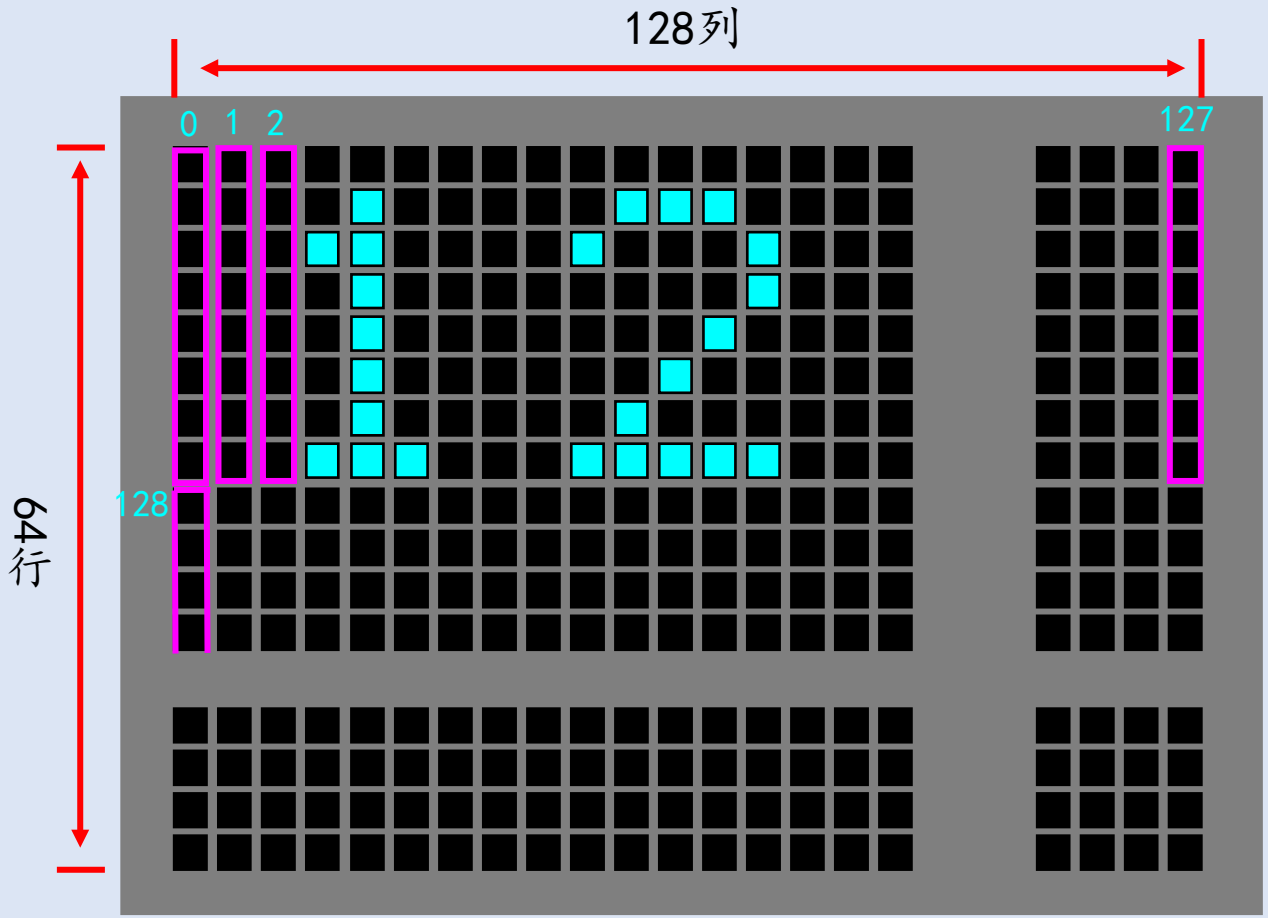
题目5：数据接收的最后一个字节为什么要发送NAK？

第三节、OLED显示器

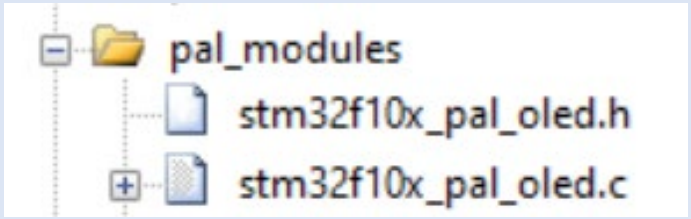
铁头山羊 TIETOUSHANYANG

3.1. OLED模块的工作原理

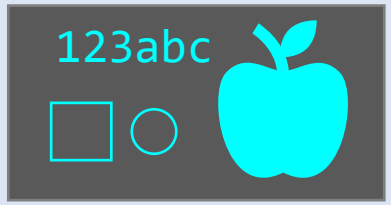
SSD1306.PDF



3.2. OLED驱动简介



uint8_t buffer[1024]

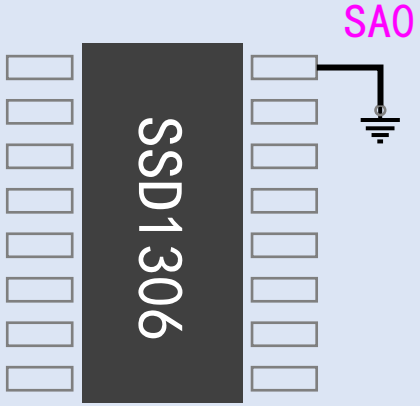


I2C总线

PAL_OLED_SendBuffer(…)

PAL_OLED_DrawLine(…)
PAL_OLED_DrawRect(…)
PAL_OLED_DrawStr(…)
PAL_OLED_DrawCircle(…)
PAL_OLED_DrawBitmap(…)

3.3. 初始化

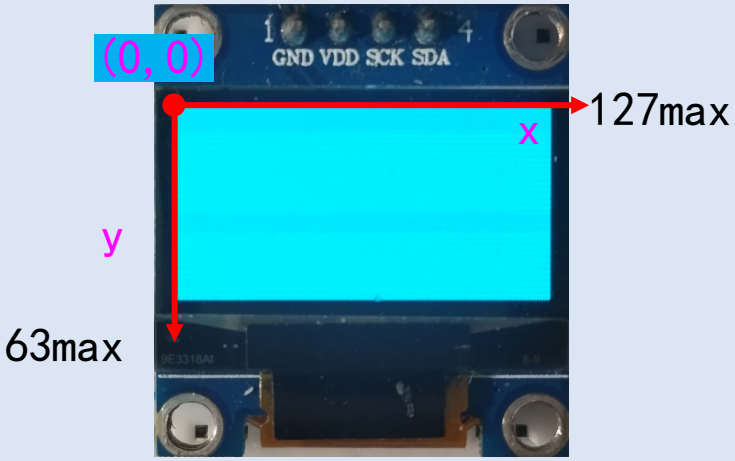


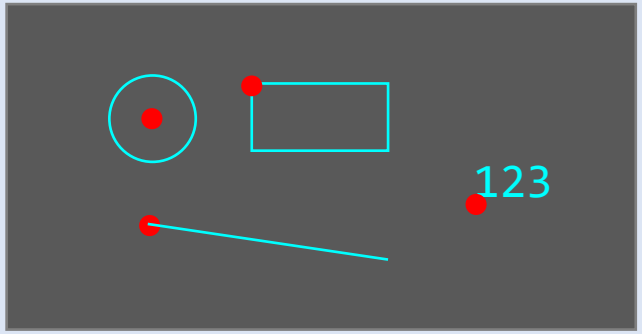
从机地址=0111 10(SA0)(RW#)

```
// @简介: 初始化OLED显示器
ErrorStatus PAL_OLED_Init(PalOled_HandleTypeDef *Handle);


static PalI2C_HandleTypeDef hi2c1; // I2C句柄
static PalOled_HandleTypeDef holed; // 屏幕句柄
// 初始化I2C总线
hi2c1.Init.I2Cx = I2C1;
hi2c1.Init.I2C_ClockSpeed = 400000;
hi2c1.Init.I2C_DutyCycle = I2C_DutyCycle_2;
PAL_I2C_Init(&hi2c1);
// 初始化屏幕
holed.Init.hi2c = &hi2c1; // I2C句柄
holed.Init.SA0 = RESET;
PAL_OLED_Init(&holed);
```

3.4. 绘制基本形状 

3.4.1. 坐标系和光标






```
// @简介：将光标设置到坐标(x,y)处
PAL_OLED_SetCursor(…,int16_t x, int16_t y);
// @简介：设置X坐标
PAL_OLED_SetCursorX(…,int16_t x);
// @简介：设置Y坐标
PAL_OLED_SetCursorY(…,int16_t y);
// @简介：在X和Y方向上移动光标
PAL_OLED_MoveCursor(…,int16_t dX, int16_t dY);
// @简介：仅在X方向上移动光标
PAL_OLED_MoveCursorX(…,int16_t dX);
// @简介：仅在Y方向上移动光标
PAL_OLED_MoveCursorY(…,int16_t dY);
// @简介：获取当前的X坐标
PAL_OLED_GetCursorX(…);
// @简介：获取当前的Y坐标
PAL_OLED_GetCursorY(…);
```

```
    SetCursor(9,3);
    MoveCursor(30,12);
    MoveCursorX(21);
    MoveCursorY(-10);
x = GetCursorX();
y = GetCursorY();
```

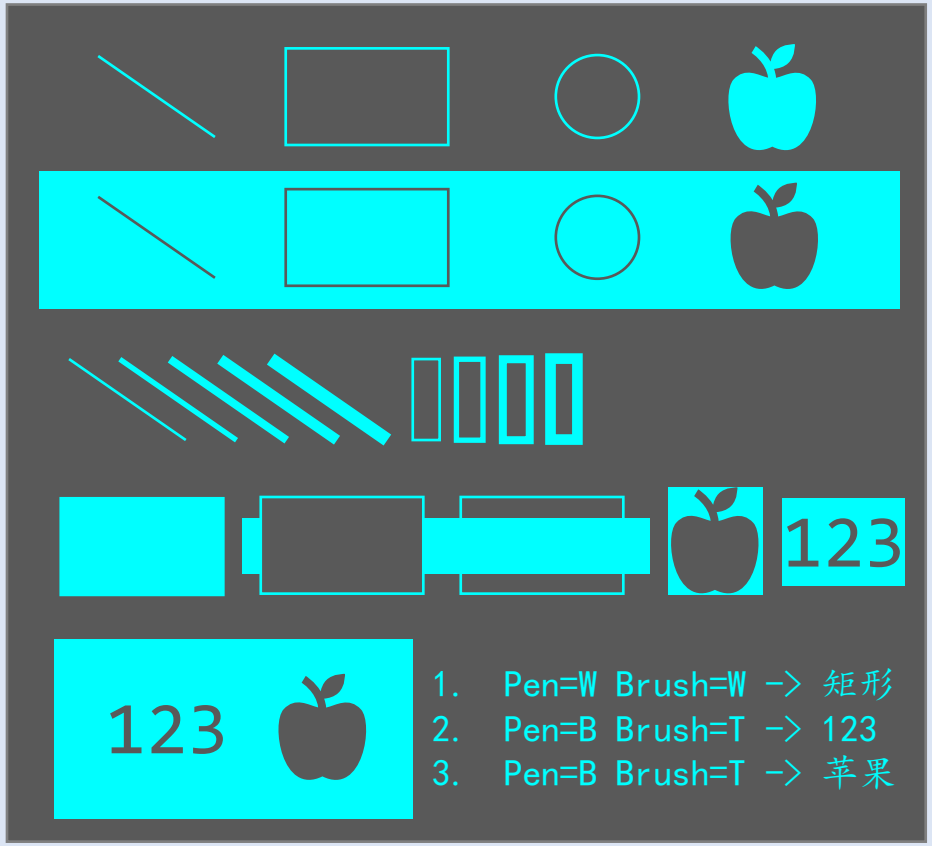
3.4.2. 画笔和画刷

白色画笔

黑色画笔

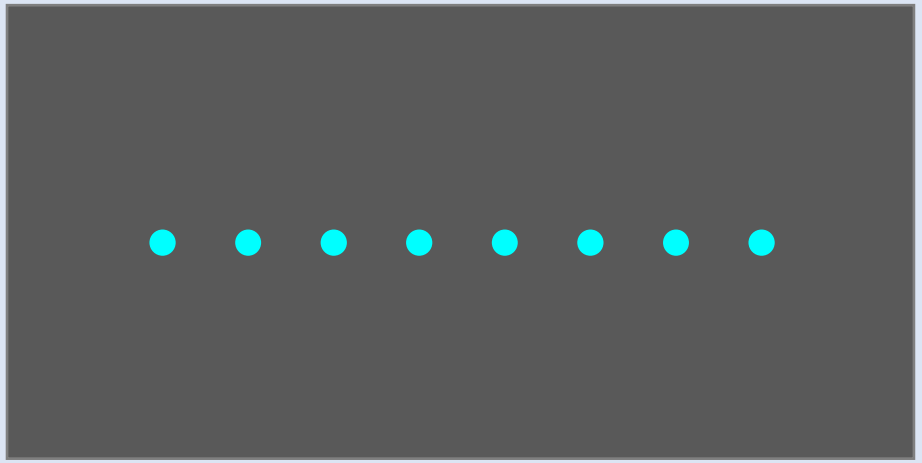
不同粗细

不同画刷




```
// @简介: 设置画笔，影响线条的颜色或者前景色
// @参数: PenColor - 画笔颜色
//                   PEN_COLOR_TRANSPARENT 透明
//                   PEN_COLOR_BLACK 黑色
//                   PEN_COLOR_WHITE 白色
// @参数: Width - 画笔宽度
PAL_OLED_SetPen(…,uint8_t PenColor, uint8_t Width);

// @简介: 设置画刷，影响填充颜色或者背景色
// @参数: BrushColor - 画刷颜色
//                     BRUSH_COLOR_TRANSPARENT 透明
//                     BRUSH_COLOR_WHITE 白色
//                     BRUSH_COLOR_BLACK 黑色
PAL_OLED_SetBrush(…,uint8_t BrushColor);
```

3.4.3. 画点

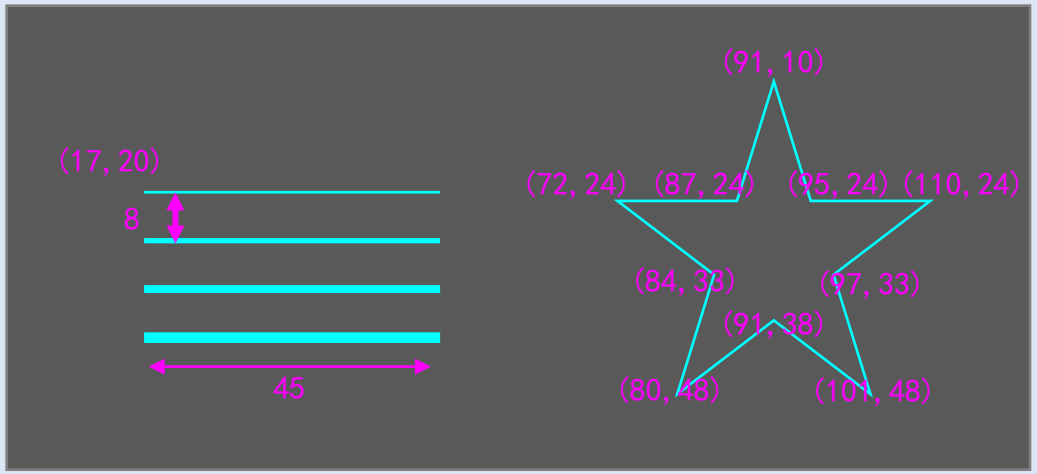


```
// @简介：在光标所在位置画点
void PAL_OLED_DrawDot(PalOLED_HandleTypeDef *Handle);
```

示例代码：

```
PAL_OLED_Clear(&holed);

PAL_OLED_SetPen(&holed, PEN_COLOR_WHITE, 3);
PAL_OLED_SetCursor(&holed, 29, 32);
PAL_OLED_DrawDot(&holed);
for(i=1;i<8;i++)
{
  PAL_OLED_MoveCursorX(&holed, 10);
  PAL_OLED_DrawDot(&holed);
}

PAL_OLED_SendBuffer(&holed);
```

3.4.4. 画线

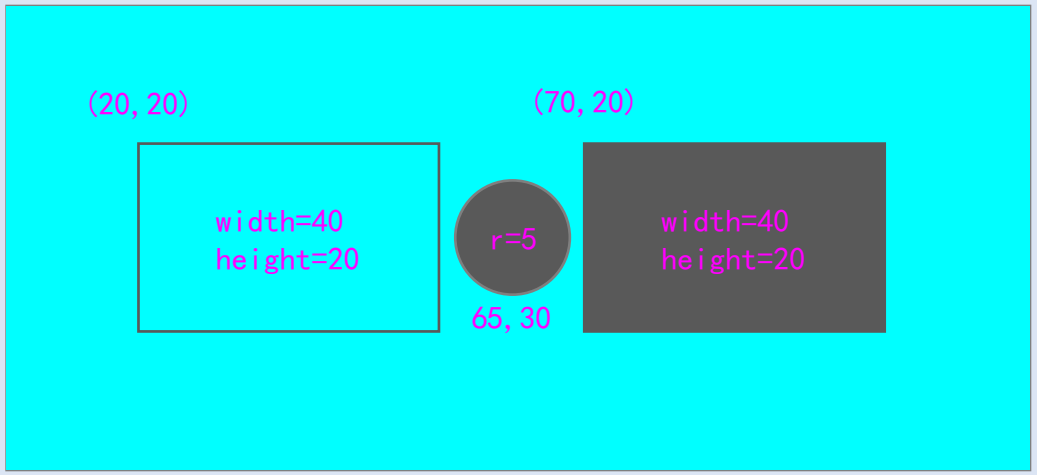



```
// @简介: 以光标为起点以(x,y)为终点绘制直线
void PAL_OLED_DrawLine(…,int16_t x, int16_t y);
// @简介: 以光标为起点以(x,y)为终点绘制直线, 绘制完成后
          移动光标到(x,y)处
void PAL_OLED_LineTo(…,int16_t x, int16_t y);
```

```
PAL_OLED_SetCursor(&holed, 17,20);
PAL_OLED_SetPen(&holed, PEN_COLOR_WHITE, 1);
PAL_OLED_DrawLine(&holed, GetCursorX(&holed)+45, GetCursorY(&holed));

PAL_OLED_MoveCursor(&holed, 0, 8);
PAL_OLED_SetPen(&holed, PEN_COLOR_WHITE, 2);
PAL_OLED_DrawLine(&holed, GetCursorX(&holed)+45, GetCursorY(&holed));

PAL_OLED_MoveCursor(&holed, 0, 8);
PAL_OLED_SetPen(&holed, PEN_COLOR_WHITE, 3);
PAL_OLED_DrawLine(&holed, GetCursorX(&holed)+45, GetCursorY(&holed));

PAL_OLED_MoveCursor(&holed, 0, 8);
PAL_OLED_SetPen(&holed, PEN_COLOR_WHITE, 4);
PAL_OLED_DrawLine(&holed, GetCursorX(&holed)+45, GetCursorY(&holed));
```

```
PAL_OLED_SetPen(&holed, PEN_COLOR_WHITE, 1);

PAL_OLED_SetCursor(&holed, 91,10);
PAL_OLED_LineTo(&holed, 95,24);
PAL_OLED_LineTo(&holed, 107,24);
PAL_OLED_LineTo(&holed, 97,33);
PAL_OLED_LineTo(&holed, 101,48);
PAL_OLED_LineTo(&holed, 91,39);
PAL_OLED_LineTo(&holed, 80,48);
PAL_OLED_LineTo(&holed, 84,33);
PAL_OLED_LineTo(&holed, 74,24);
PAL_OLED_LineTo(&holed, 87,24);
PAL_OLED_LineTo(&holed, 91,10);
```

3.4.5. 画矩形和画圆



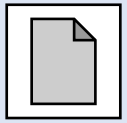

```
// @简介：绘制矩形
void PAL_OLED_DrawRect(…,uint16_t Width, uint16_t Height);
// 绘制圆形
void PAL_OLED_DrawCircle(…,uint16_t Radius);
```

```
PAL_OLED_SetPen(&holed, PEN_COLOR_TRANSPARENT, 1);
PAL_OLED_SetBrush(&holed, BRUSH_WHITE);
PAL_OLED_SetCursor(&holed, 0, 0);
PAL_OLED_DrawRect(&holed, GetScreenWidth(), GetScreenHeight());

PAL_OLED_SetCursor(&holed, 20, 20);
PAL_OLED_SetPen(&holed, PEN_COLOR_BLACK, 1);
PAL_OLED_SetBrush(&holed, BRUSH_TRANSPARENT);
PAL_OLED_DrawRect(&holed, 40, 20);

PAL_OLED_SetCursor(&holed, 70, 20);
PAL_OLED_SetPen(&holed, PEN_COLOR_TRANSPARENT, 1);
PAL_OLED_SetBrush(&holed, BRUSH_BLACK);
PAL_OLED_DrawRect(&holed, 40, 20);
```

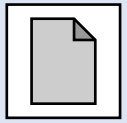
3.5. 显示文字

铁头山羊 TIETOUSHANYANG

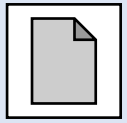
3.5.1. 基本字符显示



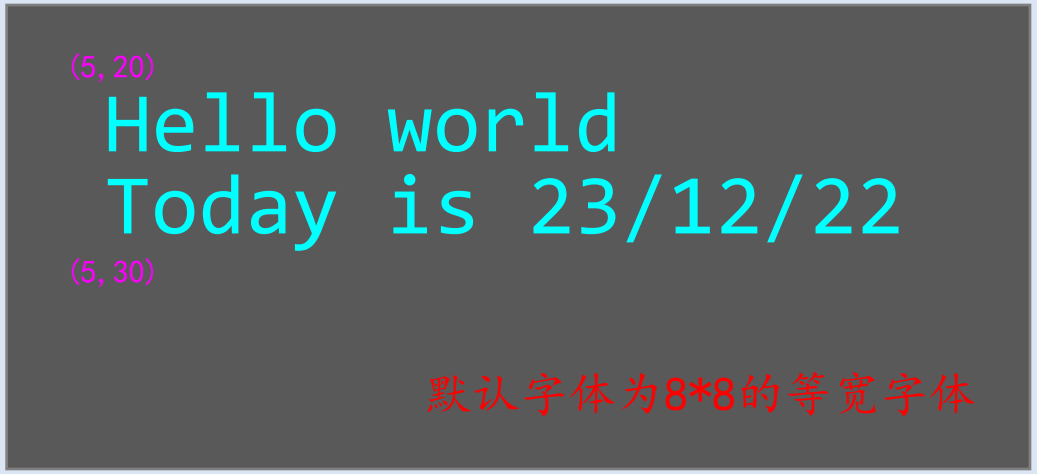
3.5.2. 文本框功能

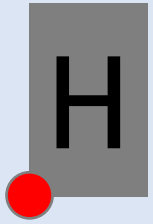


3.5.3. 改变字体



3.5.1. 基本字符显示



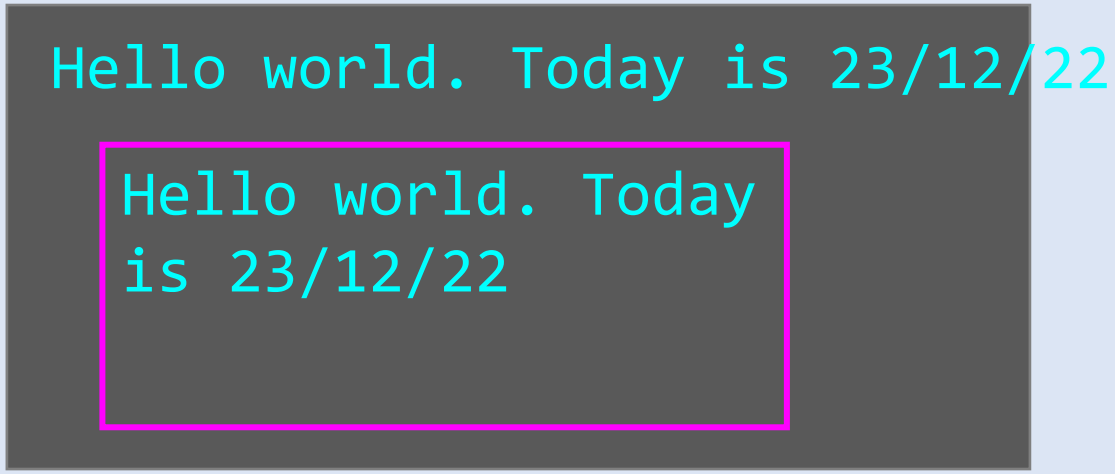


光标位置

```
// @简介: 显示字符串
void PAL_OLED_DrawString(…,const char *Str);
// @简介: 显示格式化字符串
void PAL_OLED_Printf(…,const char *Format, …);
```

3.5.2. 文本框功能

铁头山羊 TIETOUSHANYANG

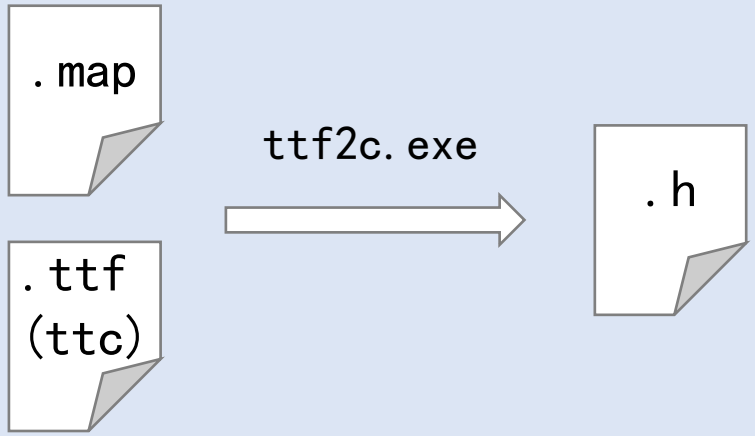



```
// @简介: 开始使用文本框区域
//        设置文本框后支持自动换行
//        光标移动到文本框的左上角的点
//        可以使用\r\n
void PAL_OLED_StartTextRegion(…,int16_t X,int16_t Y,
uint16_t Width, uint16_t Height);
// @简介: 取消文本框区域
void PAL_OLED_StopTextRegion(…);
```

```
PAL_OLED_StartTextRegion(&holed, 0, 0, 128, 64);

PAL_OLED_DrawString(&holed, "There once was a man named John, \r\n");
PAL_OLED_DrawString(&holed, "he was 70 years old. \r\n");
PAL_OLED_DrawString(&holed, "He really liked the name John, \r\n");
PAL_OLED_DrawString(&holed, "But didn't like being 70. \r\n");

PAL_OLED_StopTextRegion(&holed);
```

3.5.3. 改变字体


```
// @简介: 设置字体
//        默认字体为default_font (8*8点阵)
PAL_OLED_SetFont(…,&font_name);
```



生成字体的流程:

1. 在.map文件中输入想要显示的文字
2. 找到合适的字体(.ttf/ttc)
3. 运行ttf2c <xxx.ttf> <字号>, 生成.h文件
4. 引用头文件并使用字体

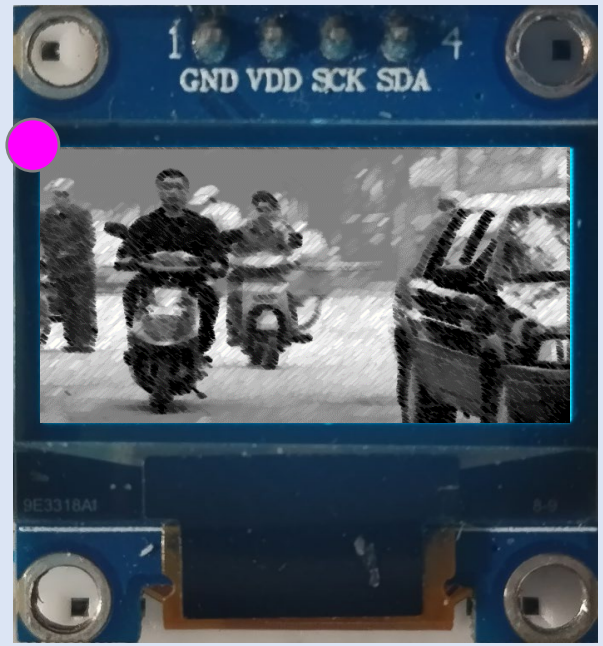
3.6. 显示图像

image2cpp (javl.github.io/image2cpp)

```
const uint8_t image[] = {
    0xff, 0xff, 0xfc, 0x00, …
    0xff, 0xff, 0xfc, 0x00, …
    0xff, 0xff, 0xfe, 0x00, …
    …
    0xff, 0xff, 0xfe, 0x00, … };
```

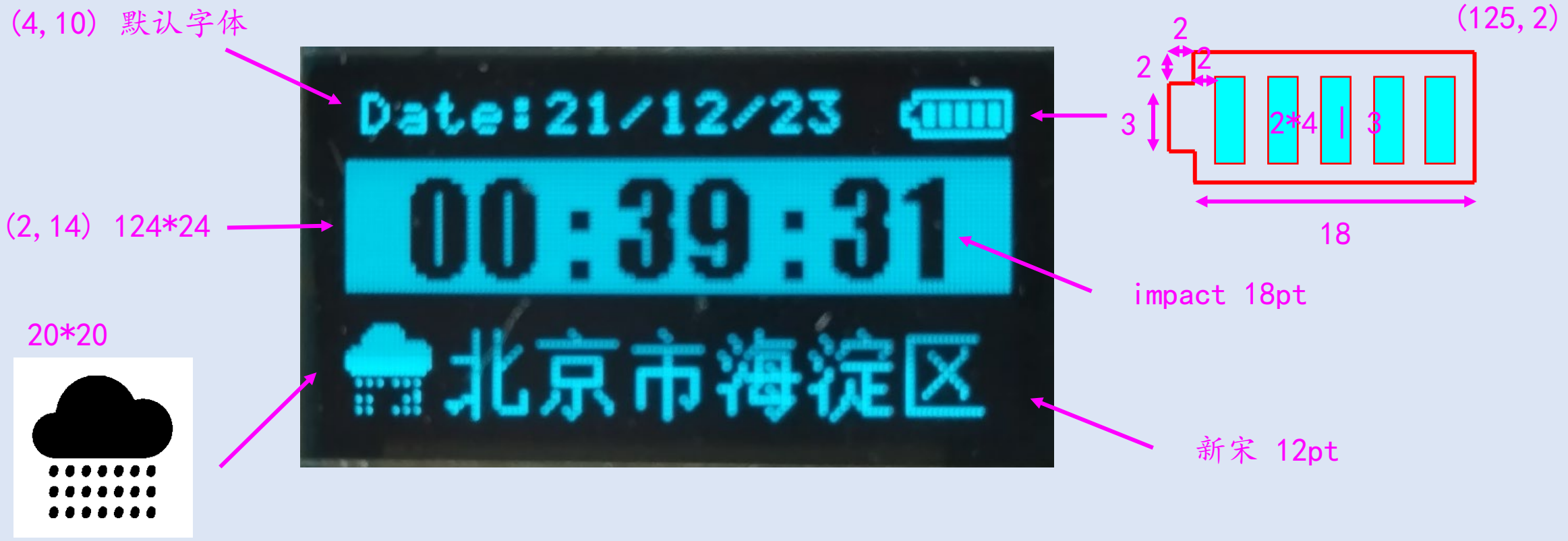
64

16*8




```
PAL_OLED_Clear(…);
PAL_OLED_SetCursor …, 0,0);
PAL_OLED_SetPen(WHITE); PAL_OLED_SetBrush(BLACK);
PAL_OLED_DrawBitmap(…, 128,64,image);
PAL_OLED_SendBuffer(…);
```

3.7. 电子手表实验


(4, 10) 默认字体
(2, 14) 124*24
20*20
Date:21/12/23
00:39:31
北京市海淀区
(125, 2)
2
2
2
3
2*4
3
18
impact 18pt
新宋 12pt

第四节、MPU6050

铁头山羊 TIETOUSHANYANG

4.1. 课程介绍

4.2. 运动传感器与姿态测量

4.3. MPU6050简介

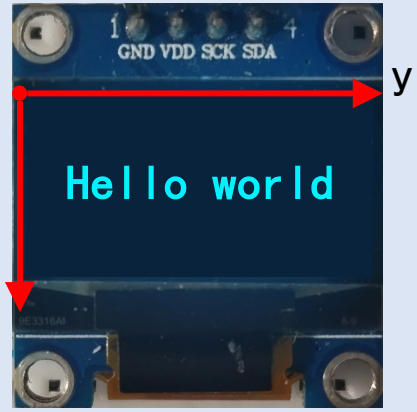
4.4. MPU6050编程

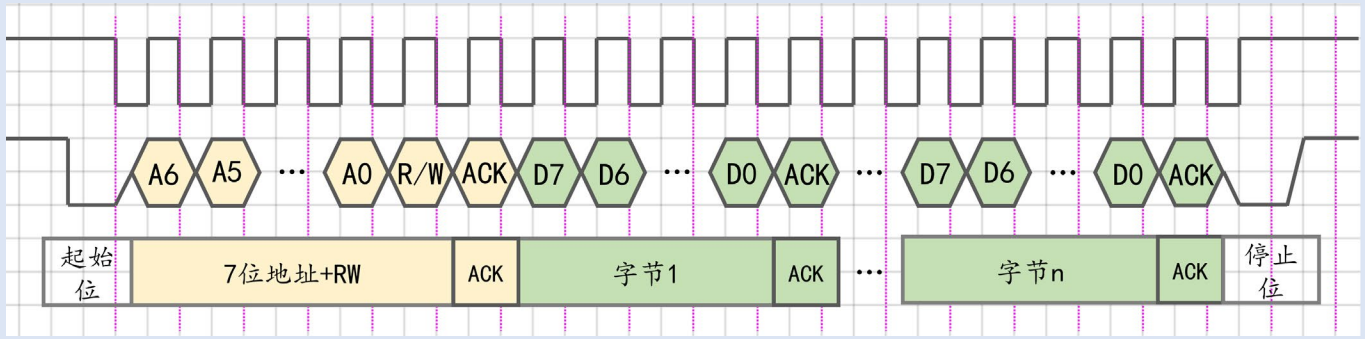
4.5. PAL库编程

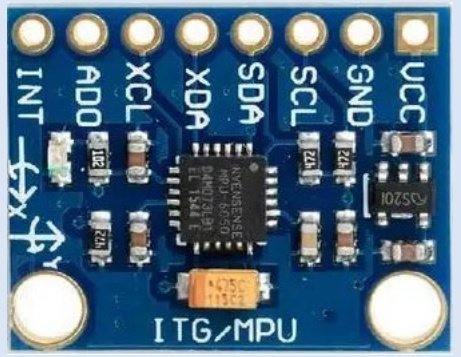
4.1. 课程介绍

铁头山羊 TIETOUSHANYANG

站在驱动编写视角进行学习







配置时钟
配置电荷泵
配置显示位置
...

复杂

忽略底层细节
只学使用方法

I2C基础知识

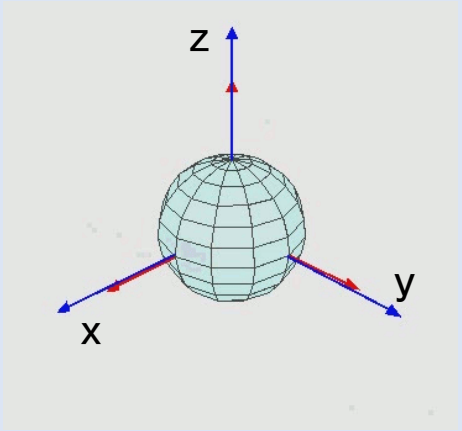
简单

从头编写驱动
窥探底层奥秘

设置采样率
设置量程
直接读取数据

4.2. 运动传感器与姿态测量

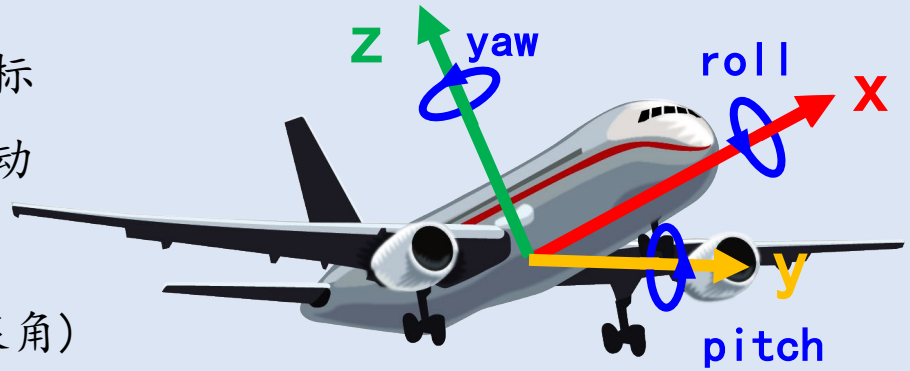
4.2.1. 欧拉角

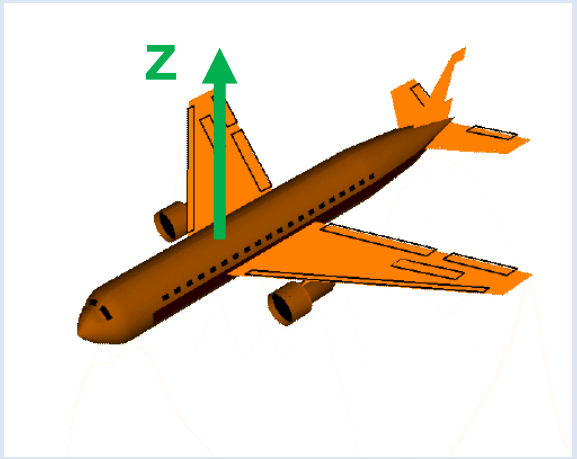


欧拉角：以运动物体建立坐标系，通过绕轴旋转来表示运动物体当前的姿态

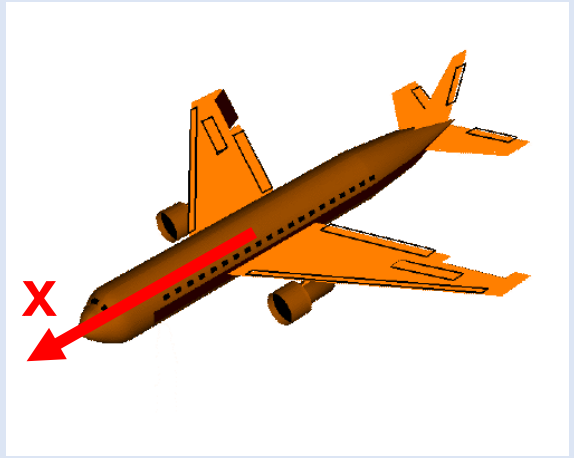
(Yaw - 偏航角 Roll - 翻滚角)

(Pitch - 俯仰角)

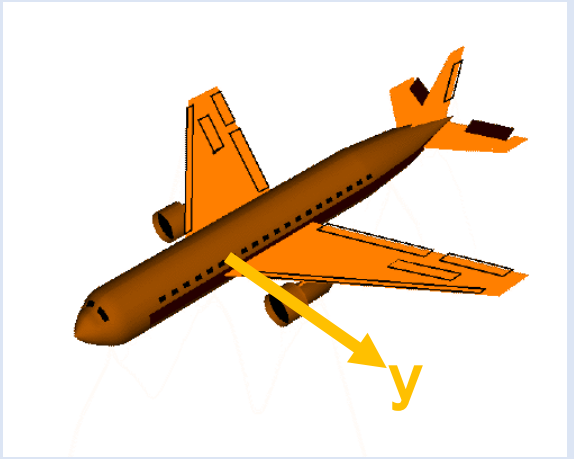




yaw - 偏航角

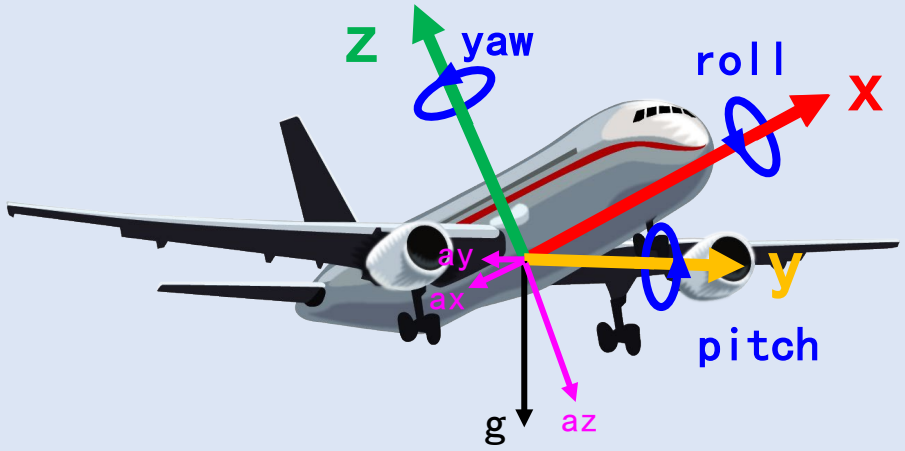


roll - 翻滚角

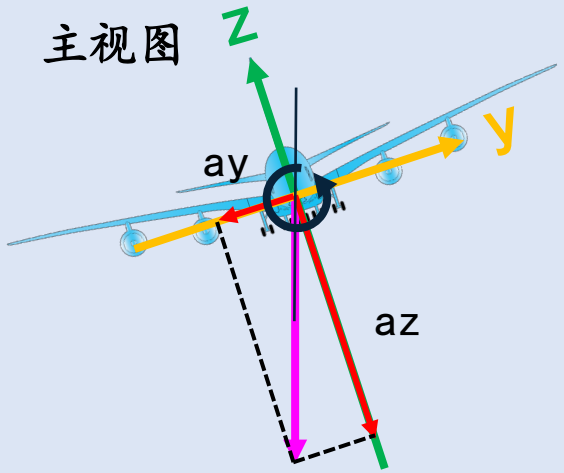


pitch - 俯仰角

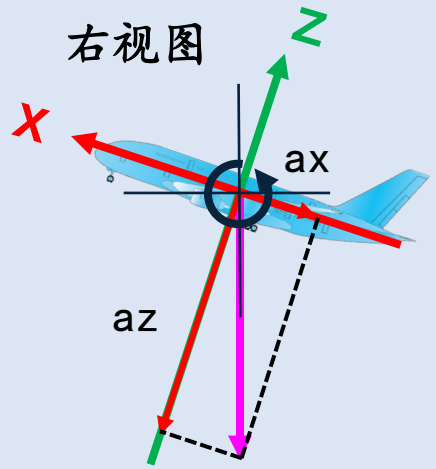
4.2.2. 加速度计(Accelerometer)与姿态测量



$\vec{a} = (ax, ay, az)$



$roll = atan2(ay, az)$

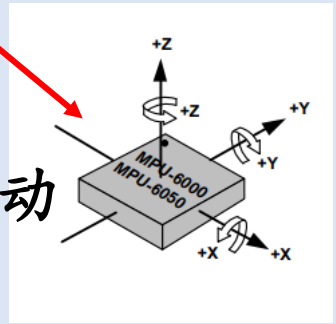


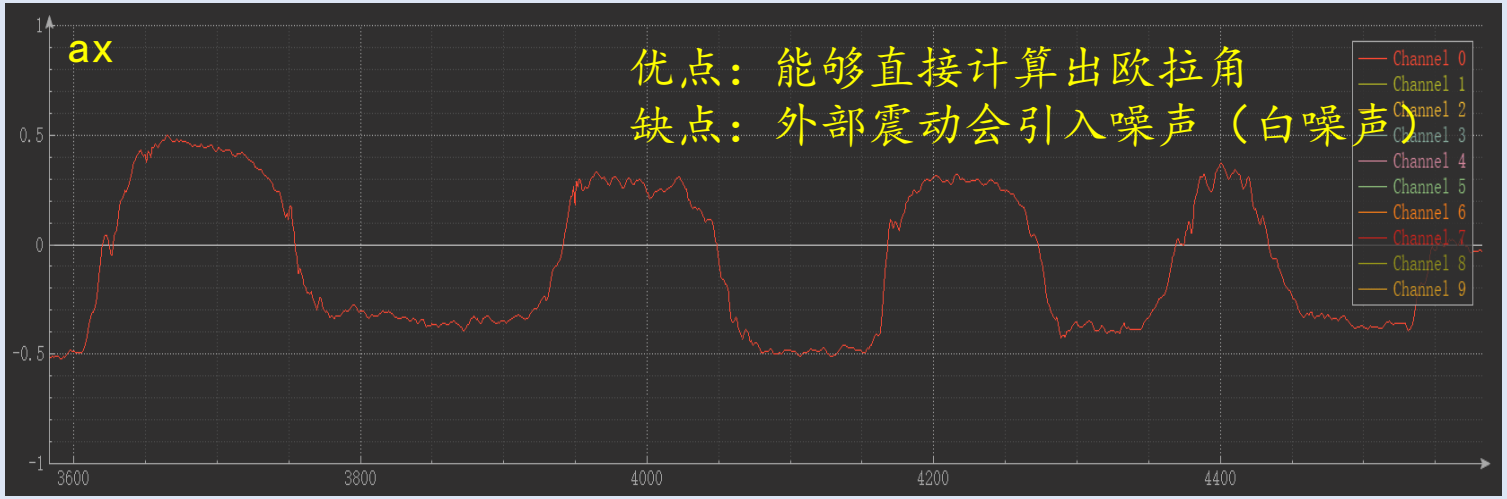
$\text{pitch} = -atan2(ax, az)$

震动

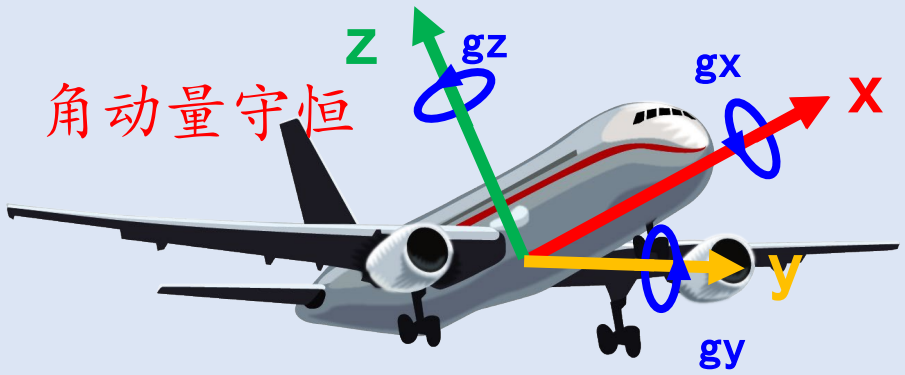
||

变速运动





4.2.3. 陀螺仪(Gyroscope)与姿态测量



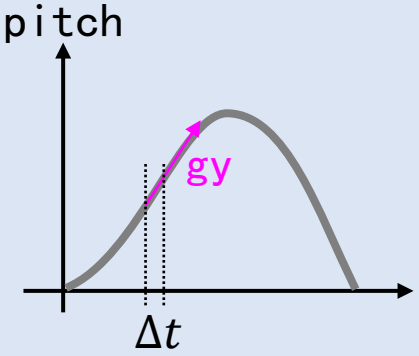
角动量守恒

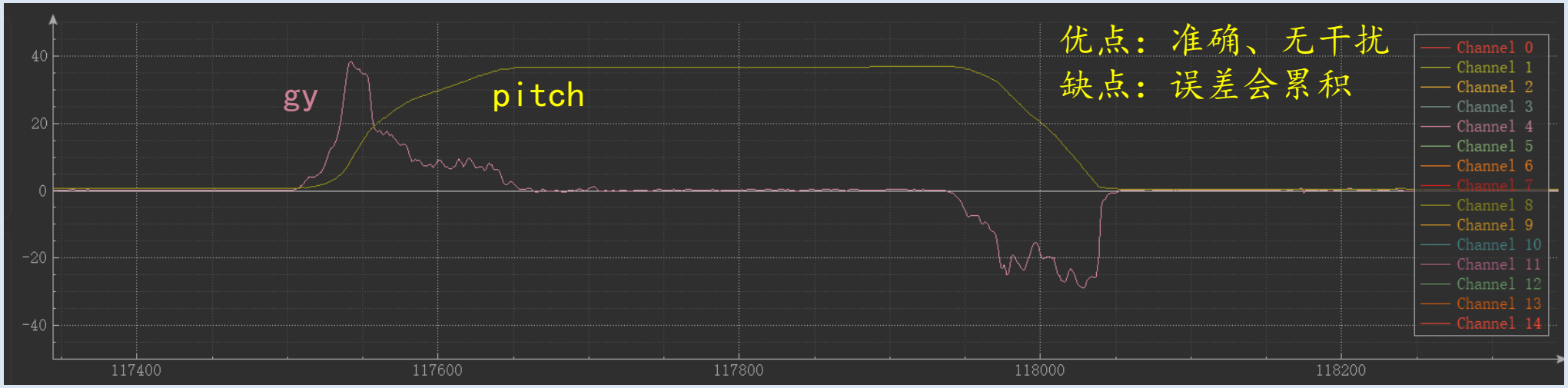
$\vec{g} = (gx, gy, gz)$

陀螺仪能够测量绕轴旋转的角速度，单位°/s

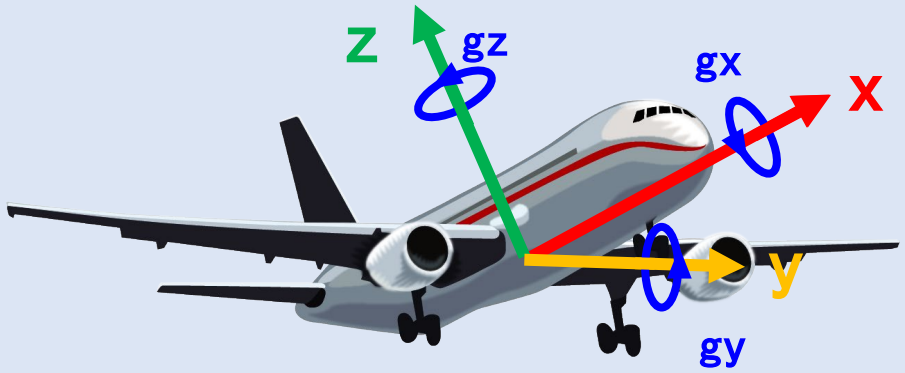
$$\begin{aligned} pitch(t) &= pitch(t-\Delta t) + gy * \Delta t \\ yaw(t) &= yaw(t-\Delta t) + gz * \Delta t \\ roll(t) &= roll(t-\Delta t) + gx * \Delta t \end{aligned}$$

新角度　上次角度　角速度　时间





4.2.4. 传感器融合



	加速度传感器	陀螺仪
测量结果	3轴加速度	3轴角速度
短期	噪声严重	不受震动影响
长期	无漂移	漂移严重

4.2.4. 传感器融合

融合后的结果 = α*陀螺仪测量结果 + （1-α）*加速度计测量结果

陀螺仪权重　　加速度计权重

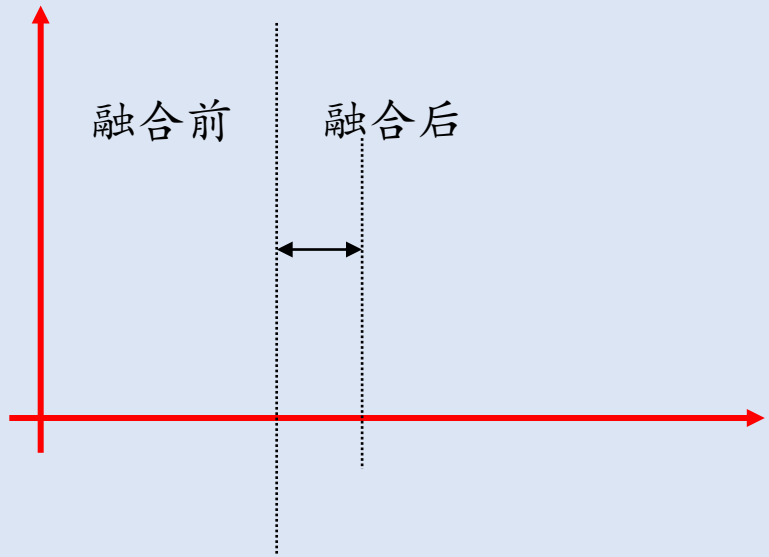
$$yaw_g = yaw + gz * \Delta t$$

$$roll_a = atan2(ay, az) \qquad roll_g = roll + gx * \Delta t$$

$$pitch_a = -atan2(ax, az) \qquad pitch_g = pitch + gy * \Delta t$$

$$yaw = yaw_g$$
$$roll = \alpha * roll_g + (1 - \alpha) * roll_a$$
$$pitch = \alpha * pitch_g + (1 - \alpha) * pitch_a$$

$$\alpha = \tau / (\tau + \Delta t)$$



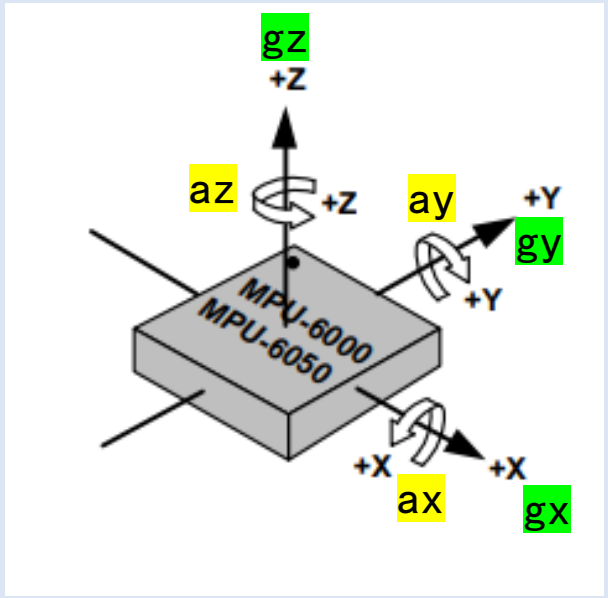
4.3. MPU6050简介

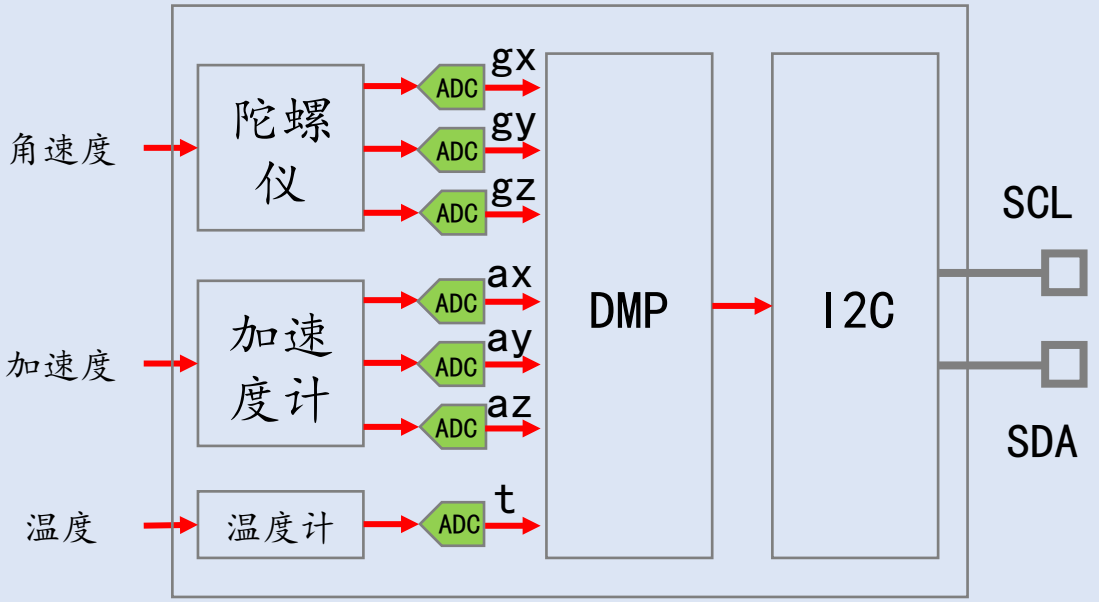
4.3.1. 功能介绍

铁头山羊 TIETOUSHANYANG

MPU6050（MPU - Motion Processing Unit - 运动处理单元）

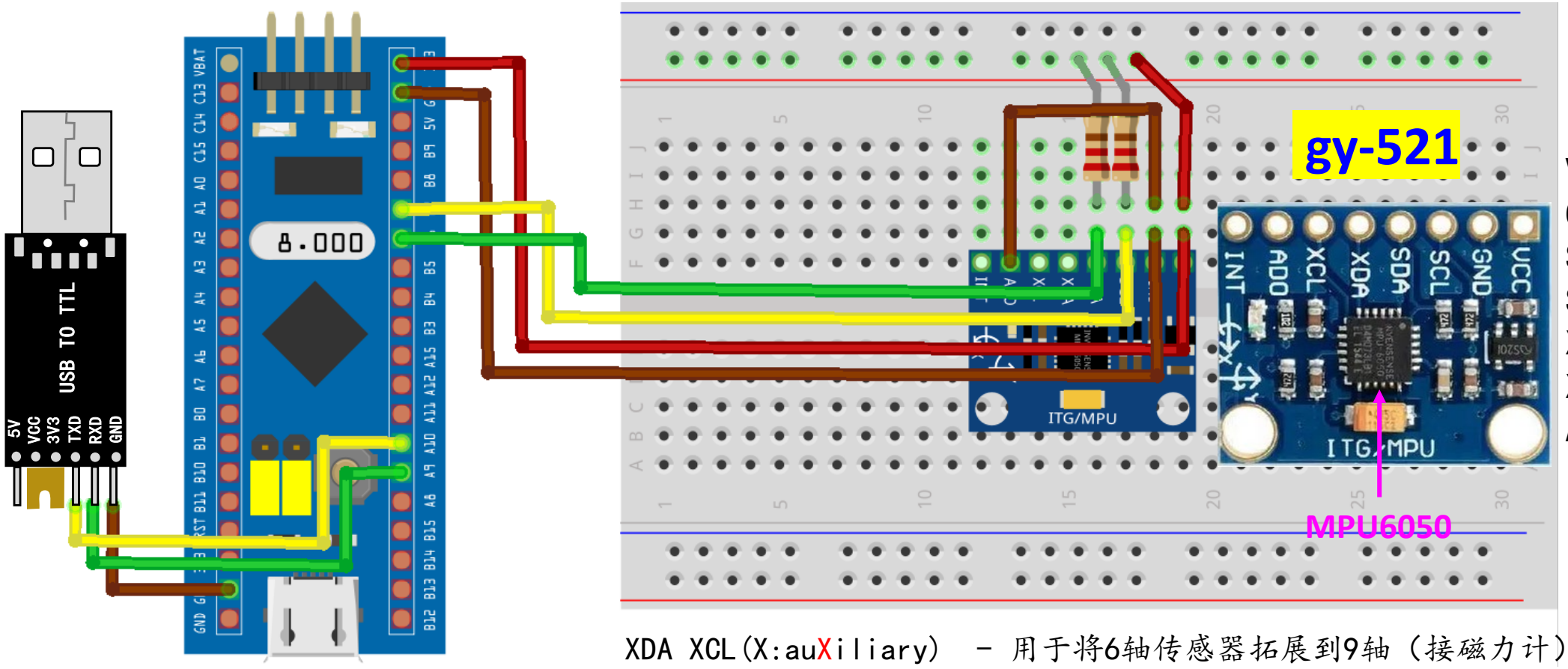
- 带DMP的6轴运动传感器





DMP = Digital Motion Processor - 数字运动处理器

4.3.2. 搭建电路



VCC-电源
GND-地
SCL-主i2c时钟
SDA-主i2c数据
XDA-辅i2c数据
XCL-辅i2c时钟
ADO-地址选择
INT-中断

XDA XCL(X:auXiliary) - 用于将6轴传感器拓展到9轴（接磁力计）

INT (Interrupt) - 中断引脚，每当一个Sample的数据采集完成后发送脉冲

4.4. MPU6050编程

4.4.1. 编程思路

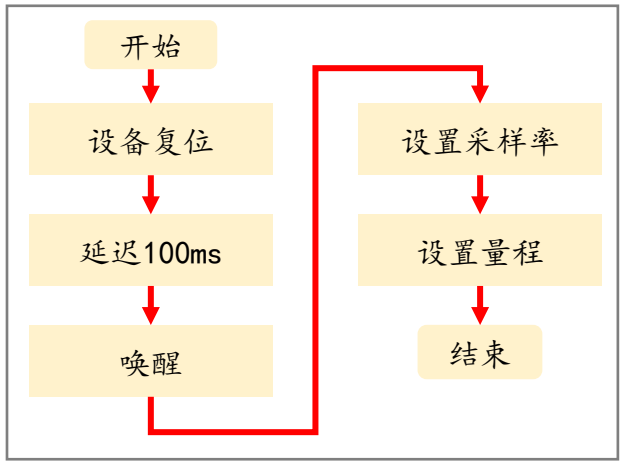
app_mpu6050.h
app_mpu6050.c

初始化

```
void App_MPU6050_Init(void){

}
```

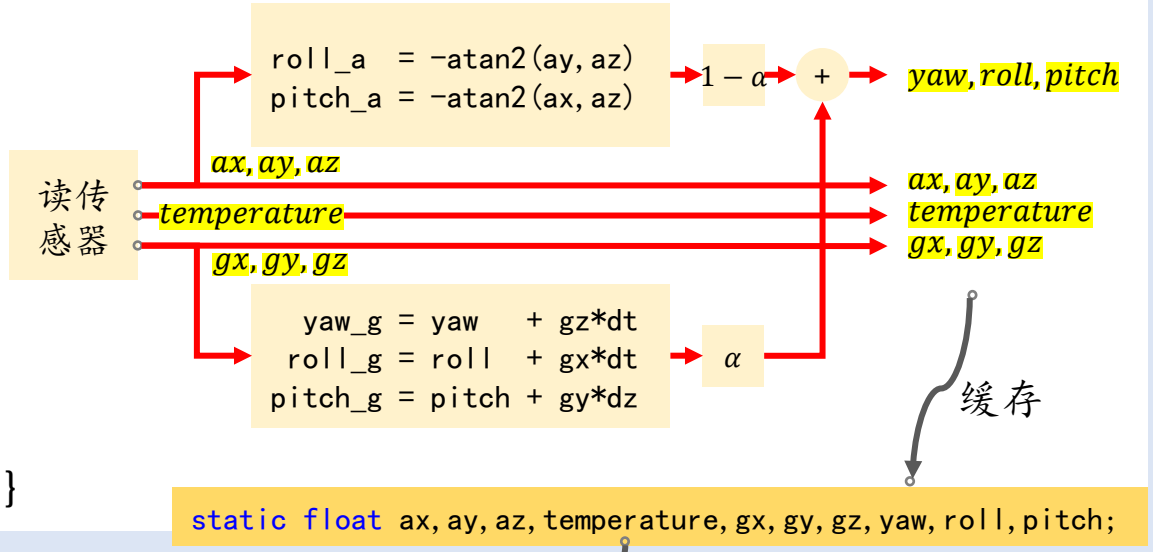
1. I2C初始化

2. mpu6050初始化



进程函数

```
void App_MPU6050_Proc(void){
  PAL_PERIODIC_PROC(5); // 每dt毫秒执行一次(dt=5)

}
```



读取结果

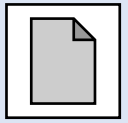
加速度 温度 陀螺仪 欧拉角

```
void App_MPU6050_GetResult(float *pAccelOut, float *pTempOut, float *pGyroOut, float *pEularAngleOut);
```

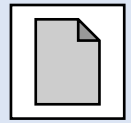
4.4.2. 初始化

铁头山羊 TIETOUSHANYANG

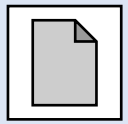
4.4.2.1. 初始化i2c接口



4.4.2.2. 使用i2c读写寄存器



4.4.2.3. 设备初始化



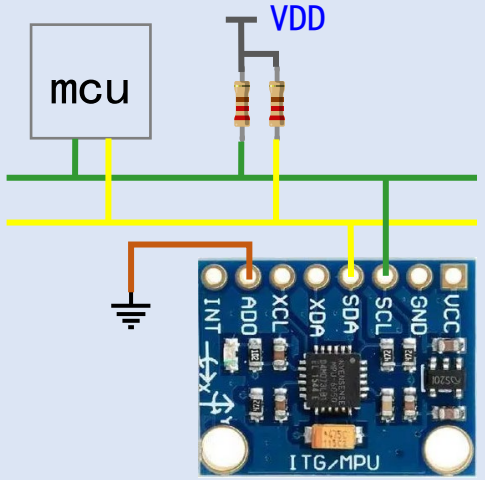
4.4.2.1. 初始化I2C接口

铁头山羊 TIETOUSHANYANG

直接使用PAL库

```
hI2C1.Init.I2Cx = I2C1;
hI2C1.Init.I2C_ClockSpeed = [          ]; <= 400 kHz
hI2C1.Init.I2C_DutyCycle = [                    ];
PAL_I2C_Init(&hI2C1);                  2:1 或 16:9
```

从机地址：b1101000r/w -> 0xd0



9.2 I²C Interface

I²C is a two-wire interface comprised of the signals serial data (SDA) and serial clock (SCL). In general, the lines are open-drain and bi-directional. In a generalized I²C interface implementation, attached devices can be a master or a slave. The master device puts the slave address on the bus, and the slave device with the matching address acknowledges the master.

The MPU-60X0 always operates as a slave device when communicating to the system processor, which thus acts as the master. SDA and SCL lines typically need pull-up resistors to VDD. The maximum bus speed is 400 kHz.

The slave address of the MPU-60X0 is b110100X which is 7 bits long. The LSB bit of the 7 bit address is determined by the logic level on pin AD0. This allows two MPU-60X0s to be connected to the same I²C bus. When used in this configuration, the address of the one of the devices should be b1101000 (pin AD0 is logic low) and the address of the other should be b1101001 (pin AD0 is logic high).

9.2 I²C 接口

I²C 是一种由两个引脚组成的接口 - 数据引脚（SDA）和时钟引脚（SCL）。通常这两个引脚都工作在开漏模式下，并且通信方向是双向的。一般情况下，连接到总线上的设备要么是主机要么是从机。主机向总线上发送从机地址，相应地址的从机会进行应答（ACK）。

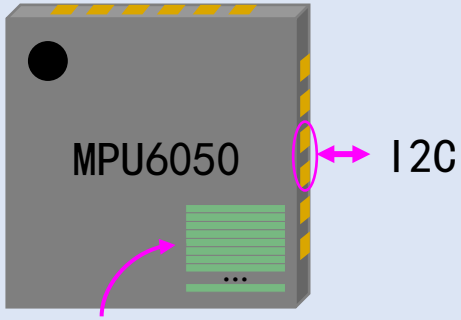
当 MPU6050 与系统处理器进行通信时，MPU6050 总是被作为从机使用，而系统处理器作为主机使用。一般情况下 SDA 和 SCL 需要上拉电阻与 VDD 连接。最大通信速率是 400 kHz。

MPU60X0 的从机地址是 b110100x，长度是 7 bit。7 位地址的 LSB 位由 A0 引脚的逻辑电平决定。这样做可以实现将两个 MPU60X0 连接到同一个 I²C 总线。当使用这种配置时，其中一个设备的地址应该是 b1101000（AD0 引脚为逻辑低电平），另一个设备的地址应该是 b1101001（AD0 引脚为逻辑高电平）。

4.4.2.2. 使用I2C读写寄存器

铁头山羊 TIETOUSHANYANG

4.4.2.2.1. MPU6050寄存器介绍



！！不光MPU6050，很多I2C设备都采用这种读写寄存器的方式进行操作

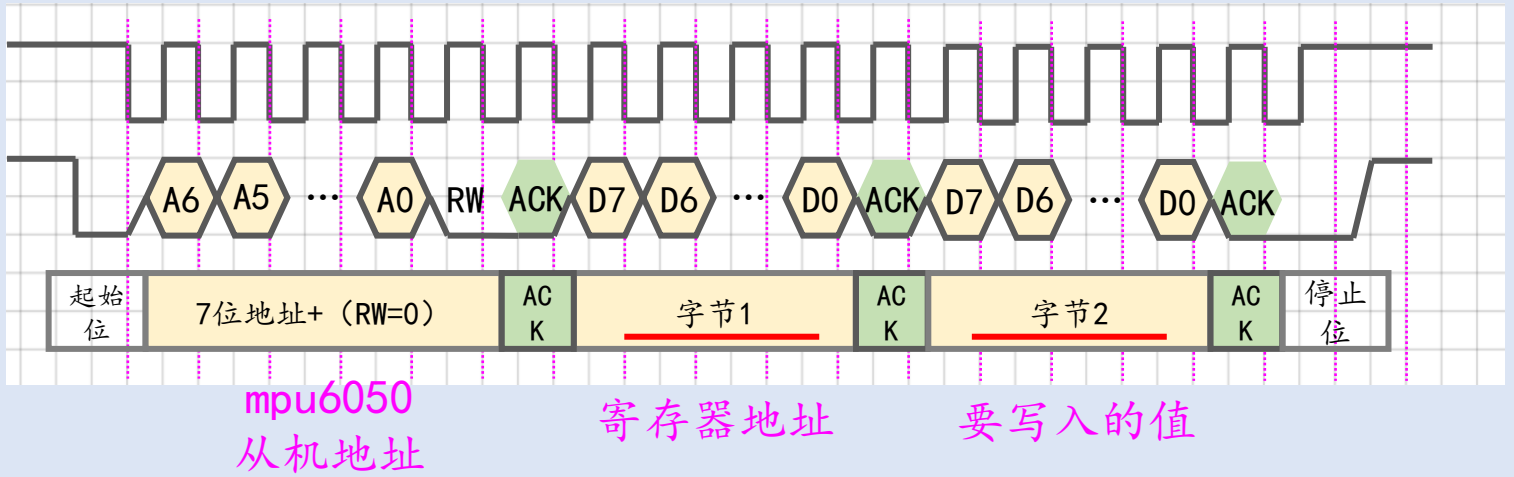
3 Register Map

地址 The register map for the MPU-60X0 is listed below.

每个寄存器占1个字节

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0D	13	SELF_TEST_X	R/W	XA_TEST[4-2]			XG_TEST[4-0]				
0E	14	SELF_TEST_Y	R/W	YA_TEST[4-2]			YG_TEST[4-0]				
0F	15	SELF_TEST_Z	R/W	ZA_TEST[4-2]			ZG_TEST[4-0]				
10	16	SELF_TEST_A	R/W	RESERVED		XA_TEST[1-0]		YA_TEST[1-0]		ZA_TEST[1-0]	
19	25	SMPLRT_DIV	R/W	SMPLRT_DIV[7:0]							
1A	26	CONFIG	R/W	-	-	EXT_SYNC_SET[2:0]			DLPF_CFG[2:0]		
1B	27	GYRO_CONFIG	R/W	-	-	-	FS_SEL [1:0]		-	-	-
1C	28	ACCEL_CONFIG	R/W	XA_ST	YA_ST	ZA_ST	AFS_SEL[1:0]				
23	35	FIFO_EN	R/W	TEMP_FIFO_EN	XG_FIFO_EN	YG_FIFO_EN	ZG_FIFO_EN	ACCEL_FIFO_EN	SLV2_FIFO_EN	SLV1_FIFO_EN	SLV0_FIFO_EN
24	36	I2C_MST_CTRL	R/W	MULT_MST_EN	WAIT_FOR_ES	SLV_3_FIFO_EN	I2C_MST_P_NSR	I2C_MST_CLK[3:0]			
25	37	I2C_SLV0_ADDR	R/W	I2C_SLV0_RW	I2C_SLV0_ADDR[6:0]						
26	38	I2C_SLV0_REG	R/W	I2C_SLV0_REG[7:0]							
27	39	I2C_SLV0_CTRL	R/W	I2C_SLV0_EN	I2C_SLV0_BYTE_SW	I2C_SLV0_REG_DIS	I2C_SLV0_GRP	I2C_SLV0_LEN[3:0]			
28	40	I2C_SLV1_ADDR	R/W	I2C_SLV1_RW	I2C_SLV1_ADDR[6:0]						
29	41	I2C_SLV1_REG	R/W	I2C_SLV1_REG[7:0]							
2A	42	I2C_SLV1_CTRL	R/W	I2C_SLV1_EN	I2C_SLV1_BYTE_SW	I2C_SLV1_REG_DIS	I2C_SLV1_GRP	I2C_SLV1_LEN[3:0]			
2B	43	I2C_SLV2_ADDR	R/W	I2C_SLV2_RW	I2C_SLV2_ADDR[6:0]						
2C	44	I2C_SLV2_REG	R/W	I2C_SLV2_REG[7:0]							
2D	45	I2C_SLV2_CTRL	R/W	I2C_SLV2_EN	I2C_SLV2_BYTE_SW	I2C_SLV2_REG_DIS	I2C_SLV2_GRP	I2C_SLV2_LEN[3:0]			
2E	46	I2C_SLV3_ADDR	R/W	I2C_SLV3_RW	I2C_SLV3_ADDR[6:0]						

4.4.2.2.2. 写寄存器



3 Register Map

The register map for the MPU-60X

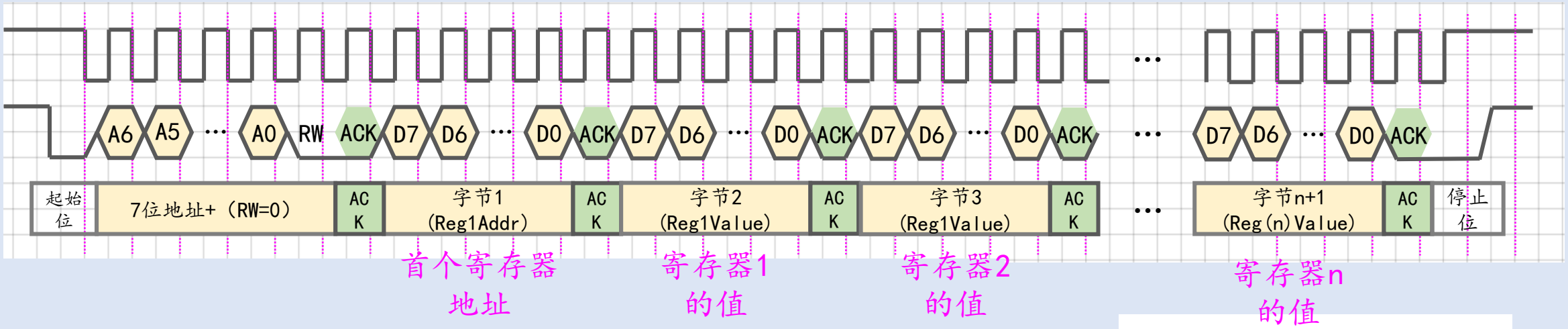
Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F
0D	13	SELF_TEST_X	R/W
0E	14	SELF_TEST_Y	R/W
0F	15	SELF_TEST_Z	R/W
10	16	SELF_TEST_A	R/W
19	25	SMPLRT_DIV	R/W
1A	26	CONFIG	R/W
1B	27	GYRO_CONFIG	R/W

例如：想把设置mpu6050的采样率设置成200Hz，需要向SMPLRT_DIV寄存器写0x04

SlaveAddr = b11010000 = 0xd0, RegAddr = 0x19, Data = 0x04

```
uint8_t tmp[2];
tmp[0] = 0x19; // SMPLRT_DIV寄存器的地址
tmp[1] = 0x04; // 分频系数，1k/(0x04+1)=200
PAL_I2C_MasterTransmit(&hi2c,0xd0, tmp, 2);
```

4.4.2.2.3. 批量写入



例如：想要向SMPLRT_DIV写入0x04，向CONFIG写入0x00，向GYRO_CONFIG写入0x18

SlaveAddr = b1101000 = 0xd0， {0x19, 0x04, 0x00, 0x18}

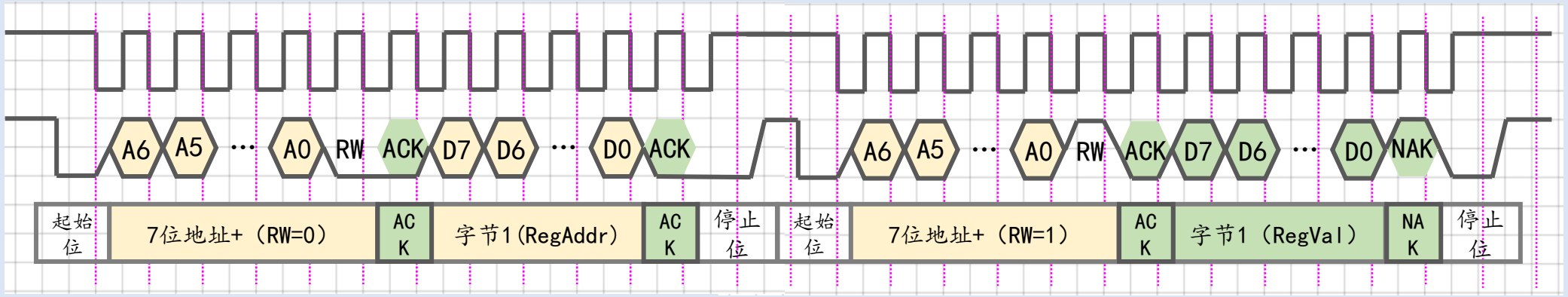
```
uint8_t tmp[4];
tmp[0] = 0x19; tmp[1] = 0x04;
tmp[2] = 0x00; tmp[3] = 0x18;
PAL_I2C_MasterTransmit(&hi2c, 0xd0, tmp, 4);
```

3 Register Map

The register map for the MPU-60X

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F
0D	13	SELF_TEST_X	R/W
0E	14	SELF_TEST_Y	R/W
0F	15	SELF_TEST_Z	R/W
10	16	SELF_TEST_A	R/W
19	25	SMPLRT_DIV	R/W
1A	26	CONFIG	R/W
1B	27	GYRO_CONFIG	R/W
1C	28	ACCEL_CONFIG	R/W

4.4.2.2.4. 读寄存器



例如：想要读取CONFIG寄存器的值 SlaveAddr = b1101000 = 0xd0, RegAddr =0x1a

3 Register Map

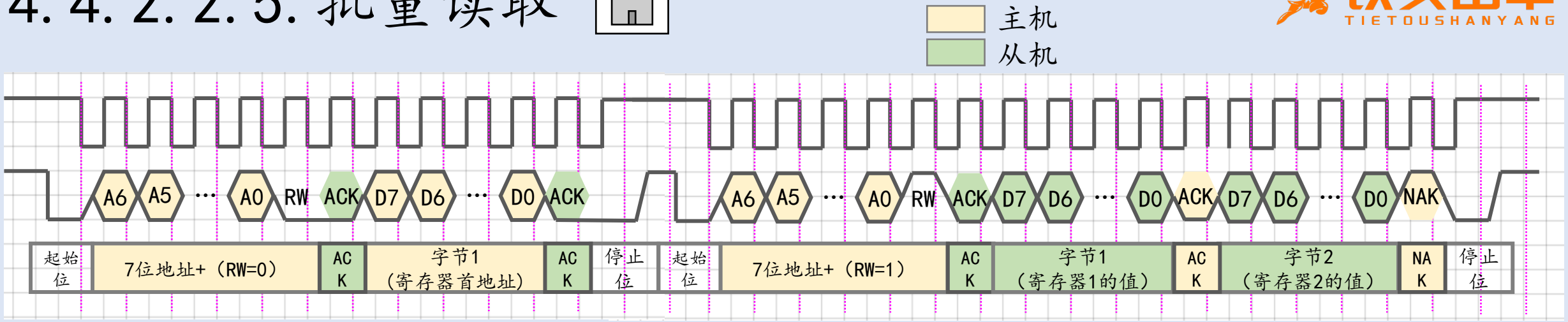
The register map for the MPU-60X

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F
0D	13	SELF_TEST_X	R/W
0E	14	SELF_TEST_Y	R/W
0F	15	SELF_TEST_Z	R/W
10	16	SELF_TEST_A	R/W
19	25	SMPLRT_DIV	R/W
1A	26	CONFIG	R/W

```
uint8_t tmp[1];
tmp[0] = 0x1a;
PAL_I2C_MasterTransmit(&hi2c, 0xd0, tmp, 1); // 发地址
uint8_t data; // 用来接收CONFIG寄存器的值
PAL_I2C_MasterReceive(&hi2c, 0xd0, &data, 1); // 收数据
```

4.4.2.2.5. 批量读取

铁头山羊 TIETOUSHANYANG



SlaveAddr = b1101000 = 0xd0

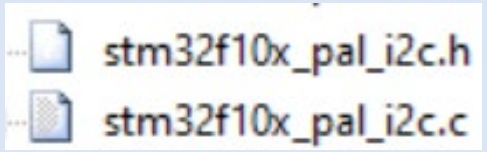
Addr (Hex)	Addr (Dec.)	Register Name
3B	59	ACCEL_XOUT_H
3C	60	ACCEL_XOUT_L
3D	61	ACCEL_YOUT_H
3E	62	ACCEL_YOUT_L
3F	63	ACCEL_ZOUT_H
40	64	ACCEL_ZOUT_L

ax高字节

ax低字节

```
uint8_t data_send[1];
tmp[0] = 0x3b;
PAL_I2C_MasterTransmit(&hi2c, 0xd0, tmp, 1); // 发地址
uint8_t data_recv[2]; // 用来接收CONFIG寄存器的值
PAL_I2C_MasterReceive(&hi2c, 0xd0, tmp, 2); // 收数据
ax = (data_recv[0] << 8) | data_recv[1];
```

4.4.2.2.6. PAL库编程接口



PAL_I2C_MasterTransmit(…)
PAL_I2C_MasterReceive(…);

写寄存器

```
ErrorStatus PAL_I2C_RegWrite(PalI2C_HandleTypeDef *Handle,          // i2c句柄
                             uint16_t SlaveAddr,                    // 从机地址
                             uint16_t RegAddr,                      // 寄存器地址
                       const uint8_t Data);                         // 要写入的数据(单个)
```

读寄存器

```
ErrorStatus PAL_I2C_RegRead(PalI2C_HandleTypeDef *Handle,           // i2c句柄
                            uint16_t SlaveAddr,                     // 从机地址
                            uint16_t RegAddr,                       // 寄存器地址
                             uint8_t *pDataOut);                    // 用于接收数据(单个)
```

批量写入

```
ErrorStatus PAL_I2C_RegWriteBytes(PalI2C_HandleTypeDef *Handle,     // i2c句柄
                                  uint16_t SlaveAddr,               // 从机地址
                                  uint16_t RegAddr,                 // 寄存器地址
                            const uint8_t *pData,                   // 要写入的数据(多个)
                                  uint16_t Size);                   // 要写入的数据长度
```

批量读取

```
ErrorStatus PAL_I2C_RegReadBytes(PalI2C_HandleTypeDef *Handle,      // i2c句柄
                                 uint16_t SlaveAddr,                // 从机地址
                                 uint16_t RegAddr,                  // 寄存器地址
                                  uint8_t *pBufferOut,              // 数据接收缓冲区
                                 uint16_t Size);                    // 要接收的数据长度
```

4.4.2.3. 设备初始化

4.4.2.3.1. 流程图

开始

设备复位 — 确保设备处于默认状态

延迟100ms

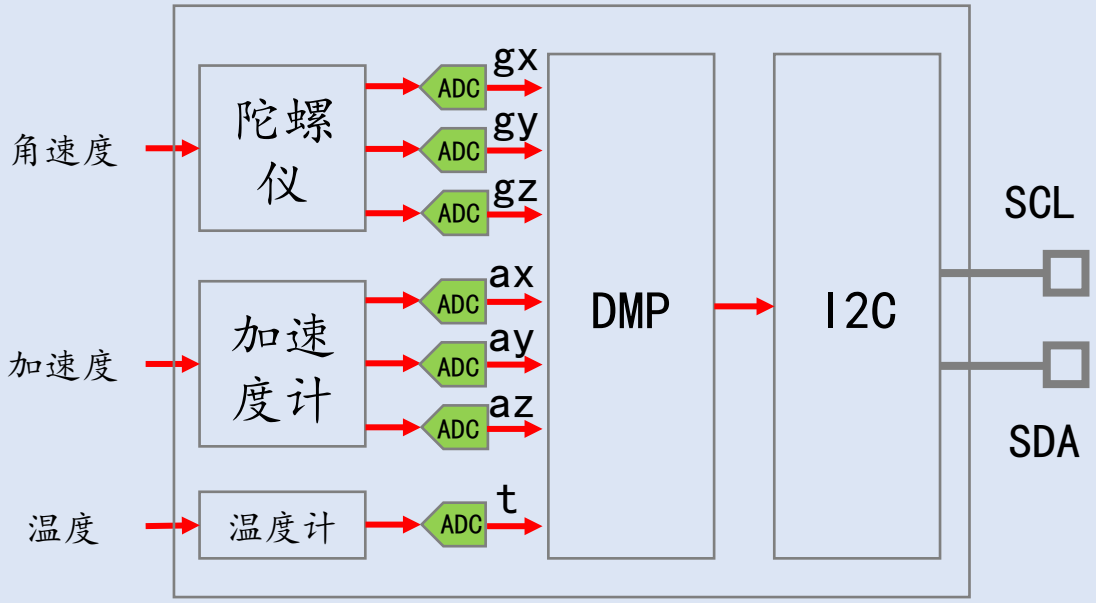
唤醒 — 默认处于休眠状态

设置采样率 — 传感器采集的频率

设置量程 — 加速度计和陀螺仪的量程

结束

阅读手册后可总结出此流程图



4.4.2.3.2. 复位和唤醒

铁头山羊 TIETOUSHANYANG

4.28 Register 107 – Power Management 1
PWR_MGMT_1

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6B	107	DEVICE_RESET	SLEEP	CYCLE	-	TEMP_DIS	CLKSEL[2:0]		

写1复位 写0唤醒

Parameters:

DEVICE_RESET When set to 1, this bit resets all internal registers to their default values.
The bit automatically clears to 0 once the reset is done.
The default values for each register can be found in Section 3.
写 1 时设备的所有寄存器复位到默认状态。
复位结束后该比特位自动清零。

SLEEP When set to 1, this bit puts the MPU-60X0 into sleep mode.
写 1 时 MPU-60X0 进入休眠模式。

CYCLE When this bit is set to 1 and *SLEEP* is disabled, the MPU-60X0 will cycle between sleep mode and waking up to take a single sample of data from active sensors at a rate determined by *LP_WAKE_CTRL* (register 108).

在 *SLEEP* 等于 1 的前提下向该位写 1，MPU-60X0 会周期性地从睡眠模式醒来，采集一个 sample 后继续进入休眠模式，自动苏醒的周期可以通过寄存器 108 的 *LP_WAKE_UP* 来设置。

示例代码：

```
/* 设备复位 */
PAL_I2C_RegWrite(&hi2c,0xd0,0x6b,0x80);
PAL_Delay(100);
/* 退出休眠模式 */
PAL_I2C_RegWrite(&hi2c,0xd0,0x6b,0x00);
```

4.4.2.3.3. 设置采样率

铁头山羊 TIETOUSHANYANG

4.2 Register 25 – Sample Rate Divider SMPRT_DIV

Type: Read/Write

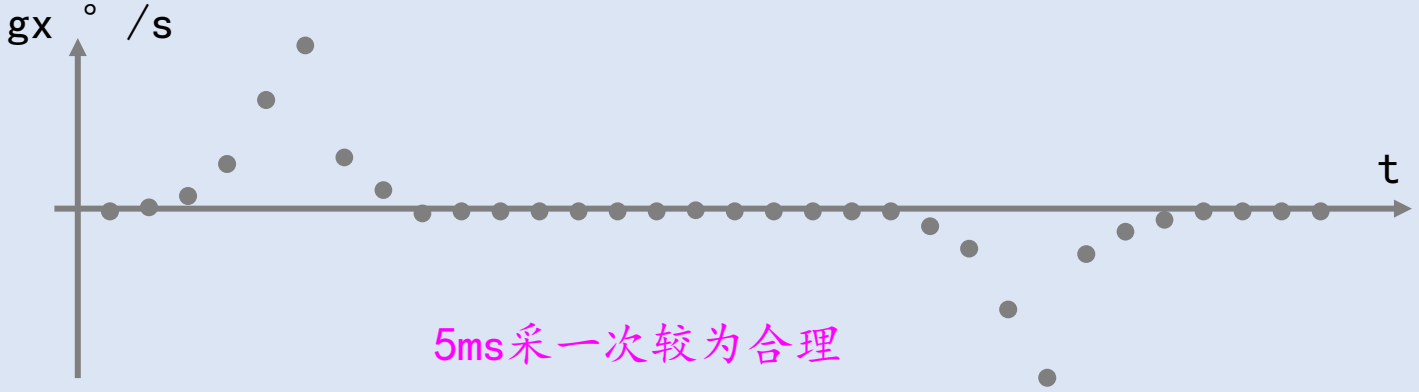
Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
19	25	SMPLRT_DIV[7:0]							

Parameters:

SMPLRT_DIV 8-bit unsigned value. The Sample Rate is determined by dividing the gyroscope output rate by this value.

采样率分频系数。8 位无符号整数。采样率=陀螺仪输出频率÷(SMPLRT_DIV + 1)。

1kHz



示例代码：

```
/* 将采样率设置为200Hz */
PAL_I2C_RegWrite(&hi2c,0xd0,0x19,0x04);
```

4.4.2.3.4. 设置加速度计量程

铁头山羊 TIETOUSHANYANG

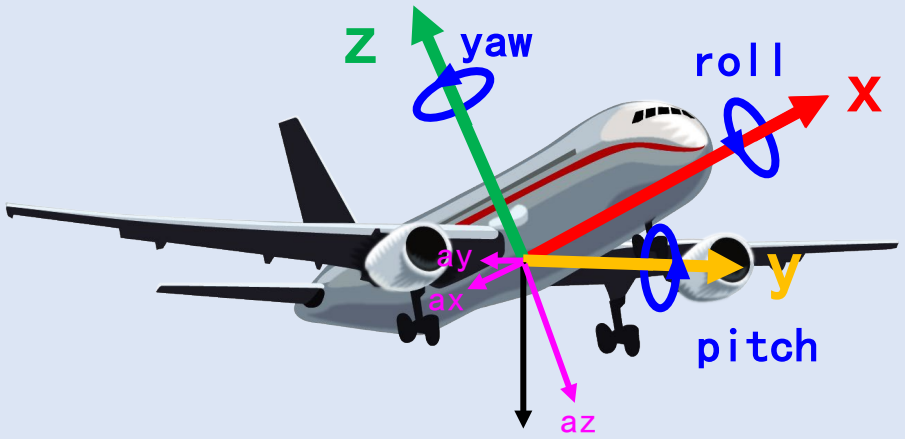
4.5 Register 28 – Accelerometer Configuration
ACCEL_CONFIG

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1C	28	XA_ST	YA_ST	ZA_ST	AFS_SEL[1:0]		-		

示例代码：

```
/* 将加速度量程设置为±2g */
PAL_I2C_RegWrite(&hi2c,0xd0,0x1c,0x00);
```



$$\vec{a} = (ax, ay, az)$$

AFS_SEL	Full Scale Range
0	± 2g
1	± 4g
2	± 8g
3	± 16g

±2 * 9.8 m/s2

4.4.2.3.5. 设置陀螺仪量程

铁头山羊 TIETOUSHANYANG

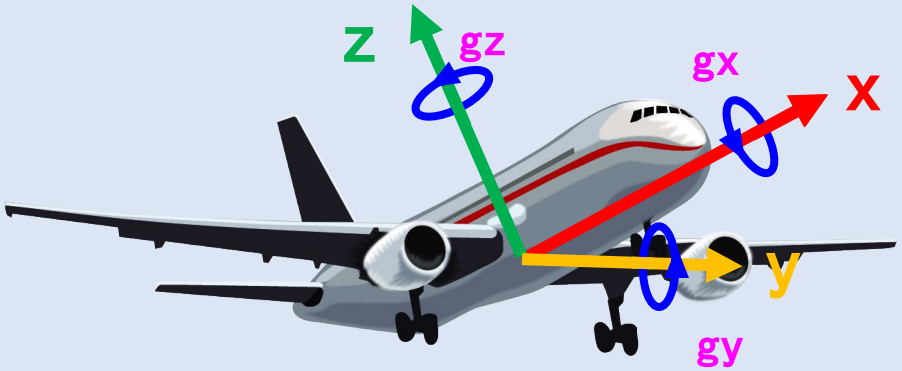
4.4 Register 27 – Gyroscope Configuration
GYRO_CONFIG

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1B	27	XG_ST	YG_ST	ZG_ST	FS_SEL[1:0]		-	-	-

示例代码：

```
/* 将陀螺仪量程设置为±2000°/s */
PAL_I2C_RegWrite(&hi2c,0xd0,0x1b,0x18);
```

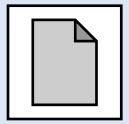


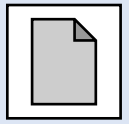
$$\vec{g} = (gx, gy, gz)$$

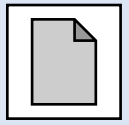
FS_SEL	Full Scale Range
0	± 250 °/s
1	± 500 °/s
2	± 1000 °/s
3	± 2000 °/s

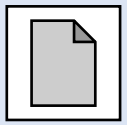
4.4.3. 进程函数

铁头山羊 TIETOUSHANYANG

4.4.3.1. 读取原始数据 

4.4.3.2. 使用加速度计算欧拉角 

4.4.3.3. 使用陀螺仪计算欧拉角 

4.4.3.4. 融合 

4.4.3.1. 读取原始数据

铁头山羊 TIETOUSHANYANG

16位2进制补码【-32768~32767】

Addr (Hex)	Addr (Dec.)	Register Name	
3B	59	ACCEL_XOUT_H	X轴加速度(i16)
3C	60	ACCEL_XOUT_L	
3D	61	ACCEL_YOUT_H	Y轴加速度(i16)
3E	62	ACCEL_YOUT_L	
3F	63	ACCEL_ZOUT_H	Z轴加速度(i16)
40	64	ACCEL_ZOUT_L	
41	65	TEMP_OUT_H	温度 (i16)
42	66	TEMP_OUT_L	
43	67	GYRO_XOUT_H	X轴陀螺仪(i16)
44	68	GYRO_XOUT_L	
45	69	GYRO_YOUT_H	Y轴陀螺仪(i16)
46	70	GYRO_YOUT_L	
47	71	GYRO_ZOUT_H	Z轴陀螺仪(i16)
48	72	GYRO_ZOUT_L	

```
uint8_t buffer[14];
PAL_I2C_RegReadBytes(&hi2c,0xd0,0x3b,buffer,14);
/* 先将两个byte拼接成int16_t, 再进行换算 */
ax          = (int16_t)((buffer[0] << 8)  | buffer[1])  / 16384.0f;
ay          = (int16_t)((buffer[2] << 8)  | buffer[3])  / 16384.0f;
az          = (int16_t)((buffer[4] << 8)  | buffer[5])  / 16384.0f;
temperature = (int16_t)((buffer[6] << 8)  | buffer[7])  / 340.0f + 36.53f;
gx          = (int16_t)((buffer[8] << 8)  | buffer[9])  / 16.4f;
gy          = (int16_t)((buffer[10] << 8) | buffer[11]) / 16.4f;
gz          = (int16_t)((buffer[12] << 8) | buffer[13]) / 16.4f;
```

加速度换算关系：

AFS_SEL	Full Scale Range	LSB Sensitivity
0	±2*g*	16384 LSB/*g*
1	±4*g*	8192 LSB/*g*
2	±8*g*	4096 LSB/*g*
3	±16*g*	2048 LSB/*g*

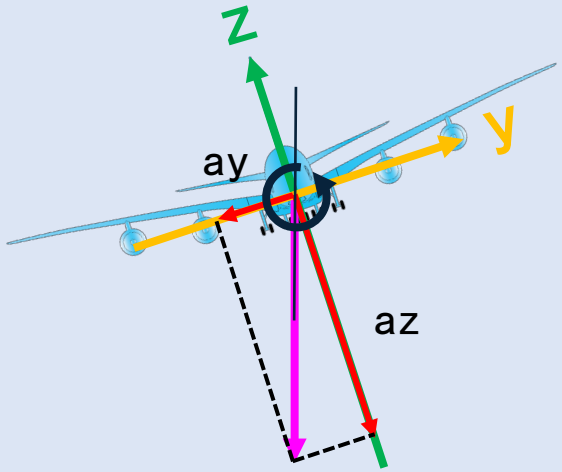
角速度换算关系：

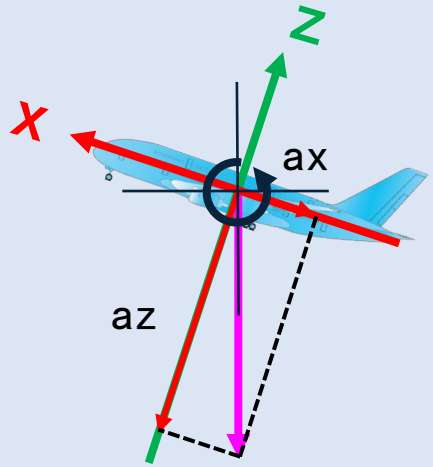
FS_SEL	Full Scale Range	LSB Sensitivity
0	± 250 °/s	131 LSB/°/s
1	± 500 °/s	65.5 LSB/°/s
2	± 1000 °/s	32.8 LSB/°/s
3	± 2000 °/s	16.4 LSB/°/s

温度换算关系：

$$温度(℃) = 原始值/340 + 36.53$$

4.4.3.2. 使用加速度计算欧拉角



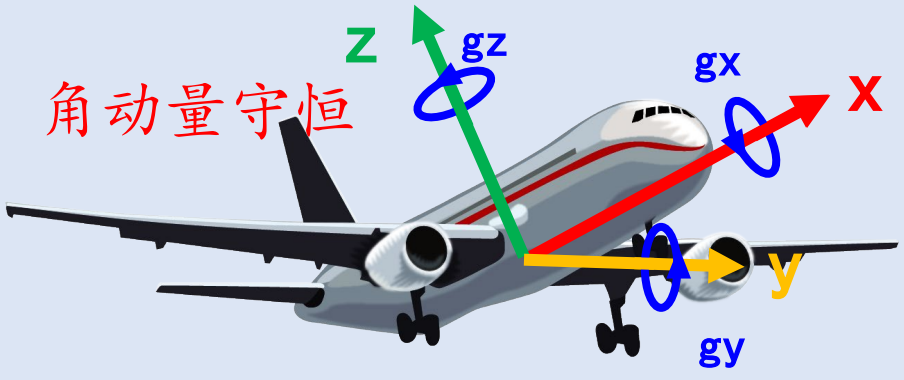


$roll = atan2(ay, az)$

$\text{pitch} = -atan2(ax, az)$

```
#include <math.h>
float roll_a  = atan2(ay,az) / 3.141593 * 180;   弧度转角度(Rad->Deg)
float pitch_a = -atan2(ax,az) / 3.141593 * 180;
```

4.4.3.3. 使用陀螺仪计算欧拉角

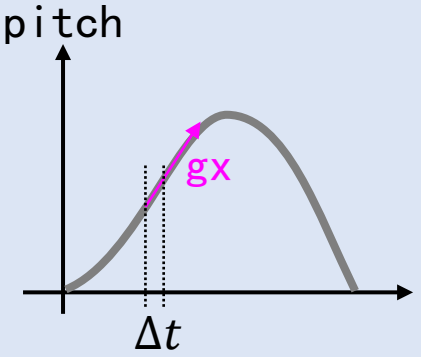


$\vec{g} = (gx, gy, gz)$

陀螺仪能够测量绕轴旋转的角速度，单位°/s

$$\begin{aligned} pitch(t) &= pitch(t-\Delta t) + gy * \Delta t \\ yaw(t) &= yaw(t-\Delta t) + gz * \Delta t \\ roll(t) &= roll(t-\Delta t) + gx * \Delta t \end{aligned}$$

新角度　　上次角度　　角速度　　时间




```
#include <math.h>
float yaw_g   =  yaw  + gz * dt;
float roll_g  =  roll + gx * dt;
float pitch_g = pitch + gy * dt;
```

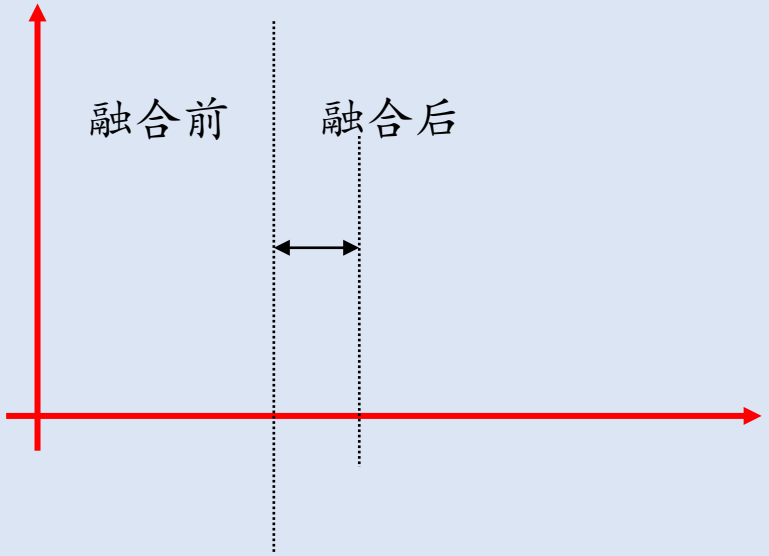
4.4.3.4. 融合

融合后的结果 = α*陀螺仪测量结果 + （1-α）*加速度计测量结果

陀螺仪权重　　加速度计权重

$yaw = yaw_g$　　$roll = \alpha * roll_g + (1-\alpha) * roll_a$　　$pitch = \alpha * pitch_g + (1-\alpha) * pitch_a$

$\alpha = \tau/(\tau + dt) = 0.1/(0.1 + 0.005) \approx 0.95238$




```
const float alpha = 0.95238;
yaw = yaw_g;
roll  = alpha * roll_g  + (1-alpha) * roll_a;
pitch = alpha * pitch_g + (1-alpha) * pitch_a;
```

4.4.4. 读取结果

铁头山羊 TIETOUSHANYANG

读取结果

```
static float ax,ay,az,temperature,gx,gy,gz,yaw,roll,pitch;
```

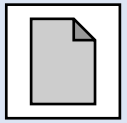
加速度 温度 陀螺仪 欧拉角

```
void App_MPU6050_GetResult(float *pAccelOut, float *pTempOut, float *pGyroOut, float *pEularAngleOut){
  pAccelOut[0] = ax;
  pAccelOut[1] = ay;
  pAccelOut[2] = az;
  *pTempOut = temperature;
  pGyroOut[0] = gx;
  pGyroOut[1] = gy;
  pGyroOut[2] = gz;
  pEularAngleOut[0] = yaw;
  pEularAngleOut[1] = roll;
  pEularAngleOut[2] = pitch;
}
```

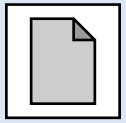
4.4.5. 联机调试

铁头山羊 TIETOUSHANYANG

4.4.5.1. 快速初始化



4.4.5.2. 代码编写



4.4.5.1. 快速初始化

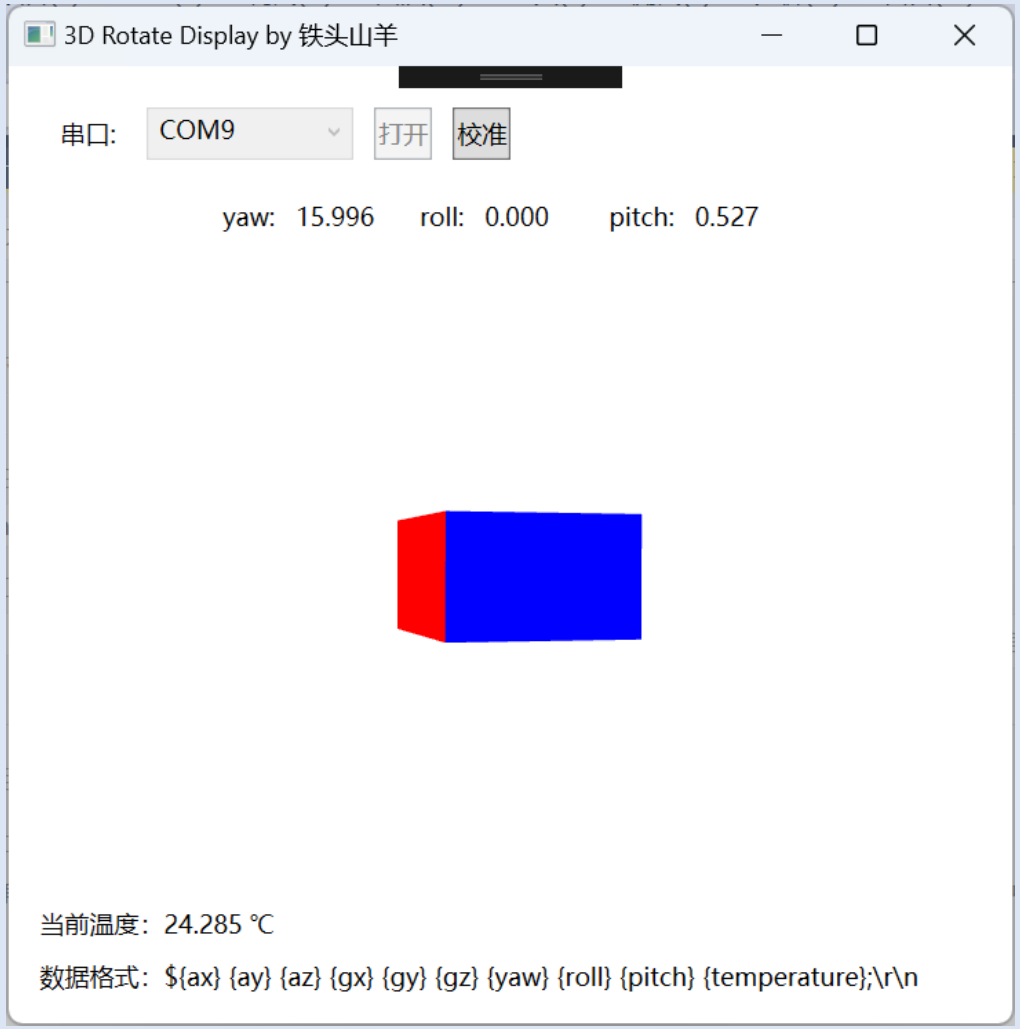
铁头山羊 TIETOUSHANYANG

```
hUSART1.Init.USARTx = USART1;
hUSART1.Init.BaudRate = 115200;
hUSART1.Init.USART_Parity = USART_Parity_No;
hUSART1.Init.USART_StopBits = USART_StopBits_1;
hUSART1.Init.USART_WordLength = USART_WordLength_8b;
hUSART1.Init.USART_IRQ_PreemptionPriority = 0;
hUSART1.Init.USART_IRQ_SubPriority = 0;
hUSART1.Init.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
hUSART1.Init.TxBufferSize = 128;
hUSART1.Init.RxBufferSize = 128;
hUSART1.Init.Advanced.LineSeparator = LineSeparator_Disable;
PAL_USART_Init(&hUSART1);
```

```
PAL_USART_InitHandle(&hUSART1);

hUSART1.Init.USARTx = USART1;
hUSART1.Init.BaudRate = 115200;

PAL_USART_Init(&hUSART1);
```

```
28  //
29  // @简介: 将句柄的初始化参数设置为默认值
30  //         USART_WordLength             = 8,      默认8位数据位
31  //         USART_StopBits               = 1,      默认1位停止位
32  //         USART_Parity                 = None,   默认不启用校验
33  //         USART_Mode                   = Tx|Rx,  默认收发双向
34  //         USART_IRQ_PreemptionPriority = 0,      默认抢占优先级=0
35  //         USART_IRQ_SubPriority        = 0,      默认子优先级=0
36  //         RxBufferSize                 = 128,    默认接收队列长度128
37  //         TxBufferSize                 = 128,    默认发送队列长度128
38  // @注意: 仍需要手动填写的项目
39  //         USARTx                       - 所使用的USART接口的名称
40  //         BaudRate                     - 波特率
41  //
```

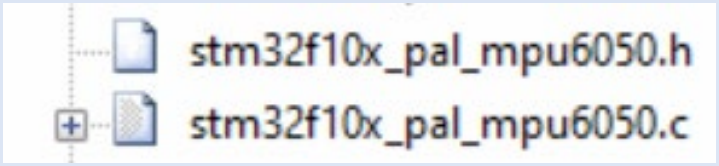
4.4.5.2. 代码编写




```c
#include “stm32f10x_usart.h”
#include “app_mpu6050.h”

static PalUSARTHandleTypeDef husart1;
static void USART_Proc(void);

int main(void){
  PAL_USART_InitHandle(&hUSART1);
  husart1.Init.USARTx = USART1;
  husart1.Init.BaudRate = 115200;
  PAL_USART_Init(&husart1);

  App_MPU6050_Init();

  while(1){
    App_MPU6050_Proc();
    USART_Proc();
  }
}

static void USART_Proc(void){
  PAL_PERIODIC_PROC(10);

  float accel[3],temperature,gyro[3],eular[3];
  App_MPU6050_GetResult(accel,&temperature,gyro,eluar);
  PAL_USART_Printf(&husart1,”$%.3f %.3f … %.3f;\r\n”, …);
}
```

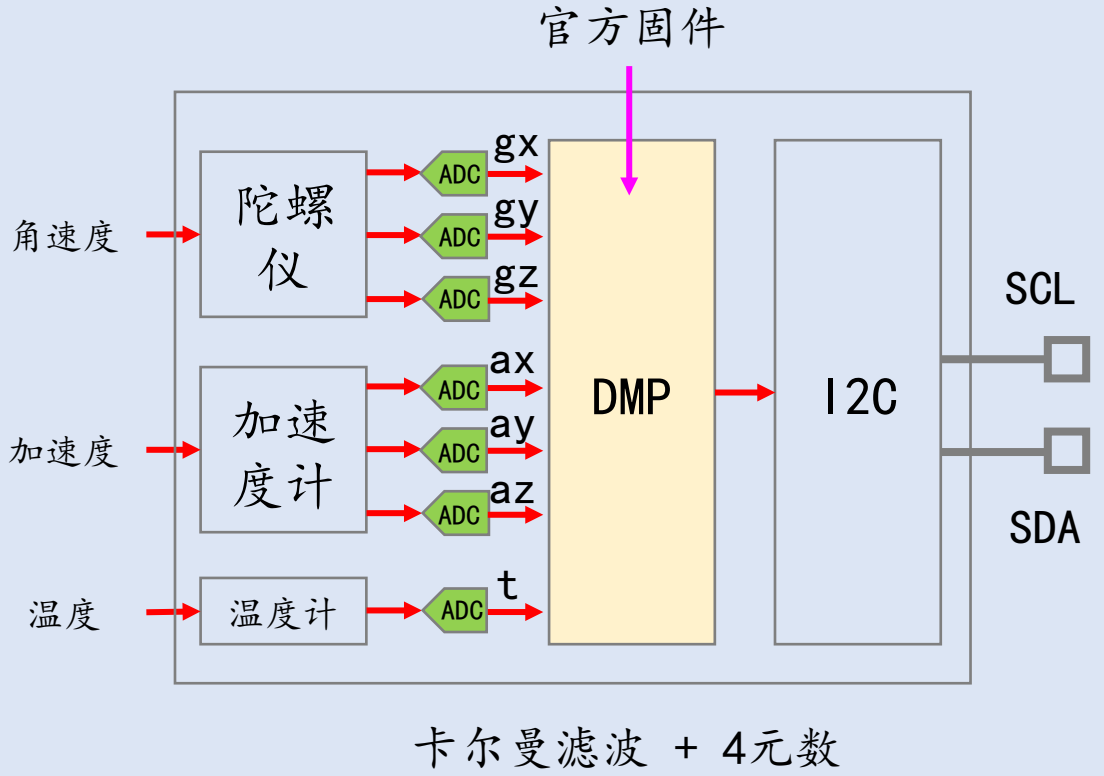
4.5. PAL库编程

4.5.1. PAL库解算原理



TDK InvenSense

eMPL



卡尔曼滤波 + 4元数

4.5.2. 编程接口

铁头山羊 TIETOUSHANYANG

```
void PAL_MPU6050_InitHandle(PalMPU6050_HandleTypeDef *Handle);
ErrorStatus PAL_MPU6050_Init(PalMPU6050_HandleTypeDef *Handle);
ErrorStatus PAL_MPU6050_Proc(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetAccelX(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetAccelY(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetAccelZ(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetGyroX(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetGyroY(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetGyroZ(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetYaw(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetRoll(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetPitch(PalMPU6050_HandleTypeDef *Handle);
float PAL_MPU6050_GetTemperature(PalMPU6050_HandleTypeDef *Handle);
```

设置默认初始化参数

初始化

进程函数